1. Introduction. This program converts an AWEB file to a TEX file. It was written (initially in PASCAL) by D. E. Knuth in October, 1981; a somewhat similar SAIL program had been developed in March, 1979, although the earlier program used a top-down parsing method that is quite different from the present scheme.

The conversion from PASCAL to ADA has been done by U. Schweigert in May, 1988.

The "banner line" defined here should be changed whenever AWEAVE is modified.

define *banner* ≡ "This␣is␣AWEAVE,␣Version␣1.1␣(NRaD␣Version)"

2. The program consists of several packages that are declared right now; each of these packages and both the specification and the body of the packages are send to a seperate file. The main program itself is declared later.

3* The first package we declare is called *int_number_io* and enables us to output integer numbers. Further we declare a package *data_structures*, which will contain all the global constants, types, variables, and some procedures used throughout the program; it will (nearly) contain code of the sections "The character set" and "Data structures."

```
output to file data_structures.ads
with TEXT_IO;
with int_number_io;
with TEXT_IO;
package data_structures is
  ⟨ Types and objects visible from data_structures 14 ⟩
  ⟨ Specification of procedures visible from data_structures 48 ⟩
  procedure initialize;
end data_structures;
output to file data_structures.adb
with input_output; use input_output;
package body data_structures is
  ⟨ Procedures in data_structures 49 ⟩
  procedure initialize is
  begin
    ⟨ Set initial values 19 ⟩
  end initialize;
end data_structures;
```

4* The following package, called *input_output*, is responsible for the input/output features needed by AWEAVE, and for the error handling. It will contain code of sections "Input and output" and "Reporting errors to the user."

```
output to file input_output.ads
with TEXT_IO;
with data_structures; use data_structures;
package input_output is
  ⟨ Specification of procedures visible from input_output 27 ⟩
end input_output;
output to file input_output.adb
with TEXT_IO, int_number_io;
package body input_output is
  ⟨ Procedures in input_output 28 ⟩
end input_output;
```

5*. Additionally we have a package *searching*, which contains the code of sections "Searching for identifiers" and "Searching for module names."

```
output to file searching.ads
with data_structures; use data_structures;
package searching is
  ⟨ Specification of procedures visible from searching 60 ⟩
end searching;
output to file searching.adb
with TEXT_IO;
with input_output; use input_output;
package body searching is
  ⟨ Procedures in searching 61 ⟩
end searching;
```

6*. The first of the major packages will manage the input process of `AWEAVE` and contains code of sections "Lexical Scanning" and "Inputting the next token."

```
output to file lexical_scanning.ads
with data_structures; use data_structures;
package lexical_scanning is
  ⟨ Specification of procedures visible from lexical_scanning 92 ⟩
end lexical_scanning;
output to file lexical_scanning.adb
with TEXT_IO, int_number_io;
with input_output; use input_output;
with searching; use searching;
package body lexical_scanning is
  ⟨ Procedures in lexical_scanning 85 ⟩
end lexical_scanning;
```

7*. The next package is responsible for the first phase of `AWEAVE` and contains code of section "Phase one processing."

```
output to file first_phase.ads
with data_structures; use data_structures;
package first_phase is
  ⟨ Specification of procedures visible from first_phase 128 ⟩
end first_phase;
output to file first_phase.adb
with TEXT_IO, int_number_io;
with input_output; use input_output;
with searching; use searching;
with lexical_scanning; use lexical_scanning;
package body first_phase is
  ⟨ Procedures in first_phase 129 ⟩
end first_phase;
```

8.* This package provides the output features needed in the second and third phase of `AWEAVE` and contains code of sections "Low-level output routines" and "Routines that copy TEX material."

output to file `output.ads`

with *data_structures*; **use** *data_structures*;
package *output* **is**
  ⟨ Specification of procedures visible from *output* 140 ⟩
end *output*;

output to file `output.adb`

with TEXT_IO, *int_number_io*;
with *input_output*; **use** *input_output*;
with *lexical_scanning*; **use** *lexical_scanning*;
package body *output* **is**
  ⟨ Procedures in *output* 141 ⟩
end *output*;

9.* Here's a package that handles the parsing of ADA-texts containing code of sections "Parsing," "Implementing the productions," and "Initializing the scraps."

output to file `parsing.ads`

with *data_structures*; **use** *data_structures*;
package *parsing* **is**
  ⟨ Specification of procedures visible from *parsing* 217 ⟩
end *parsing*;

output to file `parsing.adb`

with TEXT_IO, *int_number_io*;
with *input_output*; **use** *input_output*;
with *searching*; **use** *searching*;
with *lexical_scanning*; **use** *lexical_scanning*;
with *output*; **use** *output*;
package body *parsing* **is**
  ⟨ Procedures in *parsing* 212 ⟩
end *parsing*;

10.* Now there's a package for handling phase two of `AWEAVE` containing code of sections "Output of tokens" and "Phase two processing."

output to file `second_phase.ads`

with *data_structures*; **use** *data_structures*;
package *second_phase* **is**
  ⟨ Specification of procedures visible from *second_phase* 244 ⟩
end *second_phase*;

output to file `second_phase.adb`

with TEXT_IO, *int_number_io*;
with *input_output*; **use** *input_output*;
with *searching*; **use** *searching*;
with *lexical_scanning*; **use** *lexical_scanning*;
with *output*; **use** *output*;
with *parsing*; **use** *parsing*;
package body *second_phase* **is**
  ⟨ Procedures in *second_phase* 245 ⟩
end *second_phase*;

11.* Finally the last package handles the third phase of AWEAVE by means of "Phase three processing."

output to file `third_phase.ads`

with *data_structures*; **use** *data_structures*;
package *third_phase* **is**
⟨ Specification of procedures visible from *third_phase* 298 ⟩
end *third_phase*;

output to file `third_phase.adb`

with TEXT_IO, *int_number_io*;
with *input_output*; **use** *input_output*;
with *output*; **use** *output*;
with *second_phase*; **use** *second_phase*;
package body *third_phase* **is**
⟨ Procedures in *third_phase* 291 ⟩
end *third_phase*;

12. Some of this code is optional for use when statistics about AWEAVE's memory usage are desired; such material is enclosed between the delimiters **stat** and **tats**. Other material is optional when a somewhat optimal code is wanted; such is delimited by **optim** and **mitpo**.

define *stat* ≡ `@{` { change this to '*stat* ≡ ' when gathering usage statistics }
define *tats* ≡ `@}` { change this to '*tats* ≡ ' when gathering usage statistics }
format *stat* ≡ *declare*
format *tats* ≡ *end*
define *optim* ≡ `@{` { change this to '*optim* ≡ ' when optimal code is desired }
define *mitpo* ≡ `@}` { change this to '*mitpo* ≡ ' when optimal code is desired }
format *optim* ≡ *declare*
format *mitpo* ≡ *end*

13. Here are some macros for common programming idioms.

define *incr*(#) ≡ # ← # + 1 { increase a variable by unity }
define *decr*(#) ≡ # ← # − 1 { decrease a variable by unity }

14. The following parameters are set big enough to handle TEX, so they should be sufficient for most applications of AWEAVE.

⟨ Types and objects visible from *data_structures* 14 ⟩ ≡
max_bytes : **constant** ← 45_000; { 1/*ww* times the number of bytes in identifiers, index entries, and module names; must be less than 65_536 }
max_names : **constant** ← 5_000;
{ number of identifiers, index entries, and module names; must be less than 10_240 }
max_modules : **constant** ← 2_000; { greater than the total number of modules }
hash_size : **constant** ← 353; { should be prime }
buf_size : **constant** ← 100; { maximum length of input line }
longest_name : **constant** ← 400; { module names shouldn't be longer than this }
long_buf_size : **constant** ← 500; { *buf_size* + *longest_name* }
line_length : **constant** ← 80;
{ lines of TEX output have at most this many characters, should be less than 256 }
max_refs : **constant** ← 60_000; { number of cross references; must be less than 65_536 }
max_toks : **constant** ← 60_000;
{ number of symbols in ADA texts being parsed; must be less than 65_536 }
max_texts : **constant** ← 10_000;
{ number of phrases in ADA texts being parsed; must be less than 10_240 }
max_scraps : **constant** ← 8_000; { number of tokens in ADA texts being parsed }
stack_size : **constant** ← 800; { number of simultaneous output levels }

See also sections 15, 16, 17, 18, 20, 26, 29, 31, 34, 38, 41, 42, 43, 45, 46, 50, 51, 56, 57, 59, 66, 73, 74, 82, 84, 99, 109, 125, 131, 136, 139, 150, 167, 168, 171, 172, 243, 247, 263, 271, 276, 283, 285, 287, 288, 290, and 307.

This code is used in section 3*.

15. A global variable called *history* will contain one of four values at the end of every run: *spotless* means that no unusual messages were printed; *harmless_message* means that a message of possible interest was printed but no serious errors were detected; *error_message* means that at least one error was found; *fatal_message* means that the program terminated abnormally. The value of *history* does not influence the behavior of the program; it is simply computed for the convenience of systems that might want to use such information.

define *mark_harmless* ≡
if *history* = *spotless* **then**
history ← *harmless_message*;
end if
define *mark_error* ≡ *history* ← *error_message*
define *mark_fatal* ≡ *history* ← *fatal_message*

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
type *history_type* **is** (*spotless*, *harmless_message*, *error_message*, *fatal_message*);
history : *history_type* ← *spotless*; { how bad was this run? }

16. The character set. One of the main goals in the design of AWEB has been to make it readily portable between a wide variety of computers. Yet AWEB by its very nature must use a greater variety of characters than most computer programs deal with, and character encoding is one of the areas in which existing machines differ most widely from each other.

To resolve this problem, all input to AWEAVE and ATANGLE is converted to an internal seven-bit code that is essentially standard ASCII, the "American Standard Code for Information Interchange." The conversion is done immediately when each character is read in. Conversely, characters are converted from ASCII to the user's external representation just before they are output.

Such an internal code is relevant to users of AWEB only because it is the code used for preprocessed constants like @´A´. If you are writing a program in AWEB that makes use of such one-character constants, you should convert your input to ASCII form, like AWEAVE and ATANGLE do. Otherwise AWEB's internal coding scheme does not affect you.

Here is a table of the standard visible ASCII codes:

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*
040_8	␣	!	"	#	$	%	&	'
050_8	(	)	*	+	,	-	.	/
060_8	0	1	2	3	4	5	6	7
070_8	8	9	:	;	<	=	>	?
100_8	@	A	B	C	D	E	F	G
110_8	H	I	J	K	L	M	N	O
120_8	P	Q	R	S	T	U	V	W
130_8	X	Y	Z	[	\	]	^	_
140_8	`	a	b	c	d	e	f	g
150_8	h	i	j	k	l	m	n	o
160_8	p	q	r	s	t	u	v	w
170_8	x	y	z	{	|	}	~	

(Actually, of course, code 040_8 is an invisible blank space.) Code 136_8 was once an upward arrow (↑), and code 137_8 was once a left arrow (←), in olden times when the first draft of ASCII code was prepared; but AWEB works with today's standard ASCII in which those codes represent circumflex and underline as shown.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
subtype *ASCII_code* **is** INTEGER **range** 0 . . 127; { seven-bit numbers, a subrange of the integers }

17. We shall use the name *text_char* to stand for the data type of the characters in the input and output files. We shall also assume that *text_char* consists of the elements *text_char*′FIRST through *text_char*′LAST, inclusive. The following definition should be adjusted if necessary.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
subtype *text_char* **is** CHARACTER; { the data type of characters in text files }
subtype *text_file* **is** TEXT_IO.FILE_TYPE; { file type consisting of *text_char* }

18. The AWEAVE and ATANGLE processors convert between ASCII code and the user's external character set by means of arrays *xord* and *xchr* that are analogous to ADA's POS and VAL attributes.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
xord : **array** (*text_char*) **of** *ASCII_code*; { specifies conversion of input characters }
xchr : **array** (*ASCII_code*) **of** *text_char*; { specifies conversion of output characters }

19. If we assume that every system using AWEB is able to read and write the visible characters of standard ASCII (although not necessarily using the ASCII codes to represent them), the following assignment statements initialize most of the *xchr* array properly, without needing any system-dependent changes. For example, the statement `xchr(8#101#):=´A´;` that appears in the present AWEB file might be encoded in, say, EBCDIC code on the external medium on which it resides, but ATANGLE will convert from this external code to ASCII and back again. Therefore the assignment statement `XCHR(8#101#):=´A´;` will appear in the corresponding ADA file, and ADA will compile this statement so that $xchr(101_8)$ receives the character `A` in the external (*text_char*) code. Note that it would be quite incorrect to say `xchr(8#101#):=@´A´;`, because [`A`] is a constant of type INTEGER, not CHARACTER, and because we have [`A`] = 65 regardless of the external character set.

⟨ Set initial values 19 ⟩ ≡

$xchr(40_8) \leftarrow$ ´␣´; $xchr(41_8) \leftarrow$ ´!´; $xchr(42_8) \leftarrow$ ´"´; $xchr(43_8) \leftarrow$ ´#´; $xchr(44_8) \leftarrow$ ´$´;
$xchr(45_8) \leftarrow$ ´%´; $xchr(46_8) \leftarrow$ ´&´; $xchr(47_8) \leftarrow$ ´´´;
$xchr(50_8) \leftarrow$ ´(´; $xchr(51_8) \leftarrow$ ´)´; $xchr(52_8) \leftarrow$ ´*´; $xchr(53_8) \leftarrow$ ´+´; $xchr(54_8) \leftarrow$ ´,´;
$xchr(55_8) \leftarrow$ ´-´; $xchr(56_8) \leftarrow$ ´.´; $xchr(57_8) \leftarrow$ ´/´;
$xchr(60_8) \leftarrow$ ´0´; $xchr(61_8) \leftarrow$ ´1´; $xchr(62_8) \leftarrow$ ´2´; $xchr(63_8) \leftarrow$ ´3´; $xchr(64_8) \leftarrow$ ´4´;
$xchr(65_8) \leftarrow$ ´5´; $xchr(66_8) \leftarrow$ ´6´; $xchr(67_8) \leftarrow$ ´7´;
$xchr(70_8) \leftarrow$ ´8´; $xchr(71_8) \leftarrow$ ´9´; $xchr(72_8) \leftarrow$ ´:´; $xchr(73_8) \leftarrow$ ´;´; $xchr(74_8) \leftarrow$ ´<´;
$xchr(75_8) \leftarrow$ ´=´; $xchr(76_8) \leftarrow$ ´>´; $xchr(77_8) \leftarrow$ ´?´;
$xchr(100_8) \leftarrow$ ´@´; $xchr(101_8) \leftarrow$ ´A´; $xchr(102_8) \leftarrow$ ´B´; $xchr(103_8) \leftarrow$ ´C´; $xchr(104_8) \leftarrow$ ´D´;
$xchr(105_8) \leftarrow$ ´E´; $xchr(106_8) \leftarrow$ ´F´; $xchr(107_8) \leftarrow$ ´G´;
$xchr(110_8) \leftarrow$ ´H´; $xchr(111_8) \leftarrow$ ´I´; $xchr(112_8) \leftarrow$ ´J´; $xchr(113_8) \leftarrow$ ´K´; $xchr(114_8) \leftarrow$ ´L´;
$xchr(115_8) \leftarrow$ ´M´; $xchr(116_8) \leftarrow$ ´N´; $xchr(117_8) \leftarrow$ ´O´;
$xchr(120_8) \leftarrow$ ´P´; $xchr(121_8) \leftarrow$ ´Q´; $xchr(122_8) \leftarrow$ ´R´; $xchr(123_8) \leftarrow$ ´S´; $xchr(124_8) \leftarrow$ ´T´;
$xchr(125_8) \leftarrow$ ´U´; $xchr(126_8) \leftarrow$ ´V´; $xchr(127_8) \leftarrow$ ´W´;
$xchr(130_8) \leftarrow$ ´X´; $xchr(131_8) \leftarrow$ ´Y´; $xchr(132_8) \leftarrow$ ´Z´; $xchr(133_8) \leftarrow$ ´[´; $xchr(134_8) \leftarrow$ ´\´;
$xchr(135_8) \leftarrow$ ´]´; $xchr(136_8) \leftarrow$ ´^´; $xchr(137_8) \leftarrow$ ´_´;
$xchr(140_8) \leftarrow$ ´`´; $xchr(141_8) \leftarrow$ ´a´; $xchr(142_8) \leftarrow$ ´b´; $xchr(143_8) \leftarrow$ ´c´; $xchr(144_8) \leftarrow$ ´d´;
$xchr(145_8) \leftarrow$ ´e´; $xchr(146_8) \leftarrow$ ´f´; $xchr(147_8) \leftarrow$ ´g´;
$xchr(150_8) \leftarrow$ ´h´; $xchr(151_8) \leftarrow$ ´i´; $xchr(152_8) \leftarrow$ ´j´; $xchr(153_8) \leftarrow$ ´k´; $xchr(154_8) \leftarrow$ ´l´;
$xchr(155_8) \leftarrow$ ´m´; $xchr(156_8) \leftarrow$ ´n´; $xchr(157_8) \leftarrow$ ´o´;
$xchr(160_8) \leftarrow$ ´p´; $xchr(161_8) \leftarrow$ ´q´; $xchr(162_8) \leftarrow$ ´r´; $xchr(163_8) \leftarrow$ ´s´; $xchr(164_8) \leftarrow$ ´t´;
$xchr(165_8) \leftarrow$ ´u´; $xchr(166_8) \leftarrow$ ´v´; $xchr(167_8) \leftarrow$ ´w´;
$xchr(170_8) \leftarrow$ ´x´; $xchr(171_8) \leftarrow$ ´y´; $xchr(172_8) \leftarrow$ ´z´; $xchr(173_8) \leftarrow$ ´{´; $xchr(174_8) \leftarrow$ ´|´;
$xchr(175_8) \leftarrow$ ´}´; $xchr(176_8) \leftarrow$ ´~´;
$xchr(0) \leftarrow$ ´␣´; $xchr(177_8) \leftarrow$ ´␣´; { these ASCII codes are not used }

See also sections 21, 22, 44, 47, 52, 58, 117, 144, 146, and 289.

This code is used in section 3*.

20. Some of the ASCII codes below 40_8 have been given symbolic names in AWEAVE and ATANGLE because they are used with a special meaning.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
and_sign : **constant** *ASCII_code* ← 4_8; { equivalent to '`and`' }
not_sign : **constant** *ASCII_code* ← 5_8; { equivalent to '`not`' }
set_element_sign : **constant** *ASCII_code* ← 6_8; { equivalent to '`in`' }
tab_mark : **constant** *ASCII_code* ← 11_8; { ASCII code used as tab-skip }
line_feed : **constant** *ASCII_code* ← 12_8; { ASCII code thrown away at end of line }
form_feed : **constant** *ASCII_code* ← 14_8; { ASCII code used at end of page }
carriage_return : **constant** *ASCII_code* ← 15_8; { ASCII code used at end of line }
right_arrow : **constant** *ASCII_code* ← 21_8; { equivalent to '`=>`' }
box_sign : **constant** *ASCII_code* ← 22_8; { equivalent to '`<>`' }
double_star : **constant** *ASCII_code* ← 23_8; { equivalent to '`**`' }
left_label_bracket : **constant** *ASCII_code* ← 24_8; { equivalent to '`<<`' }
right_label_bracket : **constant** *ASCII_code* ← 25_8; { equivalent to '`>>`' }
left_arrow : **constant** *ASCII_code* ← 30_8; { equivalent to '`:=`' }
not_equal : **constant** *ASCII_code* ← 32_8; { equivalent to '`/=`' }
less_or_equal : **constant** *ASCII_code* ← 34_8; { equivalent to '`<=`' }
greater_or_equal : **constant** *ASCII_code* ← 35_8; { equivalent to '`>=`' }
equivalence_sign : **constant** *ASCII_code* ← 36_8; { equivalent to '`==`' }
or_sign : **constant** *ASCII_code* ← 37_8; { equivalent to '`or`' }

21. Here now is the system-dependent part of the character set. If AWEB is being implemented on a garden-variety ADA for which only standard ASCII codes will appear in the input and output files, you don't need to make any changes here.

Changes to the present module will make AWEB more friendly on computers that have an extended character set, so that one can type things like ≠ instead of `/=`. If you have an extended set of characters that are easily incorporated into text files, you can assign codes arbitrarily here, giving an *xchr* equivalent to whatever characters the users of AWEB are allowed to have in their input files, provided that unsuitable characters do not correspond to special codes like *carriage_return* that are listed above.

(The present file AWEAVE.AWEB does not contain any of the non-ASCII characters, because it is intended to be used with all implementations of AWEB.)

⟨ Set initial values 19 ⟩ +≡
xchr(1 . . 37_8) ← (**others** ⇒ ´␣´);

22. The following system-independent code makes the *xord* array contain a suitable inverse to the information in *xchr*.

⟨ Set initial values 19 ⟩ +≡
xord ← (**others** ⇒ 40_8);
for $i \in 1 \mathinner{..} 176_8$ **loop**
  xord(*xchr*(i)) ← i;
end loop;

23. Input and output. The input conventions of this program are identical to those of ATANGLE. Therefore people who need to make modifications to both systems should be able to do so without too many headaches.

24. Terminal output is done by writing on file STANDARD_OUTPUT, which is assumed to consist of characters of type *text_char*:

```
define new_ln ≡ TEXT_IO.NEW_LINE(TEXT_IO.STANDARD_OUTPUT)
define print(#) ≡ TEXT_IO.PUT(TEXT_IO.STANDARD_OUTPUT, #)   { 'print' means write on the terminal }
define print_ln(#) ≡ print(#); new_ln   { 'print' and then start new line }
define print_nl(#) ≡ new_ln; print(#)   { print information starting on a new line }
define print_int(#) ≡ int_number_io.PUT(TEXT_IO.STANDARD_OUTPUT, #)   { print out an integer value }
```

25. The *update_terminal* procedure is called when we want to make sure that everything we have output to the terminal so far has actually left the computer's internal buffers and been sent. (In the ADA-system used to develop the initial program this is not required.)

```
define update_terminal ≡ null   { empty the terminal output buffer }
```

26. The main input comes from *aweb_file*; this input may be overridden by changes in *change_file*. (If *change_file* is empty, there are no changes.)

```
⟨ Types and objects visible from data_structures 14 ⟩ +≡
  aweb_file : text_file;   { primary input }
  change_file : text_file;   { updates }
```

27.

```
⟨ Specification of procedures visible from input_output 27 ⟩ ≡
  procedure open_a_file(f : in out text_file; mode_of_file : TEXT_IO.FILE_MODE; default_name : STRING;
        success : out BOOLEAN);
  function create_TeX_file(TeX_file_name : STRING)return BOOLEAN;
```

See also sections 32 and 35.

This code is used in section 4*.

28. The following code opens a file and returns TRUE if succesfully. If the file is already open the procedure has no effect; this gives the ability to open the files by means of the change file, without changing the AWEB-file of AWEAVE.

```
⟨ Procedures in input_output 28 ⟩ ≡
  procedure open_a_file(f : in out text_file; mode_of_file : TEXT_IO.FILE_MODE; default_name : STRING;
        success : out BOOLEAN) is
  begin
    if TEXT_IO.IS_OPEN(f) then
      success ← TRUE;
    else
      TEXT_IO.OPEN(FILE ⇒ f, MODE ⇒ mode_of_file, NAME ⇒ default_name); success ← TRUE;
    end if;
  exception
    when TEXT_IO.STATUS_ERROR | TEXT_IO.NAME_ERROR | TEXT_IO.USE_ERROR ⇒
      success ← FALSE; return;
  end open_a_file;
```

See also sections 30, 33, and 36.

This code is used in section 4*.

29. The output goes to *TeX_file*.

```
⟨ Types and objects visible from data_structures 14 ⟩ +≡
  TeX_file : text_file;
```

30. The following code creates *TeX_file*.

```
⟨ Procedures in input_output 28 ⟩ +≡
  function create_TeX_file(TeX_file_name : STRING)return BOOLEAN is
  begin
    if TEXT_IO.IS_OPEN(TeX_file) then
      if TEXT_IO.NAME(TeX_file) = TeX_file_name then
        return TRUE;
      else
        return FALSE;
      end if;
    else
      TEXT_IO.CREATE(FILE ⇒ TeX_file, MODE ⇒ TEXT_IO.OUT_FILE, NAME ⇒ TeX_file_name);
      return TRUE;
    end if;
  exception
    when TEXT_IO.STATUS_ERROR | TEXT_IO.NAME_ERROR | TEXT_IO.USE_ERROR ⇒
      return FALSE;
  end create_TeX_file;
```

31. Input goes into an array called *buffer*.

```
⟨ Types and objects visible from data_structures 14 ⟩ +≡
  type buffer_type is array (INTEGER range <>) of ASCII_code;
  subtype long_buffer_index is INTEGER range 0 .. long_buf_size;
  subtype buffer_index is long_buffer_index range 0 .. buf_size;
  buffer : buffer_type(buffer_index);
```

32. The *input_ln* procedure brings the next line of input from the specified file into the *buffer* array and returns the value TRUE, unless the file has already been entirely read, in which case it returns FALSE. The conventions of TEX are followed; i.e., *ASCII_code* numbers representing the next line of the file are input into *buffer*(0 .. *limit* − 1); trailing blanks are ignored; and the global variable *limit* is set to the length of the line. The value of *limit* must be strictly less than *buf_size*.

We assume that none of the *ASCII_code* values of *buffer*(j) for $0 \le j < limit$ is equal to 0, 177_8, *line_feed*, *form_feed*, or *carriage_return*.

```
⟨ Specification of procedures visible from input_output 27 ⟩ +≡
  function input_ln(f : text_file)return BOOLEAN;
```

33. ⟨ Procedures in *input_output* 28 ⟩ +≡

```
function input_ln(f : text_file)return BOOLEAN is    { inputs a line or returns FALSE }
  final_limit : buffer_index ← 0;   { limit without trailing blanks }
  c : text_char;
begin
  limit ← 0;
  if ¬TEXT_IO.IS_OPEN(f) or else TEXT_IO.END_OF_FILE(f) then
    return FALSE;
  else
    while ¬TEXT_IO.END_OF_LINE(f) loop
      TEXT_IO.GET(f, c); buffer(limit) ← xord(c); incr(limit);
      if buffer(limit − 1) ≠ [␣] then
        final_limit ← limit;
      end if;
      if limit = buf_size then
        TEXT_IO.SKIP_LINE(f); decr(limit);   { keep buffer(buf_size) empty }
        print_nl("!␣Input␣line␣too␣long"); loc ← 0; error; return TRUE;
      end if;
    end loop;
    TEXT_IO.SKIP_LINE(f); limit ← final_limit; return TRUE;
  end if;
end input_ln;
```

34. Reporting errors to the user. The AWEAVE processor operates in three phases: first it inputs the source file and stores cross-reference data, then it inputs the source once again and produces the TEX output file, and finally it sorts and outputs the index.

The global variables *phase_one* and *phase_three* tell which Phase we are in.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
  phase_one : BOOLEAN; { TRUE in Phase I, FALSE in Phases II and III }
  phase_three : BOOLEAN; { TRUE in Phase III, FALSE in Phases I and II }

35. The command '*err_print*("!␣Error␣message")' will report a syntax error to the user, by printing the error message at the beginning of a new line and then giving an indication of where the error was spotted in the source file. Note that no period follows the error message, since the error routine will automatically supply a period.

The actual error indications are provided by a procedure called *error*. However, error messages are not actually reported during phase one, since errors detected on the first pass will be detected again during the second.

define *err_print*(#) ≡
    if ¬*phase_one* **then**
      print_nl(#); *error*;
    end if

⟨ Specification of procedures visible from *input_output* 27 ⟩ +≡
  procedure *error*;

36. ⟨ Procedures in *input_output* 28 ⟩ +≡
  procedure *error* **is** { prints " . " and location of error message }
    l : *buffer_index*; { index into *buffer* }
  begin
    ⟨ Print error location based on input buffer 37 ⟩;
    update_terminal; *mark_error*;
  end *error*;

37. The error locations during Phase I can be indicated by using the global variables *loc*, *line*, and *changing*, which tell respectively the first unlooked-at position in *buffer*, the current line number, and whether or not the current line is from *change_file* or *aweb_file*. This routine should be modified on systems whose standard text editor has special line-numbering conventions.

```
⟨ Print error location based on input buffer 37 ⟩ ≡
  if changing then
    print(".␣(change␣file␣");
  else
    print(".␣(");
  end if;
  print("l."); print_int(line, 1); print_ln(")");
  if loc ≥ limit then
    l ← limit;
  else
    l ← loc;
  end if;
  for k ∈ 1 .. l loop
    if buffer(k − 1) = tab_mark then
      print("␣");
    else
      print(xchr(buffer(k − 1)));   { print the characters already read }
    end if;
  end loop;
  new_ln;
  for k ∈ 1 .. l loop
    print("␣");
  end loop;   { space out the next line }
  for k ∈ l + 1 .. limit loop
    print(xchr(buffer(k − 1)));
  end loop;   { print the part not yet read }
  if buffer(limit) = [|] then
    print(xchr([|]));
  end if;   { end of ADA text in module names }
  print("␣")   { this space separates the message from future asterisks }
```

This code is used in section 36.

38. The *jump_out* macro just raises the *end_of_AWEAVE* exception. It is used when no recovery from a particular error has been provided.

```
define jump_out ≡ raise end_of_AWEAVE
define fatal_error(#) ≡ print_nl(#); error; mark_fatal; jump_out
```

```
⟨ Types and objects visible from data_structures 14 ⟩ +≡
  end_of_AWEAVE : exception;
```

39. Sometimes the program's behavior is far different from what it should be, and `AWEAVE` prints an error message that is really for the `AWEAVE` maintenance person, not the user. In such cases the program says *confusion*("indication␣of␣where␣we␣are").

```
define confusion(#) ≡ fatal_error("!␣This␣can´t␣happen␣(" & # & ")")
```

40. An overflow stop occurs if `AWEAVE`'s tables aren't large enough.

```
define overflow(#) ≡ fatal_error("!␣Sorry,␣" & # & "␣capacity␣exceeded")
```

41. Data structures. During the first phase of its processing, AWEAVE puts identifier names, index entries, and module names into the large *byte_mem* array, which is packed with seven-bit integers. Allocation is sequential, since names are never deleted.

An auxiliary array *byte_start* is used as a directory for *byte_mem*, and the *link*, *ilk*, and *xref* arrays give further information about names. These auxiliary arrays consist of sixteen-bit items.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
subtype *eight_bits* **is** INTEGER **range** 0 . . 255; { unsigned one-byte quantity }
subtype *sixteen_bits* **is** INTEGER **range** 0 . . 65_535; { unsigned two-byte quantity }

42. AWEAVE has been designed to avoid the need for indices that are more than sixteen bits wide, so that it can be used on most computers. But there are programs that need more than 65_536 bytes; TEX is one of these. To get around this problem, a slight complication has been added to the data structures: *byte_mem* is a two-dimensional array, whose first index is either 0 or 1. (For generality, the first index is actually allowed to run between 0 and $ww - 1$, where ww is defined to be 2; the program will work for any positive value of ww, and it can be simplified in obvious ways if $ww = 1$.)

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
ww : **constant** INTEGER ← 2; { we multiply the byte capacity by approximately this amount }
subtype *w_range* **is** INTEGER **range** 0 . . *ww* − 1;
subtype *byte_index* **is** INTEGER **range** 0 . . *max_bytes*;
subtype *name_index* **is** INTEGER **range** 0 . . *max_names*;

type *byte_memory* **is array** (*w_range*, *byte_index*) **of** *ASCII_code*;
optim
  pragma PACK(*byte_memory*);
mitpo

type *name_info* **is array** (*name_index*) **of** *sixteen_bits*;

byte_mem : *byte_memory*; { characters of names }
byte_start : *name_info*; { directory into *byte_mem* }
link : *name_info*; { hash table or tree links }
ilk : *name_info*; { type codes or tree links }
xref : *name_info*; { heads of cross-reference lists }

43. The names of identifiers are found by computing a hash address h and then looking at strings of bytes signified by *hash*(h), *link*(*hash*(h)), *link*(*link*(*hash*(h))), . . . , until either finding the desired name or encountering a zero.

A '*name_pointer*' variable, which signifies a name, is an index into *byte_start*. The actual sequence of characters in the name pointed to by p appears in positions *byte_start*(p) to *byte_start*($p + ww$) − 1, inclusive, in the segment of *byte_mem* whose first index is p **mod** ww. Thus, when $ww = 2$ the even-numbered name bytes appear in *byte_mem*(0, ∗) and the odd-numbered ones appear in *byte_mem*(1, ∗). The pointer 0 is used for undefined module names; we don't want to use it for the names of identifiers, since 0 stands for a null pointer in a linked list.

We usually have *byte_start*(*name_ptr* + w) = *byte_ptr*((*name_ptr* + w) **mod** ww) for $0 \le w < ww$, since these are the starting positions for the next ww names to be stored in *byte_mem*.

define *length*(#) ≡ *byte_start*(# + *ww*) − *byte_start*(#) { the length of a name }
define *null_array* ≡ (**others** ⇒ 0) { all elements have value 0 }

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
subtype *name_pointer* **is** *name_index*; { identifies a name }

name_ptr : *name_pointer* ← 1; { first unused position in *byte_start* }
byte_ptr : **array** (*w_range*) **of** *byte_index* ← *null_array*; { first unused position in *byte_mem* }

44. ⟨ Set initial values 19 ⟩ +≡
$byte_start(0 \mathinner{..} ww) \leftarrow null_array$;

45. Several types of identifiers are distinguished by their *ilk*:

normal identifiers are part of the ADA program and will appear in italic type.

predefined identifiers are part of the ADA program and will appear like 'INTEGER'.

roman identifiers are index entries that appear after `@^` in the `AWEB` file.

wildcard identifiers are index entries that appear after `@:` in the `AWEB` file.

italic identifiers are index entries that appear after `@i` in the `AWEB` file.

bold_face identifiers are index entries that appear after `@b` in the `AWEB` file.

slanted identifiers are index entries that appear after `@s` in the `AWEB` file.

typewriter identifiers are index entries that appear after `@.` in the `AWEB` file.

accept_like, *and_like*, ..., *with_like* identifiers are ADA reserved words whose *ilk* explains how they are to be treated when ADA code is being formatted.

define *reserved*(#) ≡ (*ilk*(#) > *predefined*) { tells if a name is a reserved word }
define *defined*(#) ≡ (*ilk*(#) > *typewriter*) { tells if a name is predefined or reserved }

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
normal : **constant** INTEGER ← 0; { ordinary identifiers have *normal* ilk }
roman : **constant** INTEGER ← 1; { normal index entries have *roman* ilk }
wildcard : **constant** INTEGER ← 2; { user-formatted index entries have *wildcard* ilk }
italic : **constant** INTEGER ← 3; { 'italic-type' entries have *italic* ilk }
bold_face : **constant** INTEGER ← 4; { 'boldface-type' entries have *bold_face* ilk }
slanted : **constant** INTEGER ← 5; { 'slanted-type' entries have *slanted* ilk }
typewriter : **constant** INTEGER ← 6; { 'typewriter type' entries have *typewriter* ilk }
predefined : **constant** INTEGER ← 7; { predefined identifiers have *predefined* ilk }
accept_like : **constant** INTEGER ← 8; { **accept** }
all_like : **constant** INTEGER ← 9; { **all**, **null**, **terminate** }
and_like : **constant** INTEGER ← 10; { **and** }
array_like : **constant** INTEGER ← 11; { **array** }
at_like : **constant** INTEGER ← 12; { **at** }
begin_like : **constant** INTEGER ← 13; { **begin**, **select** }
body_like : **constant** INTEGER ← 14; { **body** }
case_like : **constant** INTEGER ← 15; { **case** }
declare_like : **constant** INTEGER ← 16; { **declare** }
else_like : **constant** INTEGER ← 17; { **else** }
elsif_like : **constant** INTEGER ← 18; { **elsif** }
end_like : **constant** INTEGER ← 19; { **end** }
exception_like : **constant** INTEGER ← 20; { **exception** }
generic_like : **constant** INTEGER ← 21; { **generic** }
goto_like : **constant** INTEGER ← 22; { **abort**, ..., **while**, if not stated otherwise }
if_like : **constant** INTEGER ← 23; { **if** }
in_like : **constant** INTEGER ← 24; { **in** }
is_like : **constant** INTEGER ← 25; { **do**, **is** }
loop_like : **constant** INTEGER ← 26; { **loop** }
mod_like : **constant** INTEGER ← 27; { **mod**, **rem**, **xor** }
new_like : **constant** INTEGER ← 28; { **new** }
not_like : **constant** INTEGER ← 29; { **not** }
of_like : **constant** INTEGER ← 30; { **of** }
or_like : **constant** INTEGER ← 31; { **or** }
pragma_like : **constant** INTEGER ← 32; { **pragma** }
private_like : **constant** INTEGER ← 33; { **private** }
proc_like : **constant** INTEGER ← 34; { **entry**, **function**, **package**, **procedure**, **task** }
range_like : **constant** INTEGER ← 35; { **range** }

record_like : **constant** INTEGER ← 36; { **record** }
return_like : **constant** INTEGER ← 37; { **return**, **exit** }
separate_like : **constant** INTEGER ← 38; { **separate** }
then_like : **constant** INTEGER ← 39; { **then** }
type_like : **constant** INTEGER ← 40; { **subtype**, **type**, **for** }
when_like : **constant** INTEGER ← 41; { **when** }
with_like : **constant** INTEGER ← 42; { **use**, **with** }

46. The names of modules are stored in *byte_mem* together with the identifier names, but a hash table is not used for them because AWEAVE needs to be able to recognize a module name when given a prefix of that name. A conventional binary seach tree is used to retrieve module names, with fields called *llink* and *rlink* in place of *link* and *ilk*. The root of this tree is *rlink*(0).

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
llink : *name_info* **renames** *link*; { left link in binary search tree for module names }
rlink : *name_info* **renames** *ilk*; { right link in binary search tree for module names }
root : *sixteen_bits* **renames** *rlink*(0); { the root of the binary search tree for module names }

47. ⟨ Set initial values 19 ⟩ +≡
root ← 0; { the binary search tree starts out with nothing in it }

48. Here is a little procedure that prints the text of a given name.

⟨ Specification of procedures visible from *data_structures* 48 ⟩ ≡
procedure *print_id*(*p* : *name_pointer*);

See also section 53.

This code is used in section 3*.

49. ⟨ Procedures in *data_structures* 49 ⟩ ≡

```
procedure print_id(p : name_pointer) is    { print identifier or module name }
  w : w_range;  { segment of byte_mem }
begin
  if p ≥ name_ptr then
    print("IMPOSSIBLE");
  else
    w ← p mod ww;
    for k ∈ byte_start(p) .. byte_start(p + ww) − 1 loop
      print(xchr(byte_mem(w, k)));
    end loop;
  end if;
end print_id;
```

See also sections 54 and 55.

This code is used in section 3*.

50. We keep track of the current module number in *module_count*, which is the total number of modules that have started. Modules which have been altered by a change file entry have their *changed_module* flag turned on during the first phase.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
subtype *module_index* **is** INTEGER **range** 0 .. *max_modules*;
module_count : *module_index*; { the current module number }
changed_module : **array** (*module_index*) **of** BOOLEAN; { is it changed? }
change_exists : BOOLEAN; { has any module changed? }

51. The other large memory area in AWEAVE keeps the cross-reference data. All uses of the name p are recorded in a linked list beginning at *xref*(p), which points into the *xmem* array. Entries in *xmem* consist of two sixteen-bit items per word, called the *num* and *xlink* fields. If x is an index into *xmem*, reached from name p, the value of *num*(x) is either a module number where p is used, or it is *def_flag* plus a module number where p is defined, or it is *body_flag* plus a module number where the body of p is defined; and *xlink*(x) points to the next such cross reference for p, if any. This list of cross references is in decreasing order by module number. The current number of cross references is *xref_ptr*.

The global variable *xref_switch* is set either to *def_flag*, *body_flag* or to zero, depending on whether the next cross reference to an identifier is to be underlined, to be set in italic type, or neither of these in the index. This switch is incremented by *def_flag* when `@!` or `@d` or `@f` is scanned, and it is cleared to zero when the next identifier or index entry cross reference has been made. Similarly, the global variable *mod_xref_switch* is either *def_flag* or zero, depending on whether a module name is being defined or used.

```
define num(#) ≡ xmem(#).num_field
define xlink(#) ≡ xmem(#).xlink_field

⟨ Types and objects visible from data_structures 14 ⟩ +≡
  def_flag : constant INTEGER ← 10_240;  { must be strictly larger than max_modules }
  body_flag : constant INTEGER ← def_flag + def_flag;

  subtype xref_number is INTEGER range 0 .. max_refs;
  type xmem_field is
    record
      num_field : sixteen_bits;  { module number plus zero or def_flag }
      xlink_field : sixteen_bits;  { pointer to the previous cross reference }
    end record;

  xmem : array (xref_number) of xmem_field;
  xref_ptr : xref_number ← 0;  { the largest occupied position in xmem }
  xref_switch, mod_xref_switch : INTEGER range 0 .. body_flag ← 0;  { zero, def_flag or body_flag }
```

52. ⟨ Set initial values 19 ⟩ +≡
$num(0) \leftarrow 0$; $xref(0) \leftarrow 0$; { cross references to undefined modules }

53. A new cross reference for an identifier is formed by calling *new_xref*, which discards duplicate entries and ignores non-underlined references to one-letter identifiers or ADA's reserved words.

```
define append_xref(#) ≡
        if xref_ptr = max_refs then
          overflow("cross␣reference");
        else
          incr(xref_ptr); num(xref_ptr) ← #;
        end if

⟨ Specification of procedures visible from data_structures 48 ⟩ +≡
  procedure new_xref(p : name_pointer);
  procedure new_mod_xref(p : name_pointer);
```

54. ⟨ Procedures in *data_structures* 49 ⟩ +≡

```
procedure new_xref(p : name_pointer) is
  q : xref_number;  { pointer to previous cross reference }
  m, n : sixteen_bits;  { new and previous cross-reference value }
begin
  if (reserved(p) ∨ (byte_start(p) + 1 = byte_start(p + ww))) ∧ (xref_switch = 0) then
    return;
  end if;
  m ← module_count + xref_switch; xref_switch ← 0; q ← xref(p);
  if q > 0 then
    n ← num(q);
    if (n = m) ∨ (n − def_flag = m) ∨ (n − body_flag = m) then
      return;
    elsif (m − def_flag = n) ∨ (m − body_flag = n) then
      num(q) ← m; return;
    end if;
  end if;
  append_xref(m); xlink(xref_ptr) ← q; xref(p) ← xref_ptr;
end new_xref;
```

55. The cross reference lists for module names are slightly different. Suppose that a module name is defined in modules $m_1, \ldots, m_k$ and used in modules $n_1, \ldots, n_l$. Then its list will contain $m_1 + \mathit{def_flag}$, $m_k + \mathit{def_flag}, \ldots, m_2 + \mathit{def_flag}, n_l, \ldots, n_1$, in this order. After Phase II, however, the order will be $m_1 + \mathit{def_flag}, \ldots, m_k + \mathit{def_flag}, n_1, \ldots, n_l$.

⟨ Procedures in *data_structures* 49 ⟩ +≡

```
procedure new_mod_xref(p : name_pointer) is
  q, r : xref_number;  { pointers to previous cross references }
begin
  q ← xref(p); r ← 0;
  if q > 0 then
    if mod_xref_switch = 0 then
      while num(q) ≥ def_flag loop
        r ← q; q ← xlink(q);
      end loop;
    elsif num(q) ≥ def_flag then
      r ← q; q ← xlink(q);
    end if;
  end if;
  append_xref(module_count + mod_xref_switch); xlink(xref_ptr) ← q; mod_xref_switch ← 0;
  if r = 0 then
    xref(p) ← xref_ptr;
  else
    xlink(r) ← xref_ptr;
  end if;
end new_mod_xref;
```

56. A third large area of memory is used for sixteen-bit 'tokens', which appear in short lists similar to the strings of characters in *byte_mem*. Token lists are used to contain the result of ADA code translated into TEX form; further details about them will be explained later. A *text_pointer* variable is an index into *tok_start*.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
subtype *text_pointer* **is** INTEGER **range** 0 . . *max_texts*; { identifies a token list }

57. The first position of *tok_mem* that is unoccupied by replacement text is called *tok_ptr*, and the first unused location of *tok_start* is called *text_ptr*. Thus, we usually have *tok_start*(*text_ptr*) = *tok_ptr*.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
subtype *token_index* **is** INTEGER **range** 0 . . *max_toks*;
tok_mem : **array** (*token_index*) **of** *sixteen_bits*; { tokens }
tok_start : **array** (*text_pointer*) **of** *sixteen_bits*; { directory into *tok_mem* }
text_ptr : *text_pointer* ← 1; { first unused position in *tok_start* }
tok_ptr : *token_index* ← 1; { first unused position in *tok_mem* }
stat
max_tok_ptr, *max_txt_ptr* : *token_index* ← 1; { largest values occurring }
tats

58. ⟨ Set initial values 19 ⟩ +≡
tok_start(0 . . 1) ← (1, 1);

59. Searching for identifiers. The hash table described above is updated by the *id_lookup* procedure, which finds a given identifier and returns a pointer to its index in *byte_start*. The identifier is supposed to match character by character and it is also supposed to have a given *ilk* code; the same name may be present more than once if it is supposed to appear in the index with different typesetting conventions. If the identifier was not already present, it is inserted into the table.

Because of the way AWEAVE's scanning mechanism works, it is most convenient to let *id_lookup* search for an identifier that is present in the *buffer* array. Two other global variables specify its position in the buffer: the first character is *buffer*(*id_first*), and the last is *buffer*(*id_loc* − 1).

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
  id_first : *buffer_index*; { where the current identifier begins in the buffer }
  id_loc : *buffer_index*; { just after the current identifier in the buffer }

  subtype *hash_index* **is** INTEGER **range** 0 . . *hash_size*;
  hash : **array** (*hash_index*) **of** *sixteen_bits* ← *null_array*; { heads of hash lists—initially empty }

60. Here now is the main procedure for finding identifiers (and strings). The parameter t is set to the desired *ilk* code. The identifier must either have *ilk* = t, or we must have t = *normal* and the identifier must be predefined or a reserved word.

⟨ Specification of procedures visible from *searching* 60 ⟩ ≡
  function *id_lookup*(*t* : *eight_bits*)**return** *name_pointer*;

See also sections 75 and 79.

This code is used in section 5*.

61. ⟨ Procedures in *searching* 61 ⟩ ≡
  function *id_lookup*(*t* : *eight_bits*)**return** *name_pointer* **is** { finds current identifier }
    c : *eight_bits*; { byte being chopped }
    i : *buffer_index*; { index into *buffer* }
    h : *hash_index*; { hash code }
    k : *byte_index*; { index into *byte_mem* }
    w : *w_range*; { segment of *byte_mem* }
    l : *buffer_index* ← *id_loc* − *id_first*; { length of the given identifier }
    p, *q* : *name_pointer*; { where the identifier is being sought }
  begin
    ⟨ Compute the hash code *h* 62 ⟩;
    ⟨ Compute the name location *p* 63 ⟩;
  ⟪*found*⟫ **if** *p* = *name_ptr* **then**
      ⟨ Enter a new name into the table at position *p* 65 ⟩;
    end if;
    return *p*;
  end *id_lookup*;

See also sections 76 and 80.

This code is used in section 5*.

62. A simple hash code is used: If the sequence of ASCII codes is $c_1 c_2 \ldots c_n$, its hash value will be

$$(2^{n-1}c_1 + 2^{n-2}c_2 + \cdots + c_n) \bmod hash_size.$$

⟨ Compute the hash code h 62 ⟩ ≡
  $h \leftarrow buffer(id_first)$; $i \leftarrow id_first + 1$;
  while $i < id_loc$ **loop**
    $h \leftarrow (h + h + buffer(i))$ **mod** $hash_size$; $incr(i)$;
  end loop

This code is used in section 61.

63. If the identifier is new, it will be placed in position $p = name_ptr$, otherwise p will point to its existing location.

⟨ Compute the name location p 63 ⟩ ≡
  $p \leftarrow hash(h)$;
  while $p \neq 0$ **loop**
    if $(length(p) = l) \wedge ((ilk(p) = t) \vee ((t = normal) \wedge defined(p)))$ **then**
      ⟨ Compare name p with current identifier, **goto** *found* if equal 64 ⟩;
    end if;
    $p \leftarrow link(p)$;
  end loop;
  $p \leftarrow name_ptr$; { the current identifier is new }
  $link(p) \leftarrow hash(h)$; $hash(h) \leftarrow p$ { insert p at beginning of hash list }

This code is used in section 61.

64. ⟨ Compare name p with current identifier, **goto** *found* if equal 64 ⟩ ≡
  $i \leftarrow id_first$; $k \leftarrow byte_start(p)$; $w \leftarrow p$ **mod** ww;
  while $(i < id_loc) \wedge (buffer(i) = byte_mem(w, k))$ **loop**
    $incr(i)$; $incr(k)$;
  end loop;
  if $i = id_loc$ **then**
    goto *found*; { all characters agree }
  end if

This code is used in section 63.

65. When we begin the following segment of the program, $p = name_ptr$.

⟨ Enter a new name into the table at position p 65 ⟩ ≡
  $w \leftarrow name_ptr$ **mod** ww; $k \leftarrow byte_ptr(w)$;
  if $k + l > max_bytes$ **then**
    $overflow$(`"byte␣memory"`);
  end if;
  if $name_ptr > max_names - ww$ **then**
    $overflow$(`"name"`);
  end if;
  $i \leftarrow id_first$; { get ready to move the identifier into $byte_mem$ }
  while $i < id_loc$ **loop**
    $byte_mem(w, k) \leftarrow buffer(i)$; $incr(k)$; $incr(i)$;
  end loop;
  $byte_ptr(w) \leftarrow k$; $byte_start(name_ptr + ww) \leftarrow k$; $incr(name_ptr)$; $ilk(p) \leftarrow t$; $xref(p) \leftarrow 0$

This code is used in section 61.

66. Initializing the table of reserved and predefined words. We have to get ADA's reserved words and predefined identifiers into the hash table, and the simplest way to do this is to insert them every time `AWEAVE` is run. A few macros permit us to do the initialization with a compact program.

define *sid*(#) ≡ (#); *cur_name* ← *id_lookup*
define *id*(#) ≡ *id_loc* ← # + 1; *buffer*(1 . . #) ← *sid*
define *psid*(#) ≡ (#); *cur_name* ← *id_lookup*(*predefined*)
define *pid*(#) ≡ *id_loc* ← # + 1; *buffer*(1 . . #) ← *psid*

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
cur_name : *name_pointer*; { points to the identifier just inserted }

67. First we define the reserved words of ADA.

⟨ Store all the reserved words 67 ⟩ ≡

```
id_first ← 1;
id(5)([a], [b], [o], [r], [t])(goto_like);
id(3)([a], [b], [s])(goto_like);
id(6)([a], [c], [c], [e], [p], [t])(accept_like);
id(6)([a], [c], [c], [e], [s], [s])(goto_like);
id(3)([a], [l], [l])(all_like);
id(3)([a], [n], [d])(and_like);
id(5)([a], [r], [r], [a], [y])(array_like);
id(2)([a], [t])(at_like);
id(5)([b], [e], [g], [i], [n])(begin_like);
id(4)([b], [o], [d], [y])(body_like);
id(4)([c], [a], [s], [e])(case_like);
id(8)([c], [o], [n], [s], [t], [a], [n], [t])(goto_like);
id(7)([d], [e], [c], [l], [a], [r], [e])(declare_like);
id(5)([d], [e], [l], [a], [y])(goto_like);
id(5)([d], [e], [l], [t], [a])(goto_like);
id(6)([d], [i], [g], [i], [t], [s])(goto_like);
id(2)([d], [o])(is_like);
id(4)([e], [l], [s], [e])(else_like);
id(5)([e], [l], [s], [i], [f])(elsif_like);
id(3)([e], [n], [d])(end_like);
id(5)([e], [n], [t], [r], [y])(proc_like);
id(9)([e], [x], [c], [e], [p], [t], [i], [o], [n])(exception_like);
id(4)([e], [x], [i], [t])(return_like);
id(3)([f], [o], [r])(type_like);
id(8)([f], [u], [n], [c], [t], [i], [o], [n])(proc_like);
id(7)([g], [e], [n], [e], [r], [i], [c])(generic_like);
id(4)([g], [o], [t], [o])(goto_like);
id(2)([i], [f])(if_like);
id(2)([i], [n])(in_like);
id(2)([i], [s])(is_like);
id(7)([l], [i], [m], [i], [t], [e], [d])(goto_like);
id(4)([l], [o], [o], [p])(loop_like);
id(3)([m], [o], [d])(mod_like);
id(3)([n], [e], [w])(new_like);
id(3)([n], [o], [t])(not_like);
id(4)([n], [u], [l], [l])(all_like);
id(2)([o], [f])(of_like);
id(2)([o], [r])(or_like);
id(6)([o], [t], [h], [e], [r], [s])(goto_like);
```

See also section 68.

This code is used in section 308*.

68. ⟨ Store all the reserved words 67 ⟩ +≡

```
id(3)([o],[u],[t])(goto_like);
id(7)([p],[a],[c],[k],[a],[g],[e])(proc_like);
id(6)([p],[r],[a],[g],[m],[a])(pragma_like);
id(7)([p],[r],[i],[v],[a],[t],[e])(private_like);
id(9)([p],[r],[o],[c],[e],[d],[u],[r],[e])(proc_like);
id(5)([r],[a],[i],[s],[e])(goto_like);
id(5)([r],[a],[n],[g],[e])(range_like);
id(6)([r],[e],[c],[o],[r],[d])(record_like);
id(3)([r],[e],[m])(mod_like);
id(7)([r],[e],[n],[a],[m],[e],[s])(goto_like);
id(6)([r],[e],[t],[u],[r],[n])(return_like);
id(7)([r],[e],[v],[e],[r],[s],[e])(goto_like);
id(6)([s],[e],[l],[e],[c],[t])(begin_like);
id(8)([s],[e],[p],[a],[r],[a],[t],[e])(separate_like);
id(7)([s],[u],[b],[t],[y],[p],[e])(type_like);
id(4)([t],[a],[s],[k])(proc_like);
id(9)([t],[e],[r],[m],[i],[n],[a],[t],[e])(all_like);
id(4)([t],[h],[e],[n])(then_like);
id(4)([t],[y],[p],[e])(type_like);
id(3)([u],[s],[e])(with_like);
id(4)([w],[h],[e],[n])(when_like);
id(5)([w],[h],[i],[l],[e])(goto_like);
id(4)([w],[i],[t],[h])(with_like);
id(3)([x],[o],[r])(mod_like)
```

69. Here are the definitions for all the predefined identifiers of the ADA language; these are related to attributes, pragmas, exceptions, input/output facilities, and the standard environment.

⟨ Store all the predefined identifiers 69 ⟩ ≡

```
pid(7)([A],[D],[D],[R],[E],[S],[S]);
pid(3)([A],[F],[T]);
pid(5)([A],[S],[C],[I],[I]);
pid(4)([B],[A],[S],[E]);
pid(7)([B],[O],[O],[L],[E],[A],[N]);
pid(8)([C],[A],[L],[E],[N],[D],[A],[R]);
pid(8)([C],[A],[L],[L],[A],[B],[L],[E]);
pid(9)([C],[H],[A],[R],[A],[C],[T],[E],[R]);
pid(5)([C],[L],[O],[C],[K]);
pid(5)([C],[L],[O],[S],[E]);
pid(3)([C],[O],[L]);
pid(11)([C],[O],[N],[S],[T],[R],[A],[I],[N],[E],[D]);
pid(16)([C],[O],[N],[S],[T],[R],[A],[I],[N],[T],[_],[E],[R],[R],[O],[R]);
pid(10)([C],[O],[N],[T],[R],[O],[L],[L],[E],[D]);
pid(5)([C],[O],[U],[N],[T]);
pid(6)([C],[R],[E],[A],[T],[E]);
pid(13)([C],[U],[R],[R],[E],[N],[T],[_],[I],[N],[P],[U],[T]);
pid(14)([C],[U],[R],[R],[E],[N],[T],[_],[O],[U],[T],[P],[U],[T]);
```

See also sections 70, 71, and 72.

This code is used in section 308*.

70. ⟨ Store all the predefined identifiers 69 ⟩ +≡

```
pid(4)([D], [A], [T], [A]);
pid(10)([D], [A], [T], [A], [_], [E], [R], [R], [O], [R]);
pid(4)([D], [A], [T], [E]);
pid(3)([D], [A], [Y]);
pid(12)([D], [A], [Y], [_], [D], [U], [R], [A], [T], [I], [O], [N]);
pid(10)([D], [A], [Y], [_], [N], [U], [M], [B], [E], [R]);
pid(11)([D], [E], [F], [A], [U], [L], [T], [_], [A], [F], [T]);
pid(12)([D], [E], [F], [A], [U], [L], [T], [_], [B], [A], [S], [E]);
pid(11)([D], [E], [F], [A], [U], [L], [T], [_], [E], [X], [P]);
pid(12)([D], [E], [F], [A], [U], [L], [T], [_], [F], [O], [R], [E]);
pid(15)([D], [E], [F], [A], [U], [L], [T], [_], [S], [E], [T], [T], [I], [N], [G]);
pid(13)([D], [E], [F], [A], [U], [L], [T], [_], [W], [I], [D], [T], [H]);
pid(6)([D], [E], [L], [E], [T], [E]);
pid(5)([D], [E], [L], [T], [A]);
pid(6)([D], [E], [V], [I], [C], [E]);
pid(12)([D], [E], [V], [I], [C], [E], [_], [E], [R], [R], [O], [R]);
pid(6)([D], [I], [G], [I], [T], [S]);
pid(9)([D], [I], [R], [E], [C], [T], [_], [I], [O]);
pid(8)([D], [U], [R], [A], [T], [I], [O], [N]);
pid(8)([E], [L], [B], [O], [R], [A], [T], [E]);
pid(12)([E], [L], [E], [M], [E], [N], [T], [_], [T], [Y], [P], [E]);
pid(4)([E], [M], [A], [X]);
pid(9)([E], [N], [D], [_], [E], [R], [R], [O], [R]);
pid(11)([E], [N], [D], [_], [O], [F], [_], [F], [I], [L], [E]);
pid(11)([E], [N], [D], [_], [O], [F], [_], [L], [I], [N], [E]);
pid(11)([E], [N], [D], [_], [O], [F], [_], [P], [A], [G], [E]);
pid(4)([E], [N], [U], [M]);
pid(14)([E], [N], [U], [M], [E], [R], [A], [T], [I], [O], [N], [_], [I], [O]);
pid(7)([E], [P], [S], [I], [L], [O], [N]);
pid(3)([E], [X], [P]);
pid(5)([F], [A], [L], [S], [E]);
pid(5)([F], [I], [E], [L], [D]);
pid(4)([F], [I], [L], [E]);
pid(9)([F], [I], [L], [E], [_], [M], [O], [D], [E]);
pid(9)([F], [I], [L], [E], [_], [T], [Y], [P], [E]);
pid(10)([F], [I], [N], [E], [_], [D], [E], [L], [T], [A]);
pid(5)([F], [I], [R], [S], [T]);
pid(9)([F], [I], [R], [S], [T], [_], [B], [I], [T]);
pid(8)([F], [I], [X], [E], [D], [_], [I], [O]);
pid(5)([F], [L], [O], [A], [T]);
pid(8)([F], [L], [O], [A], [T], [_], [I], [O]);
pid(4)([F], [O], [R], [E]);
pid(4)([F], [O], [R], [M]);
pid(4)([F], [R], [O], [M]);
pid(3)([G], [E], [T]);
pid(8)([G], [E], [T], [_], [L], [I], [N], [E]);
pid(5)([I], [M], [A], [G], [E]);
pid(5)([I], [N], [D], [E], [X]);
pid(7)([I], [N], [_], [F], [I], [L], [E]);
pid(6)([I], [N], [L], [I], [N], [E]);
pid(10)([I], [N], [O], [U], [T], [_], [F], [I], [L], [E]);
```

$pid(7)([\texttt{I}], [\texttt{N}], [\texttt{T}], [\texttt{E}], [\texttt{G}], [\texttt{E}], [\texttt{R}]);$
$pid(10)([\texttt{I}], [\texttt{N}], [\texttt{T}], [\texttt{E}], [\texttt{G}], [\texttt{E}], [\texttt{R}], [_], [\texttt{I}], [\texttt{O}]);$
$pid(13)([\texttt{I}], [\texttt{O}], [_], [\texttt{E}], [\texttt{X}], [\texttt{C}], [\texttt{E}], [\texttt{P}], [\texttt{T}], [\texttt{I}], [\texttt{O}], [\texttt{N}], [\texttt{S}]);$
$pid(7)([\texttt{I}], [\texttt{S}], [_], [\texttt{O}], [\texttt{P}], [\texttt{E}], [\texttt{N}]);$
$pid(4)([\texttt{I}], [\texttt{T}], [\texttt{E}], [\texttt{M}]);$

71. ⟨ Store all the predefined identifiers 69 ⟩ +≡

```
pid(5)([L], [A], [R], [G], [E]);
pid(4)([L], [A], [S], [T]);
pid(8)([L], [A], [S], [T], [_], [B], [I], [T]);
pid(12)([L], [A], [Y], [O], [U], [T], [_], [E], [R], [R], [O], [R]);
pid(6)([L], [E], [N], [G], [T], [H]);
pid(4)([L], [I], [N], [E]);
pid(11)([L], [I], [N], [E], [_], [L], [E], [N], [G], [T], [H]);
pid(4)([L], [I], [S], [T]);
pid(10)([L], [O], [W], [E], [R], [_], [C], [A], [S], [E]);
pid(12)([L], [O], [W], [_], [L], [E], [V], [E], [L], [_], [I], [O]);
pid(12)([M], [A], [C], [H], [I], [N], [E], [_], [E], [M], [A], [X]);
pid(12)([M], [A], [C], [H], [I], [N], [E], [_], [E], [M], [I], [N]);
pid(16)([M], [A], [C], [H], [I], [N], [E], [_], [M], [A], [N], [T], [I], [S], [S], [A]);
pid(17)([M], [A], [C], [H], [I], [N], [E], [_], [O], [V], [E], [R], [F], [L], [O], [W], [S]);
pid(13)([M], [A], [C], [H], [I], [N], [E], [_], [R], [A], [D], [I], [X]);
pid(14)([M], [A], [C], [H], [I], [N], [E], [_], [R], [O], [U], [N], [D], [S]);
pid(8)([M], [A], [N], [T], [I], [S], [S], [A]);
pid(10)([M], [A], [X], [_], [D], [I], [G], [I], [T], [S]);
pid(7)([M], [A], [X], [_], [I], [N], [T]);
pid(12)([M], [A], [X], [_], [M], [A], [N], [T], [I], [S], [S], [A]);
pid(11)([M], [E], [M], [O], [R], [Y], [_], [S], [I], [Z], [E]);
pid(7)([M], [I], [N], [_], [I], [N], [T]);
pid(4)([M], [O], [D], [E]);
pid(10)([M], [O], [D], [E], [_], [E], [R], [R], [O], [R]);
pid(5)([M], [O], [N], [T], [H]);
pid(12)([M], [O], [N], [T], [H], [_], [N], [U], [M], [B], [E], [R]);
pid(4)([N], [A], [M], [E]);
pid(10)([N], [A], [M], [E], [_], [E], [R], [R], [O], [R]);
pid(7)([N], [A], [T], [U], [R], [A], [L]);
pid(8)([N], [E], [W], [_], [L], [I], [N], [E]);
pid(8)([N], [E], [W], [_], [P], [A], [G], [E]);
pid(3)([N], [U], [M]);
pid(11)([N], [U], [M], [B], [E], [R], [_], [B], [A], [S], [E]);
pid(13)([N], [U], [M], [E], [R], [I], [C], [_], [E], [R], [R], [O], [R]);
pid(3)([O], [F], [F]);
pid(2)([O], [N]);
pid(4)([O], [P], [E], [N]);
pid(8)([O], [P], [T], [I], [M], [I], [Z], [E]);
pid(8)([O], [U], [T], [_], [F], [I], [L], [E]);
pid(4)([P], [A], [C], [K]);
pid(4)([P], [A], [G], [E]);
pid(11)([P], [A], [G], [E], [_], [L], [E], [N], [G], [T], [H]);
pid(3)([P], [O], [S]);
pid(8)([P], [O], [S], [I], [T], [I], [O], [N]);
pid(8)([P], [O], [S], [I], [T], [I], [V], [E]);
pid(14)([P], [O], [S], [I], [T], [I], [V], [E], [_], [C], [O], [U], [N], [T]);
pid(4)([P], [R], [E], [D]);
pid(8)([P], [R], [I], [O], [R], [I], [T], [Y]);
pid(13)([P], [R], [O], [G], [R], [A], [M], [_], [E], [R], [R], [O], [R]);
pid(3)([P], [U], [T]);
pid(8)([P], [U], [T], [_], [L], [I], [N], [E]);
```

```
pid(5)([R], [A], [N], [G], [E]);
pid(4)([R], [E], [A], [D]);
pid(15)([R], [E], [C], [E], [I], [V], [E], [_], [C], [O], [N], [T], [R], [O], [L]);
pid(5)([R], [E], [S], [E], [T]);
```

72. ⟨ Store all the predefined identifiers 69 ⟩ +≡

```
pid(9)([S], [A], [F], [E], [_], [E], [M], [A], [X]);
pid(10)([S], [A], [F], [E], [_], [L], [A], [R], [G], [E]);
pid(10)([S], [A], [F], [E], [_], [S], [M], [A], [L], [L]);
pid(7)([S], [E], [C], [O], [N], [D], [S]);
pid(12)([S], [E], [N], [D], [_], [C], [O], [N], [T], [R], [O], [L]);
pid(13)([S], [E], [Q], [U], [E], [N], [T], [I], [A], [L], [_], [I], [O]);
pid(3)([S], [E], [T]);
pid(7)([S], [E], [T], [_], [C], [O], [L]);
pid(9)([S], [E], [T], [_], [I], [N], [D], [E], [X]);
pid(9)([S], [E], [T], [_], [I], [N], [P], [U], [T]);
pid(8)([S], [E], [T], [_], [L], [I], [N], [E]);
pid(15)([S], [E], [T], [_], [L], [I], [N], [E], [_], [L], [E], [N], [G], [T], [H]);
pid(10)([S], [E], [T], [_], [O], [U], [T], [P], [U], [T]);
pid(15)([S], [E], [T], [_], [P], [A], [G], [E], [_], [L], [E], [N], [G], [T], [H]);
pid(6)([S], [H], [A], [R], [E], [D]);
pid(4)([S], [I], [Z], [E]);
pid(9)([S], [K], [I], [P], [_], [L], [I], [N], [E]);
pid(9)([S], [K], [I], [P], [_], [P], [A], [G], [E]);
pid(5)([S], [M], [A], [L], [L]);
pid(5)([S], [P], [A], [C], [E]);
pid(7)([S], [P], [A], [C], [I], [N], [G]);
pid(5)([S], [P], [L], [I], [T]);
pid(8)([S], [T], [A], [N], [D], [A], [R], [D]);
pid(14)([S], [T], [A], [N], [D], [A], [R], [D], [_], [I], [N], [P], [U], [T]);
pid(15)([S], [T], [A], [N], [D], [A], [R], [D], [_], [O], [U], [T], [P], [U], [T]);
pid(12)([S], [T], [A], [T], [U], [S], [_], [E], [R], [R], [O], [R]);
pid(13)([S], [T], [O], [R], [A], [G], [E], [_], [E], [R], [R], [O], [R]);
pid(12)([S], [T], [O], [R], [A], [G], [E], [_], [S], [I], [Z], [E]);
pid(12)([S], [T], [O], [R], [A], [G], [E], [_], [U], [N], [I], [T]);
pid(6)([S], [T], [R], [I], [N], [G]);
pid(4)([S], [U], [C], [C]);
pid(8)([S], [U], [P], [P], [R], [E], [S], [S]);
pid(6)([S], [Y], [S], [T], [E], [M]);
pid(11)([S], [Y], [S], [T], [E], [M], [_], [N], [A], [M], [E]);
pid(13)([T], [A], [S], [K], [I], [N], [G], [_], [E], [R], [R], [O], [R]);
pid(10)([T], [E], [R], [M], [I], [N], [A], [T], [E], [D]);
pid(7)([T], [E], [X], [T], [_], [I], [O]);
pid(4)([T], [I], [C], [K]);
pid(4)([T], [I], [M], [E]);
pid(10)([T], [I], [M], [E], [_], [E], [R], [R], [O], [R]);
pid(7)([T], [I], [M], [E], [_], [O], [F]);
pid(2)([T], [O]);
pid(4)([T], [R], [U], [E]);
pid(8)([T], [Y], [P], [E], [_], [S], [E], [T]);
pid(9)([U], [N], [B], [O], [U], [N], [D], [E], [D]);
pid(20)([U], [N], [C], [H], [E], [C], [K], [E], [D], [_], [C], [O], [N], [V], [E], [R], [S], [I], [O], [N]);
pid(22)([U], [N], [C], [H], [E], [C], [K], [E], [D], [_], [D], [E], [A], [L], [L], [O], [C], [A], [T], [I], [O], [N]);
pid(10)([U], [P], [P], [E], [R], [_], [C], [A], [S], [E]);
pid(9)([U], [S], [E], [_], [E], [R], [R], [O], [R]);
pid(3)([V], [A], [L]);
pid(5)([V], [A], [L], [U], [E]);
```

$pid(5)([\texttt{W}], [\texttt{I}], [\texttt{D}], [\texttt{T}], [\texttt{H}]);$
$pid(5)([\texttt{W}], [\texttt{R}], [\texttt{I}], [\texttt{T}], [\texttt{E}]);$
$pid(4)([\texttt{Y}], [\texttt{E}], [\texttt{A}], [\texttt{R}]);$
$pid(11)([\texttt{Y}], [\texttt{E}], [\texttt{A}], [\texttt{R}], [_], [\texttt{N}], [\texttt{U}], [\texttt{M}], [\texttt{B}], [\texttt{E}], [\texttt{R}])$

73. Searching for module names. The *mod_lookup* procedure finds the module name *mod_text*(1 . . *l*) in the search tree, after inserting it if necessary, and returns a pointer to where it was found.

```
⟨ Types and objects visible from data_structures 14 ⟩ +≡
  subtype long_name_index is INTEGER range 0 . . longest_name;
  mod_text : array (long_name_index) of ASCII_code;   { name being sought for }
```

74. According to the rules of AWEB, no module name should be a proper prefix of another, so a "clean" comparison should occur between any two names. The result of *mod_lookup* is 0 if this prefix condition is violated. An error message is printed when such violations are detected during phase two of AWEAVE.

```
⟨ Types and objects visible from data_structures 14 ⟩ +≡
  type compare is (less, equal, greater, prefix, extension);
```

75.

```
⟨ Specification of procedures visible from searching 60 ⟩ +≡
  function mod_lookup(l : sixteen_bits)return name_pointer;
```

76.

```
⟨ Procedures in searching 61 ⟩ +≡
  function mod_lookup(l : sixteen_bits)return name_pointer is   { finds module name }
    c : compare ← greater;   { comparison between two names }
    j : long_name_index;   { index into mod_text }
    k : byte_index;   { index into byte_mem }
    w : w_range;   { segment of byte_mem }
    p : name_pointer ← root;   { current node of the search tree }
    q : name_pointer ← 0;   { father of node p }
  begin
    while p ≠ 0 loop
      ⟨ Set variable c to the result of comparing the given name to name p 78 ⟩;
      q ← p;
      if c = less then
        p ← llink(q);
      elsif c = greater then
        p ← rlink(q);
      else
        goto found;
      end if;
    end loop;
    ⟨ Enter a new module name into the tree 77 ⟩;
  ⟨⟨found⟩⟩ if c ≠ equal then
      err_print("!␣Incompatible␣section␣names"); p ← 0;
    end if;
    return p;
  end mod_lookup;
```

77. ⟨ Enter a new module name into the tree 77 ⟩ ≡

```
w ← name_ptr mod ww; k ← byte_ptr(w);
if k + l > max_bytes then
   overflow("byte␣memory");
end if;
if name_ptr > max_names − ww then
   overflow("name");
end if;
p ← name_ptr;
if c = less then
   llink(q) ← p;
else
   rlink(q) ← p;
end if;
llink(p) ← 0; rlink(p) ← 0; c ← equal;
for m ∈ 1 . . l loop
   byte_mem(w, k + m − 1) ← mod_text(m);
end loop;
byte_ptr(w) ← k + l; byte_start(name_ptr + ww) ← k + l; incr(name_ptr)
```

This code is used in section 76.

78. ⟨ Set variable c to the result of comparing the given name to name p 78 ⟩ ≡

```
k ← byte_start(p); w ← p mod ww; c ← equal; j ← 1;
while (k < byte_start(p + ww)) ∧ (j ≤ l) ∧ (mod_text(j) = byte_mem(w, k)) loop
   incr(k); incr(j);
end loop;
if k = byte_start(p + ww) then
   if j > l then
      c ← equal;
   else
      c ← extension;
   end if;
elsif j > l then
   c ← prefix;
elsif mod_text(j) < byte_mem(w, k) then
   c ← less;
else
   c ← greater;
end if
```

This code is used in sections 76 and 80.

79. The *prefix_lookup* procedure is supposed to find exactly one module name that has *mod_text*(1 . . l) as a prefix. Actually the algorithm silently accepts also the situation that some module name is a prefix of *mod_text*(1 . . l), because the user who painstakingly typed in more than necessary probably doesn't want to be told about the wasted effort.

⟨ Specification of procedures visible from *searching* 60 ⟩ +≡

```
function prefix_lookup(l : sixteen_bits)return name_pointer;
```

80. ⟨ Procedures in *searching* 61 ⟩ +≡

```
function prefix_lookup(l : sixteen_bits)return name_pointer is   { finds name extension }
  c : compare;  { comparison between two names }
  count : name_index ← 0;  { the number of hits }
  j : long_name_index;  { index into mod_text }
  k : byte_index;  { index into byte_mem }
  w : w_range;  { segment of byte_mem }
  p : name_pointer ← root;  { current node of the search tree }
  q : name_pointer ← 0;  { another place to resume the search after one branch is done }
  r : name_pointer ← 0;  { extension found }
begin
  while p ≠ 0 loop
    ⟨ Set variable c to the result of comparing the given name to name p 78 ⟩;
    if c = less then
      p ← llink(p);
    elsif c = greater then
      p ← rlink(p);
    else
      r ← p; incr(count); q ← rlink(p); p ← llink(p);
    end if;
    if p = 0 then
      p ← q; q ← 0;
    end if;
  end loop;
  if count ≠ 1 then
    if count = 0 then
      err_print("!␣Name␣does␣not␣match");
    else
      err_print("!␣Ambiguous␣prefix");
    end if;
  end if;
  return r;  { the result will be 0 if there was no match }
end prefix_lookup;
```

81. Lexical scanning. Let us now consider the subroutines that read the AWEB source file and break it into meaningful units. There are four such procedures: One simply skips to the next '@␣' or '@*' that begins a module; another passes over the TEX text at the beginning of a module; the third passes over the TEX text in a ADA comment; and the last, which is the most interesting, gets the next token of a ADA text.

82. But first we need to consider the low-level routine *get_line* that takes care of merging *change_file* into *aweb_file*. The *get_line* procedure also updates the line numbers for error messages.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
  line : INTEGER; { the number of the current line in the current file }
  other_line : INTEGER; { the number of the current line in the input file that is not currently being read }
  temp_line : INTEGER; { used when interchanging *line* with *other_line* }
  limit : *buffer_index*; { the last character position occupied in the buffer }
  loc : *buffer_index*; { the next character position to be read from the buffer }
  input_has_ended : BOOLEAN; { if TRUE, there is no more input }
  changing : BOOLEAN; { if TRUE, the current line is from *change_file* }

83. As we change *changing* from TRUE to FALSE and back again, we must remember to swap the values of *line* and *other_line* so that the *err_print* routine will be sure to report the correct line number.

define *change_changing* ≡ *changing* ← ¬*changing*; *temp_line* ← *other_line*; *other_line* ← *line*;
  line ← *temp_line* { *line* ↔ *other_line* }

84. When *changing* is FALSE, the next line of *change_file* is kept in *change_buffer*(0 .. *change_limit*), for purposes of comparison with the next line of *aweb_file*. After the change file has been completely input, we set *change_limit* ← 0, so that no further matches will be made.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
  change_buffer : *buffer_type*(*buffer_index*);
  change_limit : *buffer_index*; { the last position occupied in *change_buffer* }

85. Here's a simple function that checks if the two buffers are different.

⟨ Procedures in *lexical_scanning* 85 ⟩ ≡
```
function lines_dont_match return BOOLEAN is
begin
  if change_limit ≠ limit then
    return TRUE;
  end if;
  if (limit > 0) and then change_buffer(0 .. limit − 1) ≠ buffer(0 .. limit − 1) then
    return TRUE;
  end if;
  return FALSE;
end lines_dont_match;
```
See also sections 86, 90, 93, 95, 101, 103, 105, 107, and 111.

This code is used in section 6*.

86. Procedure *prime_the_change_buffer* sets *change_buffer* in preparation for the next matching operation. Since blank lines in the change file are not used for matching, we have $(change_limit = 0) \wedge \neg changing$ if and only if the change file is exhausted. This procedure is called only when *changing* is true; hence error messages will be reported correctly.

```
⟨ Procedures in lexical_scanning 85 ⟩ +≡
  procedure prime_the_change_buffer is
  begin
    change_limit ← 0;  { this value will be used if the change file ends }
    ⟨ Skip over comment lines in the change file; return if end of file 87 ⟩;
    ⟨ Skip to the next nonblank line; return if end of file 88 ⟩;
    ⟨ Move buffer and limit to change_buffer and change_limit 89 ⟩;
  end prime_the_change_buffer;
```

87. While looking for a line that begins with `@x` in the change file, we allow lines that begin with `@`, as long as they don't begin with `@y` or `@z` (which would probably indicate that the change file is fouled up).

```
⟨ Skip over comment lines in the change file; return if end of file 87 ⟩ ≡
  loop
    incr(line);
    if ¬input_ln(change_file) then
      return;
    elsif limit < 2 then
      goto continue;
    elsif buffer(0) ≠ [@] then
      goto continue;
    elsif buffer(1) ∈ [X] .. [Z] then
      buffer(1) ← buffer(1) + 40_8;  { lowercasify }
    end if;
    exit when buffer(1) = [x];
    if buffer(1) ∈ [y] .. [z] then
      loc ← 2;  err_print("!␣Where␣is␣the␣matching␣@x?");
    end if;
  ⟪continue⟫ null;
  end loop
```

This code is used in section 86.

88. Here we are looking at lines following the `@x`.

```
⟨ Skip to the next nonblank line; return if end of file 88 ⟩ ≡
  loop
    incr(line);
    if ¬input_ln(change_file) then
      err_print("!␣Change␣file␣ended␣after␣@x");
      return;
    end if;
    exit when limit > 0;
  end loop
```

This code is used in section 86.

89. ⟨ Move *buffer* and *limit* to *change_buffer* and *change_limit* 89 ⟩ ≡

```
change_limit ← limit;
if limit > 0 then
  change_buffer(0 .. limit − 1) ← buffer(0 .. limit − 1);
end if
```

This code is used in sections 86 and 90.

90. The following procedure is used to see if the next change entry should go into effect; it is called only when *changing* is false. The idea is to test whether or not the current contents of *buffer* matches the current contents of *change_buffer*. If not, there's nothing more to do; but if so, a change is called for: All of the text down to the `@y` is supposed to match. An error message is issued if any discrepancy is found. Then the procedure prepares to read the next line from *change_file*.

⟨ Procedures in *lexical_scanning* 85 ⟩ +≡

```
procedure check_change is   { switches to change_file if the buffers match }
  n : INTEGER ← 0;   { the number of discrepancies found }
begin
  if lines_dont_match then
    return;
  end if;
  loop
    change_changing;   { now it's TRUE }
    incr(line);
    if ¬input_ln(change_file) then
      err_print("!␣Change␣file␣ended␣before␣@y");
      change_limit ← 0; change_changing;   { FALSE again }
      return;
    end if;
    ⟨ If the current line starts with @y, report any discrepancies and return 91 ⟩;
    ⟨ Move buffer and limit to change_buffer and change_limit 89 ⟩;
    change_changing;   { now it's FALSE }
    incr(line);
    if ¬input_ln(aweb_file) then
      err_print("!␣AWEB␣file␣ended␣during␣a␣change");
      input_has_ended ← TRUE; return;
    end if;
    if lines_dont_match then
      incr(n);
    end if;
  end loop;
end check_change;
```

91. ⟨ If the current line starts with `@y`, report any discrepancies and **return** 91 ⟩ ≡

```
if (limit > 1) and then buffer(0) = [@] then
  if buffer(1) ∈ [X] .. [Z] then
    buffer(1) ← buffer(1) + 40₈;  { lowercasify }
  end if;
  if (buffer(1) = [x]) ∨ (buffer(1) = [z]) then
    loc ← 2; err_print("!␣Where␣is␣the␣matching␣@y?");
  elsif buffer(1) = [y] then
    if n > 0 then
      loc ← 2; new_ln; print("!␣Hmm...␣"); print_int(n, 1);
      print("␣of␣the␣preceding␣lines␣failed␣to␣match"); error;
    end if;
    return;
  end if;
end if
```

This code is used in section 90.

92. The *reset_input* procedure, which gets `AWEAVE` ready to read the user's `AWEB` input, is used at the beginning of phases one and two.

⟨ Specification of procedures visible from *lexical_scanning* 92 ⟩ ≡

```
procedure reset_input;
```

See also sections 94, 100, 102, 104, 106, and 110.

This code is used in section 6*.

93. ⟨ Procedures in *lexical_scanning* 85 ⟩ +≡

```
procedure reset_input is
begin
  TEXT_IO.RESET(aweb_file);
  if TEXT_IO.IS_OPEN(change_file) then
    TEXT_IO.RESET(change_file);
  end if;
  line ← 0; other_line ← 0;
  changing ← TRUE; prime_the_change_buffer; change_changing;
  limit ← 0; loc ← 1; buffer(0) ← [␣]; input_has_ended ← FALSE;
end reset_input;
```

94. The *get_line* procedure is called when *loc* > *limit*; it puts the next line of merged input into the buffer and updates the other variables appropriately. A space is placed at the right end of the line.

⟨ Specification of procedures visible from *lexical_scanning* 92 ⟩ +≡

```
procedure get_line;
```

95. ⟨ Procedures in *lexical_scanning* 85 ⟩ +≡

```
procedure get_line is   { inputs the next line }
begin
⟪restart⟫ if changing then
      changed_module(module_count) ← TRUE;
  else
      ⟨ Read from aweb_file and maybe turn on changing 96 ⟩;
  end if;
  if changing then
      ⟨ Read from change_file and maybe turn off changing 97 ⟩;
      if ¬changing then
          changed_module(module_count) ← TRUE; goto restart;
      end if;
  end if;
  loc ← 0; buffer(limit) ← [␣];
end get_line;
```

96. ⟨ Read from *aweb_file* and maybe turn on *changing* 96 ⟩ ≡

```
incr(line);
if ¬input_ln(aweb_file) then
    input_has_ended ← TRUE;
elsif (limit = change_limit) ∧ ((buffer(0) = change_buffer(0)) ∧ (change_limit > 0)) then
    check_change;
end if
```

This code is used in section 95.

97. ⟨ Read from *change_file* and maybe turn off *changing* 97 ⟩ ≡

```
incr(line);
if ¬input_ln(change_file) then
    err_print("!␣Change␣file␣ended␣without␣@z");
    buffer(0 .. 1) ← ([@], [z]); limit ← 2;
end if;
if (limit > 1) and then (buffer(0) = [@]) then   { check if the change has ended }
    if buffer(1) ∈ [X] .. [Z] then
        buffer(1) ← buffer(1) + 40₈;   { lowercasify }
    end if;
    if buffer(1) ∈ [x] .. [y] then
        loc ← 2; err_print("!␣Where␣is␣the␣matching␣@z?");
    elsif buffer(1) = [z] then
        prime_the_change_buffer; change_changing;
    end if;
end if
```

This code is used in section 95.

98. At the end of the program, we will tell the user if the change file had a line that didn't match any relevant line in *aweb_file*.

⟨ Check that all changes have been read 98 ⟩ ≡

```
if change_limit ≠ 0 then   { changing is FALSE }
    buffer(0 .. change_limit) ← change_buffer(0 .. change_limit); limit ← change_limit; changing ← TRUE;
    line ← other_line; loc ← change_limit; err_print("!␣Change␣file␣entry␣did␣not␣match");
end if
```

This code is used in section 308*.

99. Control codes in AWEB, which begin with '@', are converted into a numeric code designed to simplify AWEAVE's logic; for example, larger numbers are given to the control codes that denote more significant milestones, and the code of *new_module* should be the largest of all. Some of these numeric control codes take the place of ASCII control codes that will not otherwise appear in the output of the scanning routines.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
ignore : **constant** *eight_bits* ← 0; { control code of no interest to AWEAVE }
verbatim : **constant** *eight_bits* ← 2_8; { extended ASCII alpha will not appear }
force_line : **constant** *eight_bits* ← 3_8; { extended ASCII beta will not appear }
begin_comment : **constant** *eight_bits* ← 11_8; { ASCII tab mark will not appear }
end_comment : **constant** *eight_bits* ← 12_8; { ASCII line feed will not appear }
character_tok : **constant** *eight_bits* ← 14_8; { ASCII form feed will not appear }
ascii_tok : **constant** *eight_bits* ← 15_8; { ASCII carriage return will not appear }
output_tok : **constant** *eight_bits* ← 16_8; { ASCII DLE will not appear }
double_dot : **constant** *eight_bits* ← 40_8; { ASCII space will not appear except in strings }
no_underline : **constant** *eight_bits* ← 175_8; { this code will be intercepted without confusion }
underline : **constant** *eight_bits* ← 176_8; { this code will be intercepted without confusion }
param : **constant** *eight_bits* ← 177_8; { ASCII delete will not appear }
xref_roman : **constant** *eight_bits* ← 204_8; { control code for '@^' }
xref_wildcard : **constant** *eight_bits* ← 205_8; { control code for '@:' }
xref_italic : **constant** *eight_bits* ← 206_8; { control code for '@i' }
xref_bold_face : **constant** *eight_bits* ← 207_8; { control code for '@b' }
xref_slanted : **constant** *eight_bits* ← 210_8; { control code for '@s' }
xref_typewriter : **constant** *eight_bits* ← 211_8; { control code for '@.' }
xref_predefined : **constant** *eight_bits* ← 212_8; { control code for '@p' }
xref_reserved : **constant** *eight_bits* ← 213_8; { control code for '@r' }
TeX_string : **constant** *eight_bits* ← 214_8; { control code for '@t' }
join : **constant** *eight_bits* ← 215_8; { control code for '@&' }
thin_space : **constant** *eight_bits* ← 216_8; { control code for '@,' }
math_break : **constant** *eight_bits* ← 217_8; { control code for '@|' }
line_break : **constant** *eight_bits* ← 220_8; { control code for '@/' }
big_line_break : **constant** *eight_bits* ← 221_8; { control code for '@#' }
no_line_break : **constant** *eight_bits* ← 222_8; { control code for '@+' }
pseudo_semi : **constant** *eight_bits* ← 223_8; { control code for '@;' }
format : **constant** *eight_bits* ← 224_8; { control code for '@f' }
definition : **constant** *eight_bits* ← 225_8; { control code for '@d' }
begin_ada : **constant** *eight_bits* ← 226_8; { control code for '@a' }
module_name : **constant** *eight_bits* ← 227_8; { control code for '@<' }
new_module : **constant** *eight_bits* ← 230_8; { control code for '@␣' and '@*' }

100. Control codes are converted from ASCII to AWEAVE's internal representation by the *control_code* routine.

⟨ Specification of procedures visible from *lexical_scanning* 92 ⟩ +≡
function *control_code*(*c* : *ASCII_code*)**return** *eight_bits*;

101. ⟨ Procedures in *lexical_scanning* 85 ⟩ +≡

```
function control_code(c : ASCII_code)return eight_bits is   { convert c after @ }
begin
  case c is
    when [@] ⇒ return [@];  { 'quoted' at sign }
    when [´] ⇒ return ascii_tok;  { precedes an ASCII constant }
    when [~] ⇒ return output_tok;  { output file name }
    when [␣] | tab_mark | [*] ⇒ return new_module;  { beginning of a new module }
    when [=] ⇒ return verbatim;  { verbatim string }
    when [\] ⇒ return force_line;  { force line in ADA output of ATANGLE }
    when [D] | [d] ⇒ return definition;  { macro definition }
    when [F] | [f] ⇒ return format;  { format definition }
    when [{] ⇒ return begin_comment;  { begin-comment delimiter }
    when [}] ⇒ return end_comment;  { end-comment delimiter }
    when [A] | [a] ⇒ return begin_ada;  { ADA text in unnamed module }
    when [&] ⇒ return join;  { concatenate two tokens }
    when [<] ⇒ return module_name;  { beginning of a module name }
    when [>] ⇒
      err_print("!␣Extra␣@>");
      return ignore;  { end of module name should not be discovered in this way }
    when [T] | [t] ⇒ return TeX_string;  { TEX box within ADA }
    when [!] ⇒ return underline;  { increment definition flag }
    when [?] ⇒ return no_underline;  { decrement definition flag }
    when [^] ⇒ return xref_roman;  { index entry to be typeset normally }
    when [:] ⇒ return xref_wildcard;  { index entry to be in user format }
    when [i] | [I] ⇒ return xref_italic;  { index entry to be in italic type }
    when [b] | [B] ⇒ return xref_bold_face;  { index entry to be in boldface type }
    when [s] | [S] ⇒ return xref_slanted;  { index entry to be in slanted type }
    when [p] | [P] ⇒ return xref_predefined;  { index entry like predefined identifier }
    when [r] | [R] ⇒ return xref_reserved;  { reserved word to be indexed }
    when [.] ⇒ return xref_typewriter;  { index entry to be in typewriter type }
    when [,] ⇒ return thin_space;  { puts extra space in ADA format }
    when [|] ⇒ return math_break;  { allows a break in a formula }
    when [/] ⇒ return line_break;  { forces end-of-line in ADA format }
    when [#] ⇒ return big_line_break;  { forces end-of-line and some space besides }
    when [+] ⇒ return no_line_break;  { cancels end-of-line down to single space }
    when [;] ⇒ return pseudo_semi;  { acts like a semicolon, but is invisible }
    when others ⇒
      err_print("!␣Unknown␣control␣code");
      return ignore;
  end case;
end control_code;
```

102. The *skip_limbo* routine is used on the first pass to skip through portions of the input that are not in any modules, i.e., that precede the first module. After this procedure has been called, the value of *input_has_ended* will tell whether or not a new module has actually been found.

⟨ Specification of procedures visible from *lexical_scanning* 92 ⟩ +≡

```
procedure skip_limbo;
```

103. ⟨ Procedures in *lexical_scanning* 85 ⟩ +≡

```
procedure skip_limbo is    { skip to next module }
  c : ASCII_code;   { character following @ }
begin
  loop
    if loc > limit then
      get_line;
      if input_has_ended then
        return;
      end if;
    else
      buffer(limit + 1) ← [@];
      while buffer(loc) ≠ [@] loop
        incr(loc);
      end loop;
      if loc ≤ limit then
        loc ← loc + 2;  c ← buffer(loc − 1);
        if (c = [␣]) ∨ (c = tab_mark) ∨ (c = [*]) then
          return;
        end if;
      end if;
    end if;
  end loop;
end skip_limbo;
```

104. The *skip_TeX* routine is used on the first pass to skip through the TEX code at the beginning of a module. It returns the next control code or '|' found in the input. A *new_module* is assumed to exist at the very end of the file.

⟨ Specification of procedures visible from *lexical_scanning* 92 ⟩ +≡

```
function skip_TeX return eight_bits;
```

105. ⟨ Procedures in *lexical_scanning* 85 ⟩ +≡

```
function skip_TeX return eight_bits is   { skip past pure TEX code }
  c : eight_bits;   { control code found }
begin
  loop
    if loc > limit then
      get_line;
      if input_has_ended then
        return new_module;
      end if;
    end if;
    buffer(limit + 1) ← [@];
    loop
      c ← buffer(loc); incr(loc);
      if c = [|] then
        return [|];
      end if;
      exit when c = [@];
    end loop;
    if loc ≤ limit then
      c ← control_code(buffer(loc)); incr(loc); return c;
    end if;
  end loop;
end skip_TeX;
```

106. The *skip_comment* routine is used on the first pass to skip through TEX code in ADA comments. The *bal* parameter tells how many left braces are assumed to have been scanned when this routine is called, and the procedure returns a corresponding value of *bal* at the point that scanning has stopped. Scanning stops either at a '|' that introduces ADA text, in which case the returned value is positive, or it stops at the end of the comment, in which case the returned value is zero. The scanning also stops in anomalous situations when the comment doesn't end or when it contains an illegal use of @. One should call *skip_comment*(1) when beginning to scan a comment.

⟨ Specification of procedures visible from *lexical_scanning* 92 ⟩ +≡

```
function skip_comment(bal : eight_bits)return eight_bits;
```

107. ⟨ Procedures in *lexical_scanning* 85 ⟩ +≡

```
function skip_comment(bal : eight_bits)return eight_bits is    { skips TEX code in comments }
  b : eight_bits ← bal;  { depth of brace nesting }
  c : ASCII_code;  { the current character }
begin
  loop
    if loc > limit then
      get_line;
      if input_has_ended then
        return 0;  { an error message will occur in phase two }
      end if;
    end if;
    c ← buffer(loc); incr(loc);
    if c = [|] then
      return b;
    end if;
    ⟨ Do special things when c = [@], [\], [{], [}]; return at end 108 ⟩;
  end loop;
end skip_comment;
```

108. ⟨ Do special things when $c = [@], [\backslash], [\{], [\}]$; **return** at end 108 ⟩ ≡

```
if c = [@] then
  c ← buffer(loc);
  if (c ≠ [␣]) ∧ (c ≠ tab_mark) ∧ (c ≠ [*]) then
    incr(loc);
  else
    decr(loc); return 0;  { an error message will occur in phase two }
  end if;
elsif (c = [\]) ∧ (buffer(loc) ≠ [@]) then
  incr(loc);
elsif c = [{] then
  incr(b);
elsif c = [}] then
  decr(b);
  if b = 0 then
    return 0;
  end if;
end if
```

This code is used in section 107.

109. Inputting the next token. As stated above, AWEAVE's most interesting lexical scanning routine is the *get_next* function that inputs the next token of ADA input. However, *get_next* is not especially complicated.

The result of *get_next* is either an ASCII code for some special character, or it is a special code representing a pair of characters (e.g., '`:=`' or '`..`'), or it is the numeric value computed by the *control_code* procedure, or it is one of the following special codes:

exponent: The '`E`' in a real constant.

identifier: In this case the global variables *id_first* and *id_loc* will have been set to the appropriate values needed by the *id_lookup* routine.

str: In this case the global variables *id_first* and *id_loc* will have been set to the beginning and ending-plus-one locations in the buffer. The string ends with the first reappearance of its initial delimiter; thus, for example,

```
"This is ""not"" a single string"
```

will be treated as three consecutive strings, the first being `"This is"`.

There are some special cases that need to be considered:

character_tok is returned if a character literal, e.g. `´a´`, is sensed and *id_first* and *id_loc* are set appropriate as described for *str*.

based literals: Between two `#` signs characters are treated as digits; this is done be means of the global variable *scanning_based_number*.

Furthermore, some of the control codes cause *get_next* to take additional actions:

xref_roman, ..., *TeX_string*: The values of *id_first* and *id_loc* will be set so that the string in question appears in *buffer*(*id_first* .. (*id_loc* − 1)).

module_name: In this case the global variable *cur_module* will point to the *byte_start* entry for the module name that has just been scanned.

output_tok: The file name is treated the same way as decribed above for *str*.

ascii_tok: The ASCII code constant is treated the same way as described above for *character_tok*.

If *get_next* sees '`@!`' or '`@?`', it increments respectively decrements *xref_switch* by *def_flag*, supposed that $0 \le$ *xref_switch* $\le$ *body_flag* afterwards, and goes on to the next token.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
  exponent : **constant** *eight_bits* ← 201_8; { `E` or `e` following a digit }
  str : **constant** *eight_bits* ← 202_8; { ADA string }
  identifier : **constant** *eight_bits* ← 203_8; { ADA identifier or reserved word }
  cur_module : *name_pointer*; { name of module just scanned }
  scanning_based_number : BOOLEAN ← FALSE; { are we scanning a based number? }

110. As one might expect, *get_next* consists mostly of a big switch that branches to the various special cases that can arise. Some care is needed for the handling of ADA's based literals.

⟨ Specification of procedures visible from *lexical_scanning* 92 ⟩ +≡
  function *get_next* **return** *eight_bits*;

111. ⟨ Procedures in *lexical_scanning* 85 ⟩ +≡

```
function get_next return eight_bits is   { produces the next input token }
  c : eight_bits;  { the current character }
  d : eight_bits;  { the next character }
  j, k : long_name_index;  { indices into mod_text }
begin
⟨⟨restart⟩⟩ if loc > limit then
    get_line;
    if input_has_ended then
      return new_module;
    elsif scanning_based_number then
      err_print("!␣Improper␣based␣number,␣#␣sign␣missing");
          { a based literal should fit in one line }
    end if;
  end if;
  c ← buffer(loc); incr(loc);
  if scanning_based_number then
    if c = [#] then
      scanning_based_number ← FALSE;
    end if;
    return c;
  end if;
  case c is
    when [#] ⇒
      if (loc > 1) and then (buffer(loc − 2) ∈ [0] .. [9]) then
        scanning_based_number ← TRUE;
      end if;
      return [#];
    when [A] .. [Z] | [a] .. [z] ⇒
      ⟨ Get an identifier 113 ⟩;
    when [´] ⇒
      ⟨ Get a character token 123 ⟩;
    when ["] ⇒
      ⟨ Get a string 114 ⟩;
    when [@] ⇒
      ⟨ Get control code and possible module name 115 ⟩;
    ⟨ Compress two-symbol combinations like ':=' 112 ⟩
    when [␣] | tab_mark ⇒
      goto restart;  { ignore spaces and tabs }
    when others ⇒
      null;
  end case;
  return c;
end get_next;
```

112.

```
define compress(#) ≡
        if loc ≤ limit then
          c ← #; incr(loc);
        end if
⟨Compress two-symbol combinations like ':=' 112⟩ ≡
when [.] ⇒
  if buffer(loc) = [.] then
    compress(double_dot);
  end if;
when [:] ⇒
  if buffer(loc) = [=] then
    compress(left_arrow);
  end if;
when [=] ⇒
  if buffer(loc) = [=] then
    compress(equivalence_sign);
  elsif buffer(loc) = [>] then
    compress(right_arrow);
  end if;
when [>] ⇒
  if buffer(loc) = [=] then
    compress(greater_or_equal);
  elsif buffer(loc) = [>] then
    compress(right_label_bracket);
  end if;
when [<] ⇒
  if buffer(loc) = [=] then
    compress(less_or_equal);
  elsif buffer(loc) = [<] then
    compress(left_label_bracket);
  elsif buffer(loc) = [>] then
    compress(box_sign);
  end if;
when [/] ⇒
  if buffer(loc) = [=] then
    compress(not_equal);
  end if;
when [*] ⇒
  if buffer(loc) = [*] then
    compress(double_star);
  end if;
```

This code is used in section 111.

113. ⟨ Get an identifier 113 ⟩ ≡

```
if (((c = [E]) ∨ (c = [e])) ∧ (loc > 1)) and then ((buffer(loc − 2) ∈ [0] .. [9]) ∨ buffer(loc − 2) = [#]) then
  c ← exponent;
end if;
if c ≠ exponent then
  decr(loc); id_first ← loc;
  loop
    incr(loc); d ← buffer(loc);
    exit when (d ∉ [0] .. [9]) ∧ (d ∉ [A] .. [Z]) ∧ (d ∉ [a] .. [z]) ∧ (d ≠ [_]);
  end loop;
  c ← identifier; id_loc ← loc;
end if
```

This code is used in section 111.

114. A string that starts and ends with double quote marks is scanned by the following piece of the program.

⟨ Get a string 114 ⟩ ≡

```
id_first ← loc − 1;
loop
  d ← buffer(loc); incr(loc);
  if loc > limit then
    err_print("!␣String␣constant␣didn´t␣end");
    loc ← limit; d ← c;
  end if;
  exit when d = c;
end loop;
id_loc ← loc; c ← str
```

This code is used in section 111.

115. After an @ sign has been scanned, the next character tells us whether there is more work to do.

```
⟨Get control code and possible module name 115⟩ ≡
  c ← control_code(buffer(loc)); incr(loc);
  case c is
    when underline ⇒
      if xref_switch < body_flag then
        xref_switch ← xref_switch + def_flag;
      end if;
      goto restart;
    when no_underline ⇒
      if xref_switch > 0 then
        xref_switch ← xref_switch − def_flag;
      end if;
      goto restart;
    when xref_roman .. TeX_string ⇒
      ⟨Scan to the next @> 121⟩;
    when module_name ⇒
      ⟨Scan the module name and make cur_module point to it 116⟩;
    when verbatim | output_tok ⇒
      ⟨Scan a verbatim or output string 122⟩;
    when ascii_tok ⇒
      ⟨Scan an ASCII token 124⟩;
    when others ⇒
      null;
  end case
```

This code is used in section 111.

116. The occurrence of a module name sets *xref_switch* to zero, because the module name might (for example) follow **type**.

```
⟨Scan the module name and make cur_module point to it 116⟩ ≡
  ⟨Put module name into mod_text(1 .. k) 118⟩;
  if (k > 3) and then (mod_text(k − 2 .. k) = ([.], [.], [.])) then
    cur_module ← prefix_lookup(k − 3);
  else
    cur_module ← mod_lookup(k);
  end if;
  xref_switch ← 0
```

This code is used in section 115.

117. Module names are placed into the *mod_text* array with consecutive spaces, tabs, and carriage-returns replaced by single spaces. There will be no spaces at the beginning or the end. (We set *mod_text*(0) ← [␣] to facilitate this, since the *mod_lookup* routine uses *mod_text*(1) as the first character of the name.)

```
⟨Set initial values 19⟩ +≡
  mod_text(0) ← [␣];
```

118. ⟨ Put module name into *mod_text*(1 . . *k*) 118 ⟩ ≡

```
k ← 0;
loop
  if loc > limit then
    get_line;
    if input_has_ended then
      err_print("!␣Input␣ended␣in␣section␣name");
      loc ← 1;
      exit;
    end if;
  end if;
  d ← buffer(loc); ⟨ If end of name, exit 119 ⟩;
  incr(loc);
  if k < longest_name − 1 then
    incr(k);
  end if;
  if (d = [␣]) ∨ (d = tab_mark) then
    d ← [␣];
    if mod_text(k − 1) = [␣] then
      decr(k);
    end if;
  end if;
  mod_text(k) ← d;
end loop;
⟨ Check for overlong name 120 ⟩;
if (mod_text(k) = [␣]) ∧ (k > 0) then
  decr(k);
end if
```

This code is used in section 116.

119. ⟨ If end of name, **exit** 119 ⟩ ≡

```
if d = [@] then
  d ← buffer(loc + 1);
  if d = [>] then
    loc ← loc + 2;
    exit;
  end if;
  if (d = [␣]) ∨ (d = tab_mark) ∨ (d = [*]) then
    err_print("!␣Section␣name␣didn´t␣end");
    exit;
  end if;
  incr(k); mod_text(k) ← [@]; incr(loc);   { now d = buffer(loc) again }
end if
```

This code is used in section 118.

120. ⟨ Check for overlong name 120 ⟩ ≡

```
if k ≥ longest_name − 2 then
  print_nl("!␣Section␣name␣too␣long:␣");
  for i ∈ 1 . . 25 loop
    print(xchr(mod_text(i)));
  end loop;
  print("...");  mark_harmless;
end if
```

This code is used in section 118.

121. ⟨ Scan to the next @> 121 ⟩ ≡

```
id_first ← loc;  buffer(limit + 1) ← [@];
while buffer(loc) ≠ [@] loop
  incr(loc);
end loop;
id_loc ← loc;
if loc > limit then
  err_print("!␣Control␣text␣didn´t␣end");
  loc ← limit;
else
  loc ← loc + 2;
  if buffer(loc − 1) ≠ [>] then
    err_print("!␣Control␣codes␣are␣forbidden␣in␣control␣text");
  end if;
end if
```

This code is used in section 115.

122. A verbatim ADA string respectively an output string will be treated like ordinary strings, but with no surrounding delimiters. At the present point in the program we have *buffer*(*loc* − 1) = *verbatim* or *buffer*(*loc* − 1) = *output_tok*; we must set *id_first* to the beginning of the string itself, and *id_loc* to its ending-plus-one location in the buffer. We also set *loc* to the position just after the ending delimiter.

⟨ Scan a verbatim or output string 122 ⟩ ≡

```
id_first ← loc;  incr(loc);  buffer(limit + 1) ← [@];  buffer(limit + 2) ← [>];
while (buffer(loc) ≠ [@]) ∨ (buffer(loc + 1) ≠ [>]) loop
  incr(loc);
end loop;
if loc ≥ limit then
  err_print("!␣Verbatim␣string␣didn´t␣end");
end if;
id_loc ← loc;  loc ← loc + 2
```

This code is used in section 115.

123. Getting a character token and getting an ASCII token is essentially the same: *id_first* is set to the position of the first '´' and *id_loc* to the position after the final '´'.

⟨ Get a character token 123 ⟩ ≡
  id_first ← *loc* − 1;
  if *buffer*(*loc* + 1) = [´] **then**
    loc ← *loc* + 2; *id_loc* ← *loc*; *c* ← *character_tok*;
  elsif *buffer*(*loc* . . *loc* + 2) = ([@], [@], [´]) **then**
    loc ← *loc* + 3; *id_loc* ← *loc*; *c* ← *character_tok*;
  end if

This code is used in section 111.

124. ⟨ Scan an ASCII token 124 ⟩ ≡
  id_first ← *loc* − 1; *incr*(*loc*);
  if *buffer*(*loc*) ≠ [´] **then**
    if *buffer*(*loc* − 1 . . *loc*) ≠ ([@], [@]) **then**
      err_print("!␣Improper␣ASCII-code␣sequence,␣´␣sign␣missing");
    else
      incr(*loc*);
    end if;
  end if;
  incr(*loc*); *id_loc* ← *loc*

This code is used in section 115.

125. Phase one processing. We now have accumulated enough subroutines to make it possible to carry out AWEAVE's first pass over the source file. If everything works right, both phase one and phase two of AWEAVE will assign the same numbers to modules, and these numbers will agree with what ATANGLE does.

The global variable *next_control* often contains the most recent output of *get_next*; in interesting cases, this will be the control code that ended a module or part of a module.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
  next_control : *eight_bits*; { control code waiting to be acting upon }

126. The overall processing strategy in phase one has the following straightforward outline.

⟨ Phase I: Read all the user's text and store the cross references 126 ⟩ ≡
  phase_one ← TRUE; *phase_three* ← FALSE; *reset_input*; *module_count* ← 0; *skip_limbo*;
  change_exists ← FALSE;
  while ¬*input_has_ended* **loop**
    ⟨ Store cross reference data for the current module 127 ⟩;
  end loop;
  changed_module(*module_count*) ← *change_exists*; { the index changes if anything does }
  phase_one ← FALSE; { prepare for second phase }
  ⟨ Print error messages about unused or undefined module names 138 ⟩

This code is used in section 302.

127. ⟨ Store cross reference data for the current module 127 ⟩ ≡
  incr(*module_count*);
  if *module_count* = *max_modules* **then**
    overflow("section␣number");
  end if;
  changed_module(*module_count*) ← FALSE; { it will become TRUE if any line changes }
  if *buffer*(*loc* − 1) = [*] **then**
    print("*"); *print_int*(*module_count*, 1); *update_terminal*; { print a progress report }
  end if;
  ⟨ Store cross references in the TEX part of a module 132 ⟩;
  ⟨ Store cross references in the definition part of a module 133 ⟩;
  ⟨ Store cross references in the ADA part of a module 135 ⟩;
  if *changed_module*(*module_count*) **then**
    change_exists ← TRUE;
  end if

This code is used in section 126.

128. The *ADA_xref* subroutine stores references to identifiers in ADA text material beginning with the current value of *next_control* and continuing until *next_control* is '{' or '|', or until the next "milestone" is passed (i.e., *next_control* ≥ *format*). If *next_control* ≥ *format* when *ADA_xref* is called, nothing will happen; but if *next_control* = [|] upon entry, the procedure assumes that this is the '|' preceding ADA text that is to be processed.

The program uses the fact that our internal code numbers satisfy the relations *xref_roman* = *identifier* + *roman*, *xref_wildcard* = *identifier* + *wildcard*, ..., *xref_typewriter* = *identifier* + *typewriter*, and *normal* = 0.

After **entry**, **for**, **function**, **package**, **procedure**, **subtype**, **task**, and **type** *xref_switch* is set to *def_flag*; after **accept** and **body** it is set to *body_flag*.

⟨ Specification of procedures visible from *first_phase* 128 ⟩ ≡
  procedure *ADA_xref*;

See also section 301.

This code is used in section 7*.

129. ⟨ Procedures in *first_phase* 129 ⟩ ≡

```
procedure ADA_xref is   { makes cross references for ADA identifiers }
  p : name_pointer;   { a referenced name }
  i : INTEGER;   { temporary ilk code of p }
begin
  while next_control < format loop
    if next_control ∈ identifier .. xref_predefined then
      p ← id_lookup(next_control − identifier);
      if (ilk(p) = body_like) ∨ (ilk(p) = type_like) then
        xref_switch ← 0;   { no index entry for body and type }
      end if;
      new_xref(p);
      if (ilk(p) = proc_like) ∨ (ilk(p) = type_like) then
        xref_switch ← def_flag;   { implied def_flag }
      elsif (ilk(p) = accept_like) ∨ (ilk(p) = body_like) then
        xref_switch ← body_flag;   { implied body_flag }
      end if;
    elsif next_control = xref_reserved then
      p ← id_lookup(normal); i ← ilk(p); ilk(p) ← normal; new_xref(p); ilk(p) ← i;
    end if;
    next_control ← get_next;
    if (next_control = [|]) ∨ (next_control = [{]) then
      return;
    end if;
  end loop;
end ADA_xref;
```

See also sections 130, 137, and 302.

This code is used in section 7*.

130. The *outer_xref* subroutine is like *ADA_xref* but it begins with *next_control* ≠ [|] and ends with *next_control* ≥ *format*. Thus, it handles ADA text with embedded comments.

⟨ Procedures in *first_phase* 129 ⟩ +≡

```
procedure outer_xref is   { extension of ADA_xref }
  bal : eight_bits;   { brace level in comment }
begin
  while next_control < format loop
    if next_control ≠ [{] then
      ADA_xref;
    else
      bal ← skip_comment(1); next_control ← [|];
      while bal > 0 loop
        ADA_xref;
        if next_control = [|] then
          bal ← skip_comment(bal);
        else
          bal ← 0;   { an error will be reported in phase two }
        end if;
      end loop;
    end if;
  end loop;
end outer_xref;
```

131. During the definition and ADA parts of a module, cross references are made for all identifiers except reserved words; however, the identifiers in a format definition are referenced even if they are reserved. The TEX code in comments is, of course, ignored, except for ADA portions enclosed in | ... |; the text of a module name is skipped entirely, even if it contains | ... | constructions.

The variables *lhs* and *rhs* point to the respective identifiers involved in a format definition.

Additionally to the points just stated a reserved identifier is referenced by means of the '@r' control code; the variable *temp_ilk* contains the *ilk* of an identifier during the *new_xref* subroutine is executing, because we have to set the *ilk* to *normal* before the call.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
  lhs, *rhs* : *name_pointer*; { indices into *byte_start* for format identifiers }
  temp_ilk : *sixteen_bits*; { temporarily contains the *ilk* of *lhs* }

132. In the TEX part of a module, cross reference entries are made only for the identifiers in ADA texts enclosed in | ... |, or for control texts enclosed in @^ ... @> or @. ... @> or @: ... @> or @i ... @> or @p ... @> or @r ... @> or @b ... @> or @s ... @>.

⟨ Store cross references in the TEX part of a module 132 ⟩ ≡

```
loop
  next_control ← skip_TeX;
  case next_control is
    when underline ⇒
      if xref_switch < body_flag then
        xref_switch ← xref_switch + def_flag;
      end if;
    when no_underline ⇒
      if xref_switch > 0 then
        xref_switch ← xref_switch − def_flag;
      end if;
    when [|] ⇒
      ADA_xref;
    when xref_roman .. xref_predefined | module_name ⇒
      loc ← loc − 2; next_control ← get_next; { scan to @> }
      if next_control ≠ module_name then
        new_xref(id_lookup(next_control − identifier));
      end if;
    when xref_reserved ⇒
      loc ← loc − 2; next_control ← get_next; { scan to @> }
      lhs ← id_lookup(normal); temp_ilk ← ilk(lhs); { get the ilk }
      ilk(lhs) ← normal; new_xref(lhs); { make the cross reference }
      ilk(lhs) ← temp_ilk; { reset the ilk }
    when others ⇒
      null;
  end case;
  exit when next_control ≥ format;
end loop
```

This code is used in section 127.

133. When we get to the following code we have *next_control* ≥ *format*.

⟨ Store cross references in the definition part of a module 133 ⟩ ≡

```
while next_control ≤ definition loop   { format or definition }
  xref_switch ← def_flag;   { implied @! }
  if next_control = definition then
    next_control ← get_next;
  else
    ⟨ Process a format definition 134 ⟩;
  end if;
  outer_xref;
end loop
```

This code is used in section 127.

134. Error messages for improper format definitions will be issued in phase two. Our job in phase one is to define the *ilk* of a properly formatted identifier, and to fool the *new_xref* routine into thinking that the identifier on the right-hand side of the format definition is not a reserved word.

⟨ Process a format definition 134 ⟩ ≡

```
next_control ← get_next;
if next_control = identifier then
  lhs ← id_lookup(normal); ilk(lhs) ← normal; new_xref(lhs); next_control ← get_next;
  if next_control = equivalence_sign then
    next_control ← get_next;
    if next_control = identifier then
      rhs ← id_lookup(normal); ilk(lhs) ← ilk(rhs); ilk(rhs) ← normal; new_xref(rhs);
      ilk(rhs) ← ilk(lhs); next_control ← get_next;
    end if;
  end if;
end if
```

This code is used in section 133.

135. Finally, when the TEX and definition parts have been treated, we have *next_control* ≥ *begin_ada*.

⟨ Store cross references in the ADA part of a module 135 ⟩ ≡

```
if next_control ≤ module_name then   { begin_ada or module_name }
  if next_control = begin_ada then
    mod_xref_switch ← 0;
  else
    mod_xref_switch ← def_flag;
  end if;
  loop
    if next_control = module_name then
      new_mod_xref(cur_module);
    end if;
    next_control ← get_next;
    outer_xref;
    exit when next_control > module_name;
  end loop;
end if
```

This code is used in section 127.

136. After phase one has looked at everything, we want to check that each module name was both defined and used. The variable *cur_xref* will point to cross references for the current module name of interest.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
  cur_xref : *xref_number*; { temporary cross reference pointer }

137. The following recursive procedure walks through the tree of module names and prints out anomalies.

```
⟨ Procedures in first_phase 129 ⟩ +≡
  procedure mod_check(p : name_pointer) is    { print anomalies in subtree p }
  begin
    if p > 0 then
      mod_check(llink(p));
      cur_xref ← xref(p);
      if num(cur_xref) < def_flag then
        print_nl("!␣Never␣defined:␣<"); print_id(p); print(">"); mark_harmless;
      end if;
      while num(cur_xref) ≥ def_flag loop
        cur_xref ← xlink(cur_xref);
      end loop;
      if cur_xref = 0 then
        print_nl("!␣Never␣used:␣<"); print_id(p); print(">"); mark_harmless;
      end if;
      mod_check(rlink(p));
    end if;
  end mod_check;
```

138. ⟨ Print error messages about unused or undefined module names 138 ⟩ ≡ *mod_check*(*root*)

This code is used in section 126.

139. Low-level output routines. The TEX output is supposed to appear in lines at most *line_length* characters long, so we place it into an output buffer. During the output process, *out_line* will hold the current line number of the line about to be output.

```
⟨ Types and objects visible from data_structures 14 ⟩ +≡
  subtype length_index is INTEGER range 0 .. line_length;
  out_buf : array (length_index) of ASCII_code;  { assembled characters }
  out_ptr : length_index;  { number of characters in out_buf }
  out_line : INTEGER;  { coordinates of next line to be output }
```

140. The *flush_buffer* routine empties the buffer up to a given breakpoint, and moves any remaining characters to the beginning of the next line. If the *per_cent* parameter is TRUE, a [%] is appended to the line that is being output; in this case the breakpoint b should be strictly less than *line_length*. If the *per_cent* parameter is FALSE, trailing blanks are suppressed. The characters emptied from the buffer form a new line of output.

```
⟨ Specification of procedures visible from output 140 ⟩ ≡
  procedure flush_buffer(b : eight_bits; per_cent : BOOLEAN);
```

See also sections 142, 147, 151, 154, 156, 159, and 162.

This code is used in section 8*.

141.

```
⟨ Procedures in output 141 ⟩ ≡
  procedure flush_buffer(b : eight_bits; per_cent : BOOLEAN) is
        { outputs out_buf(1 .. b), where b ≤ out_ptr }
    j : length_index ← b;
  begin
    if ¬per_cent then  { remove trailing blanks }
      loop
        exit when (j = 0) or else (out_buf(j) ≠ [␣]);
        decr(j);
      end loop;
    end if;
    for k ∈ 1 .. j loop
      TEXT_IO.PUT(TeX_file, xchr(out_buf(k)));
    end loop;
    if per_cent then
      TEXT_IO.PUT(TeX_file, xchr([%]));
    end if;
    TEXT_IO.NEW_LINE(TeX_file); incr(out_line);
    if b < out_ptr then
      out_buf(1 .. out_ptr − b) ← out_buf(b + 1 .. out_ptr);
    end if;
    out_ptr ← out_ptr − b;
  end flush_buffer;
```

See also sections 143, 148, 152, 155, 157, 160, and 163.

This code is used in section 8*.

142. When we are copying TEX source material, we retain line breaks that occur in the input, except that an empty line is not output when the TEX source line was nonempty. For example, a line of the TEX file that contains only an index cross-reference entry will not be copied. The *finish_line* routine is called just before *get_line* inputs a new line, and just after a line break token has been emitted during the output of translated ADA text.

⟨ Specification of procedures visible from *output* 140 ⟩ +≡
  procedure *finish_line*;

143. ⟨ Procedures in *output* 141 ⟩ +≡
```
procedure finish_line is   { do this at the end of a line }
begin
  if out_ptr > 0 then
    flush_buffer(out_ptr, FALSE);
  else
    for k ∈ 0 .. limit loop
      if (buffer(k) ≠ [␣]) ∧ (buffer(k) ≠ tab_mark) then
        return;
      end if;
    end loop;
    flush_buffer(0, FALSE);
  end if;
end finish_line;
```

144. In particular, the *finish_line* procedure is called near the very beginning of phase two. We initialize the output variables in a slightly tricky way so that the first line of the output file will be '`\input awebmac`'.

⟨ Set initial values 19 ⟩ +≡
  out_ptr ← 1; *out_line* ← 1; *out_buf*(1) ← [c]; TEXT_IO.PUT(*TeX_file*, "\input␣awebma");

145. When we wish to append the character c to the output buffer, we write '*out1*(c)'; this will cause the buffer to be emptied if it was already full. Similarly, '*out2*(c_1)(c_2)' appends a pair of characters. A line break will occur at a space or after a single-nonletter TEX control sequence.

```
define out1(#) ≡
        if out_ptr = line_length then
          break_out;
        end if;
        incr(out_ptr); out_buf(out_ptr) ← #
define out2(#) ≡ out1(#); out1
define out3(#) ≡ out1(#); out2
define out4(#) ≡ out1(#); out3
define out5(#) ≡ out1(#); out4
```

146. The *break_out* routine is called just before the output buffer is about to overflow. To make this routine a little faster, we initialize position 0 of the output buffer to '\'; this character isn't really output.

⟨ Set initial values 19 ⟩ +≡
  out_buf(0) ← [\];

147. A long line is broken at a blank space or just before a backslash that isn't preceded by another backslash. In the latter case, a [%] is output at the break.

⟨ Specification of procedures visible from *output* 140 ⟩ +≡
  procedure *break_out*;

148. ⟨ Procedures in *output* 141 ⟩ +≡

```
procedure break_out is   { finds a way to break the output line }
  k : length_index ← out_ptr;   { index into out_buf }
  c, d : ASCII_code;   { characters from the buffer }
begin
  loop
    if k = 0 then
      ⟨ Print warning message, break the line, return 149 ⟩;
    end if;
    d ← out_buf(k);
    if d = [␣] then
      flush_buffer(k, FALSE); return;
    end if;
    if (d = [\]) ∧ (out_buf(k − 1) ≠ [\]) then   { in this case k > 1 }
      flush_buffer(k − 1, TRUE); return;
    end if;
    decr(k);
  end loop;
end break_out;
```

149. We get to this module only in unusual cases that the entire output line consists of a string of backslashes followed by a string of nonblank non-backslashes. In such cases it is almost always safe to break the line by putting a [%] just before the last character.

⟨ Print warning message, break the line, **return** 149 ⟩ ≡

```
print_nl("!␣Line␣had␣to␣be␣broken␣(output␣l."); print_int(out_line, 1); print_ln("):");
for j ∈ 1 .. out_ptr − 1 loop
  print(xchr(out_buf(j)));
end loop;
new_ln; mark_harmless; flush_buffer(out_ptr − 1, TRUE); return
```

This code is used in section 148.

150. Here is a procedure that outputs a module number in decimal notation.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡

```
dig : array (0 .. 4) of INTEGER range 0 .. 9;   { digits to output }
```

151. The number to be converted by *out_mod* is known to be less than *def_flag*, so it cannot have more than five decimal digits. If the module is changed, we output '`\*`' just after the number and if $c > 1$ we output '`(`c`)`'.

⟨ Specification of procedures visible from *output* 140 ⟩ +≡

```
procedure out_mod(m : INTEGER; c : INTEGER ← 1);
```

152. ⟨ Procedures in *output* 141 ⟩ +≡

```
procedure out_mod(m : INTEGER; c : INTEGER ← 1) is    { output a module number }
  k : INTEGER range 0 .. 5;   { index into dig }
  a : INTEGER;   { accumulator }
begin
  a ← m; ⟨ Output the digits of a 153 ⟩;
  if changed_module(m) then
    out2([\])([*]);
  end if;
  if c > 1 then
    out1([(]); a ← c; ⟨ Output the digits of a 153 ⟩;
    out1([)]);
  end if;
end out_mod;
```

153. Here we output the digits of a using the *dig* array.

⟨ Output the digits of a 153 ⟩ ≡

```
k ← 0;
loop
  dig(k) ← a mod 10; a ← a/10; incr(k);
  exit when a = 0;
end loop;
loop
  decr(k); out1(dig(k) + [0]);
  exit when k = 0;
end loop
```

This code is used in section 152(2).

154. The *out_name* subroutine is used to output an identifier or index entry, enclosing it in braces.

⟨ Specification of procedures visible from *output* 140 ⟩ +≡

```
procedure out_name(p : name_pointer);
```

155. ⟨ Procedures in *output* 141 ⟩ +≡

```
procedure out_name(p : name_pointer) is    { outputs a name }
  w : w_range;   { row of byte_mem }
begin
  out1([{]); w ← p mod ww;
  for k ∈ byte_start(p) .. byte_start(p + ww) − 1 loop
    if byte_mem(w, k) = [_] then
      out1([\]);
    end if;
    out1(byte_mem(w, k));
  end loop;
  out1([}]);
end out_name;
```

156. Routines that copy TEX material. During phase two, we use the subroutines *copy_limbo*, *copy_TeX*, and *copy_comment* in place of the analogous *skip_limbo*, *skip_TeX*, and *skip_comment* that were used in phase one.

The *copy_limbo* routine, for example, takes TEX material that is not part of any module and transcribes it almost verbatim to the output file. No '@' signs should occur in such material except in '@@' pairs; such pairs are replaced by singletons.

```
⟨Specification of procedures visible from output 140⟩ +≡
  procedure copy_limbo;
```

157.
```
⟨Procedures in output 141⟩ +≡
  procedure copy_limbo is   {copy TEX code until the next module begins}
    c : ASCII_code;   {character following @ sign}
  begin
    loop
      if loc > limit then
        finish_line; get_line;
        if input_has_ended then
          return;
        end if;
      else
        buffer(limit + 1) ← [@];
        ⟨Copy up to control code, return if finished 158⟩;
      end if;
    end loop;
  end copy_limbo;
```

158.
```
⟨Copy up to control code, return if finished 158⟩ ≡
  while buffer(loc) ≠ [@] loop
    out1(buffer(loc)); incr(loc);
  end loop;
  if loc ≤ limit then
    loc ← loc + 2; c ← buffer(loc − 1);
    if (c = [␣]) ∨ (c = tab_mark) ∨ (c = [*]) then
      return;
    end if;
    if (c ≠ [z]) ∧ (c ≠ [Z]) then
      out1([@]);
      if c ≠ [@] then
        err_print("!␣Double␣@␣required␣outside␣of␣sections");
      end if;
    end if;
  end if
```

This code is used in section 157.

159. The *copy_TeX* routine processes the TEX code at the beginning of a module; for example, the words you are now reading were copied in this way. It returns the next control code or '|' found in the input.

```
⟨Specification of procedures visible from output 140⟩ +≡
  function copy_TeX return eight_bits;
```

160. ⟨ Procedures in *output* 141 ⟩ +≡

```
function copy_TeX return eight_bits is    { copy pure TEX material }
  c : eight_bits;  { control code found }
begin
  loop
    if loc > limit then
      finish_line; get_line;
      if input_has_ended then
        return new_module;
      end if;
    end if;
    buffer(limit + 1) ← [@];
    ⟨ Copy up to '|' or control code, return c if finished 161 ⟩;
  end loop;
end copy_TeX;
```

161. We don't copy spaces or tab marks into the beginning of a line. This makes the test for empty lines in *finish_line* work.

⟨ Copy up to '|' or control code, **return** c if finished 161 ⟩ ≡

```
loop
  c ← buffer(loc); incr(loc);
  if c = [|] then
    return [|];
  end if;
  if c ≠ [@] then
    out1(c);
    if (out_ptr = 1) ∧ ((c = [␣]) ∨ (c = tab_mark)) then
      decr(out_ptr);
    end if;
  end if;
  exit when c = [@];
end loop;
if loc ≤ limit then
  c ← control_code(buffer(loc)); incr(loc); return c;
end if
```

This code is used in section 160.

162. The *copy_comment* uses and returns a brace-balance value, following the conventions of *skip_comment* above. Instead of copying the TEX material into the output buffer, this procedure copies it into the token memory. The abbreviation *app_tok*(t) is used to append token t to the current token list, and it also makes sure that it is possible to append at least one further token without overflow.

define *app_tok*(#) ≡

```
if tok_ptr + 2 > max_toks then
  overflow("token");
end if;
tok_mem(tok_ptr) ← #; incr(tok_ptr)
```

⟨ Specification of procedures visible from *output* 140 ⟩ +≡

```
function copy_comment(bal : eight_bits)return eight_bits;
```

163. ⟨ Procedures in *output* 141 ⟩ +≡

```
function copy_comment(bal : eight_bits)return eight_bits is   { copies TEX code in comments }
  c : ASCII_code;  { current character being copied }
  b : eight_bits ← bal;  { brace-balance value }
begin
  loop
    if loc > limit then
      get_line;
      if input_has_ended then
        err_print("!␣Input␣ended␣in␣mid-comment");
        loc ← 1; ⟨ Clear b and return 0 165 ⟩;
      end if;
    end if;
    c ← buffer(loc); incr(loc);
    if c = [|] then
      return b;
    end if;
    app_tok(c);
    ⟨ Copy special things when c = [@], [\], [{], [}]; return at end 164 ⟩;
  end loop;
end copy_comment;
```

164. ⟨ Copy special things when $c = [@], [\backslash], [\{], [\}]$; **return** at end 164 ⟩ ≡

```
if c = [@] then
  incr(loc);
  if buffer(loc − 1) ≠ [@] then
    err_print("!␣Illegal␣use␣of␣@␣in␣comment");
    loc ← loc − 2; decr(tok_ptr); ⟨ Clear b and return 0 165 ⟩;
  end if;
elsif (c = [\]) ∧ (buffer(loc) ≠ [@]) then
  app_tok(buffer(loc)); incr(loc);
elsif c = [{] then
  incr(b);
elsif c = [}] then
  decr(b);
  if b = 0 then
    return 0;
  end if;
end if
```

This code is used in section 163.

165. When the comment has terminated abruptly due to an error, we output enough right braces to keep TEX happy.

⟨ Clear b and **return** 0 165 ⟩ ≡

```
app_tok([␣]);  { this is done in case the previous character was '\' }
while b > 0 loop
  app_tok([}]); decr(b);
end loop;
return 0
```

This code is used in sections 163 and 164.

166. Parsing. The most intricate part of `AWEAVE` is its mechanism for converting ADA-like code into TEX code, and we might as well plunge into this aspect of the program now. A "bottom up" approach is used to parse the ADA-like material, since `AWEAVE` must deal with fragmentary constructions whose overall "part of speech" is not known.

At the lowest level, the input is represented as a sequence of entities that we shall call *scraps*, where each scrap of information consists of two parts, its *category* and its *translation*. The category is essentially a syntactic class, and the translation is a token list that represents TEX code. Rules of syntax and semantics tell us how to combine adjacent scraps into larger ones, and if we are lucky an entire ADA text that starts out as hundreds of small scraps will join together into one gigantic scrap whose translation is the desired TEX code. If we are unlucky, we will be left with several scraps that don't combine; their translations will simply be output, one by one.

The combination rules are given as context-sensitive productions that are applied from left to right. Suppose that we are currently working on the sequence of scraps $s_1\, s_2 \ldots s_n$. We try first to find the longest production that applies to an initial substring $s_1\, s_2 \ldots$; but if no such productions exist, we find to find the longest production applicable to the next substring $s_2\, s_3 \ldots$; and if that fails, we try to match $s_3\, s_4 \ldots$, etc.

A production applies if the category codes have a given pattern. For example, one of the productions is

$$\textit{open_scrap}\ \textit{decl} \ \rightarrow\ \textit{open_scrap}$$

and it means that two consecutive scraps whose respective categories are *open_scrap* and *decl* are converted to one scrap whose category is *open_scrap*. This production also has an associated rule that tells how to combine the translation parts:

$$O_2 = O_1\ \textit{pdollar}\ D\ \textit{pdollar}\ \backslash\texttt{,}\ \textit{opt}\ \texttt{5}$$

This means that the new *open_scrap* scrap has a translation O_2 composed of the translation O_1 of the original *open_scrap* scrap followed by *pdollar* followed by the translation D of the *decl* scrap followed by *pdollar* followed by '`\,`' followed by '*opt*' followed by '`5`'. (In the TEX file, this will specify an additional thin space after the *pdollar*, followed by an optional line break with penalty 50.) Translation rules use subscripts to distinguish between translations of scraps whose categories have the same initial letter; these subscripts are assigned from left to right.

There is also the production rule

$$\textit{loop_scrap} \ \rightarrow\ \textit{beginning}$$

(meaning that a **loop** begins a block-like ADA statement). Since productions are applied from left to right, this rule will be activated only if the *loop_scrap* is not preceded by scraps that match other productions.

The translation rule corresponding to *loop_scrap* $\rightarrow$ *beginning* is

$$B = L$$

but we shall not mention translation rules in the common case that the translation of the new scrap on the right-hand side is simply the concatenation of the disappearing scraps on the left-hand side.

167. Here is a list of the category codes that scraps can have.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
accept_scrap : **constant** *eight_bits* ← 1; { denotes **accept** }
and_scrap : **constant** *eight_bits* ← 2; { denotes **and** }
arrow : **constant** *eight_bits* ← 3; { denotes '⇒' }
base : **constant** *eight_bits* ← 4; { denotes '#' in based literals }
begin_label_scrap : **constant** *eight_bits* ← 5; { denotes '<<', i.e. the beginning of a label }
beginning : **constant** *eight_bits* ← 6; { denotes beginning of block, loop, etc. }
case_scrap : **constant** *eight_bits* ← 7; { denotes **case** }
clause : **constant** *eight_bits* ← 8;
{ denotes a clause in **case** statements or variant parts, i.e. **when** . . . ⇒ }
close_scrap : **constant** *eight_bits* ← 9; { denotes a right parenthesis after which indentation ends }
colon : **constant** *eight_bits* ← 10; { denotes a colon }
decl : **constant** *eight_bits* ← 11; { denotes a declaration }
digit_scrap : **constant** *eight_bits* ← 12; { denotes a digit—possibly (in based literals) a character }
else_scrap : **constant** *eight_bits* ← 13; { denotes a quantity starting an else-part }
end_label_scrap : **constant** *eight_bits* ← 14; { denotes '>>', i.e. the ending of a label }
ending : **constant** *eight_bits* ← 15; { denotes **end** }
exception_scrap : **constant** *eight_bits* ← 16; { denotes **exception** }
exp : **constant** *eight_bits* ← 17; { denotes the 'E' in a real literal }
follows : **constant** *eight_bits* ← 18; { denotes **is** and **do** }
generic_scrap : **constant** *eight_bits* ← 19; { denotes the beginning of a generic formal part }
gproc : **constant** *eight_bits* ← 20;
{ denotes **procedure** and **function** preceded by **with** in a generic formal part }
if_scrap : **constant** *eight_bits* ← 21; { denotes **if** and **elsif** }
in_scrap : **constant** *eight_bits* ← 22; { denotes **in** }
label : **constant** *eight_bits* ← 23; { denotes a complete label }
loop_scrap : **constant** *eight_bits* ← 24; { denotes **loop** }
math : **constant** *eight_bits* ← 25; { denotes a quantity to be used in TEX's math mode only }
mod_scrap : **constant** *eight_bits* ← 26; { denotes a module name }
new_scrap : **constant** *eight_bits* ← 27; { denotes **new** }
not_scrap : **constant** *eight_bits* ← 28; { denotes **not** }
open_scrap : **constant** *eight_bits* ← 29; { denotes a left parenthesis after which indentation begins }
period : **constant** *eight_bits* ← 30; { denotes a '.' }
private_scrap : **constant** *eight_bits* ← 31; { denotes **private** }
proc : **constant** *eight_bits* ← 32; { denotes **entry**, **function**, etc. }
record_scrap : **constant** *eight_bits* ← 33; { denotes **record** }
return_scrap : **constant** *eight_bits* ← 34; { denotes **return** and **exit** }
semi : **constant** *eight_bits* ← 35; { denotes a semicolon }
separate_scrap : **constant** *eight_bits* ← 36; { denotes **separate** }
stmt : **constant** *eight_bits* ← 37; { denotes a statement }
terminator : **constant** *eight_bits* ← 38;
{ denotes the ending of statements, blocks, declaration parts, etc. }
then_scrap : **constant** *eight_bits* ← 39; { denotes **then** }
when_scrap : **constant** *eight_bits* ← 40; { denotes **when** }
with_scrap : **constant** *eight_bits* ← 41; { denotes **use** and **with** }

168. The token lists for translated TEX output contain some special control symbols as well as ordinary characters. These control symbols are interpreted by AWEAVE before they are written to the output file.

pdollar denotes a possible $ sign in TEX output;

pspace denotes a possible blank space; two or more consecutive *pspace* symbols combine to a *sspace*;

sspace denotes a blank space in TEX output;

break_space denotes an optional line break or an en space;

force denotes a line break;

big_force denotes a line break with additional vertical space;

opt denotes an optional line break (with the continuation line indented two ems with respect to the normal starting position)—this code is followed by an integer n, and the break will occur with penalty $10n$;

backup denotes a backspace of one em;

cancel obliterates any *break_space* or *force* or *big_force* tokens that immediately precede or follow it and also cancels any *backup* tokens that follow it;

indent causes future lines to be indented one more em;

outdent causes future lines to be indented one less em.

All of these tokens, except *pdollar* and *sspace*, are removed from the TEX output that comes from ADA text between | ... | signs; *break_space* and *force* and *big_force* become single spaces in this mode. The translation of other ADA texts results in TEX control sequences \1, \2, \3, \4, \5, \6, \7 corresponding respectively to *indent*, *outdent*, *opt*, *backup*, *break_space*, *force*, and *big_force*. However, a sequence of consecutive '␣', *break_space*, *force*, and/or *big_force* tokens is first replaced by a single token (the maximum of the given ones). The *pdollar* token results in \?, supposed there are n consecutive *pdollar* tokens and $n \bmod 2 = 1$, i.e. n is odd. The *sspace* token results in \8.

The tokens *math_rel*, *math_bin*, *math_op* will be translated into \mathrel{, \mathbin{, and \mathop{, respectively. Other control sequences in the TEX output will be '\\{...}' surrounding normal identifiers, '\p{...}' surrounding predefined identifiers, '\&{...}' surrounding reserved words, '\.{...}' surrounding strings, '\C{...}*force*' surrounding comments, and '\X$n$:...\X' surrounding module names, where n is the module number.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
math_bin : **constant** *eight_bits* ← 204_8;
math_rel : **constant** *eight_bits* ← 205_8;
math_op : **constant** *eight_bits* ← 206_8;
pdollar : **constant** *eight_bits* ← 207_8; { possible dollar sign (\?) }
pspace : **constant** *eight_bits* ← 210_8; { possible space }
sspace : **constant** *eight_bits* ← 211_8; { sure space (\8) }
big_cancel : **constant** *eight_bits* ← 212_8; { like *cancel*, also overrides spaces }
cancel : **constant** *eight_bits* ← 213_8; { overrides *backup*, *break_space*, *force*, *big_force* }
indent : **constant** *eight_bits* ← 214_8; { one more tab (\1) }
outdent : **constant** *eight_bits* ← 215_8; { one less tab (\2) }
opt : **constant** *eight_bits* ← 216_8; { optional break in mid-statement (\3) }
backup : **constant** *eight_bits* ← 217_8; { stick out one unit to the left (\4) }
break_space : **constant** *eight_bits* ← 220_8; { optional break between statements (\5) }
force : **constant** *eight_bits* ← 221_8; { forced break between statements (\6) }
big_force : **constant** *eight_bits* ← 222_8; { forced break with additional space (\7) }
end_translation : **constant** *eight_bits* ← 223_8; { special sentinel token at end of list }

169. The raw input is converted into scraps according to the following table, which gives category codes followed by the translations. (The symbol '***' stands for '\&{identifier}', i.e., the identifier itself treated as a reserved word. In a few cases the category is given as '*comment*'; this is not an actual category code, it means that the translation will be treated as a comment, as explained below.)

/=	*math* : \I
<=	*math* : \L
>=	*math* : \G
:=	*math* : \K
==	*math* : \S
=>	*math* : \KR
**	*math* : \dst
<<	*math* : \BL
>>	*math* : \EL
<>	*math* : \BOX
" string "	*math* : *pdollar* \.{" modified string "} *pdollar*
´ character ´	*math* : *pdollar* \.{\´ modified character \´} *pdollar*
@= string @>	*math* : *pdollar* \={ modified string } *pdollar*
@~ string @>	*math* : *pdollar* \O{ modified string } *pdollar force*
# in based literal	*base* :
# otherwise	*math* : *pdollar* \# *pdollar*
$	*math* : *pdollar* \$ *pdollar*
_	*math* : *pdollar* _ *pdollar*
%	*math* : *pdollar* \% *pdollar*
^	*math* : *pdollar* \^ *pdollar*
&	*math* : *math_bin* \.\&}
!	*math* : *math_rel* \.|}
(	*open_scrap* : (
)	*close_scrap* :)
*	*math* : \ast
,	*math* : , *opt* 9
..	*math* : \to
.	*period* : .
:	*colon* : :
;	*semi* : ;
predefined identifier	*math* : *pspace pdollar* \p{ identifier } *pdollar pspace*
normal identifier	*math* : *pspace pdollar* \\{ identifier } *pdollar pspace*
E in constant	*exp* : \E{
digit d, not in based number	*digit_scrap* : *pdollar d pdollar*
digit d, in based number	*digit_scrap* : d
other character c	*math* : c
abort	*math* : *pspace pdollar* *** *pdollar sspace*
abs	*math* : *pspace pdollar* *** *pdollar sspace*
accept	*accept_scrap* : *** *sspace*
access	*math* : *pspace pdollar* *** *pdollar sspace*
all	*math* : *pspace pdollar* *** *pdollar pspace*
and	*and_scrap* : \W
array	*open_scrap* : *** *sspace*
at	*colon* : *sspace* *** *sspace*
begin	*beginning* : ***
body	*math* : *pdollar* *** *pdollar pspace*
case	*case_scrap* : ***
constant	*math* : *pspace pdollar* *** *pdollar sspace*

declare	*decl* : *force* *** *indent force*
delay	*math* : *pspace pdollar* *** *pdollar sspace*
delta	*math* : *pspace pdollar* *** *pdollar sspace*
digits	*math* : *pspace pdollar* *** *pdollar sspace*
do	*follows* : *sspace* *** *sspace*
else	*else_scrap* : ***
elsif	*if_scrap* : *backup* ***
end	*ending* : ***
entry	*proc* : *** *sspace*
exception	*exception_scrap* : ***
exit	*return_scrap* : ***
for	*math* : *pspace pdollar* *** *pdollar sspace*
function	*proc* : *** *sspace*
generic	*generic_scrap* : ***
goto	*math* : *pspace pdollar* *** *pdollar sspace*
if	*if_scrap* : *** *indent*
in	*in_scrap* : \in
is	*follows* : *sspace* *** *sspace*
limited	*math* : *pspace pdollar* *** *pdollar sspace*
loop	*loop_scrap* : ***
mod	*math* : *math_bin* *** }
new	*new_scrap* : *pspace* *** *pspace*
not	*not_scrap* : \R
null	*math* : *pspace pdollar* *** *pdollar pspace*
of	*close_scrap* : *sspace* *** *sspace*
or	*else_scrap* : ***
others	*math* : *pspace pdollar* *** *pdollar sspace*
out	*math* : *pspace pdollar* *** *pdollar sspace*
package	*proc* : *** *sspace*
pragma	*proc* : *** *sspace*
private	*private_scrap* : ***
procedure	*proc* : *** *sspace*
raise	*math* : *pspace pdollar* *** *pdollar sspace*
range	*math* : *sspace pdollar* *** *pdollar sspace*
record	*record_scrap* : ***
rem	*math* : *math_bin* *** }
renames	*math* : *pspace pdollar* *** *pdollar sspace*
return	*return_scrap* : ***
reverse	*math* : *pspace pdollar* *** *pdollar sspace*
select	*beginning* : ***
separate	*separate_scrap* : ***
subtype	*math* : *pdollar* *** *pdollar sspace*
task	*proc* : *** *sspace*
terminate	*math* : *pspace pdollar* *** *pdollar pspace*
then	*then_scrap* : *sspace* ***
type	*math* : *pdollar* *** *pdollar sspace*
use	*with_scrap* : *pspace* *** *sspace*
when	*when_scrap* : *** *sspace*
while	*math* : *pspace pdollar* *** *pdollar sspace*
with	*with_scrap* : *pspace* *** *sspace*
xor	*math* : *math_bin* *** }
@´*c*´	*math* : *pdollar* \H{*c*} *pdollar*

`@\`	*math* : `\)`
`@,`	*math* : `\,`
`@t` stuff `@>`	*math* : *pdollar* `\hbox{` stuff `}` *pdollar*
`@<` module `@>`	*mod_scrap* : `\X`*n*`:` module `\X`
`@#`	*comment* : *big_force*
`@/`	*comment* : *force*
`@|`	*math* : *opt* `0`
`@+`	*comment* : *big_cancel* `\8` *big_cancel*
`@;`	*semi* :
`@&`	*math* : `\J`
`@{`	*math* : `\B`
`@}`	*math* : `\T`

When a string is output, certain characters are preceded by '\' signs so that they will print properly.

A comment in the input will be combined with the preceding scrap. An additional "comment" is effectively appended at the end of the ADA text, just before translation begins; this consists of a *cancel* token in the case of ADA text in `|` ... `|`, otherwise it consists of a *force* token.

From this table it is evident that `AWEAVE` will parse a lot of non-ADA programs. For example, the reserved words '`or`' and '`else`' are treated in an identical way by `AWEAVE`; ADA programmers may well be surprised at this similarity. The idea is to keep `AWEAVE`'s rules as simple as possible, consistent with doing a reasonable job on syntactically correct ADA programs. The production rules below have been formulated in the same spirit of "almost anything goes."

170. Here is a table of all the productions. The reader can best get a feel for how they work by trying them out by hand on small examples; no amount of explanation will be as effective as watching the rules in action.

Production categories [translations]	Remarks
1 accept_scrap math follows → beginning [B = A indent pdollar M pdollar outdent F]	**accept** *name* **do**
2 accept_scrap math semi → stmt [S_2 = A pdollar M pdollar S_1]	**accept** *name*;
3 and_scrap then_scrap → math [M = \ANT]	**and then**
4 arrow → math	'⇒' in aggregate etc.
5 base base → math [M = \bn{ B_1 }{ B_2 }]	complete based literal
6 begin_label_scrap math end_label_scrap → label [L = B pdollar M pdollar E]	label
7 beginning stmt → stmt [S_2 = force indent B force S_1]	beginning of block
8 case_scrap math follows decl → decl [D_2 = C indent pdollar sspace M pdollar indent F force D_1]	beginning of variant part
9 case_scrap math follows stmt → stmt [S_2 = force C indent pdollar sspace M pdollar indent F force S_1]	beginning of **case** statement
10 clause decl → decl [D_2 = force backup C force D_1]	element of variant part
11 clause stmt → stmt [S_2 = force backup C force S_1]	element of **case** statement
12 colon exception_scrap → colon math	exception declaration
13 colon in_scrap → colon [C_2 = C_1 \&{in} sspace]	**in** as parameter mode
14 decl beginning → stmt [S = D force backup B force]	beginning of statements
15 decl decl → decl [D_3 = D_1 force D_2]	declaration part grows
16 decl mod_scrap semi → decl [D_2 = D_1 force M S]	module as declaration
17 decl terminator → decl [D_2 = D_1 cancel outdent force T force]	end of declarative part
18 decl with_scrap → with_scrap [W_2 = D force W_1]	context clause grows
19 digit_scrap → math	digit not in based number
20 digit_scrap base → base [B_2 = D]	part of based number grows
21 digit_scrap digit_scrap → digit_scrap	concatenation
22 digit_scrap period → digit_scrap	concatenation
23 else_scrap else_scrap → math [M = \ORE]	**or else**
24 else_scrap when_scrap → when_scrap [W_2 = \ORW]	**or when** in **select** statement
25 ending beginning semi → terminator [T = E sspace B S]	**end select**;
26 ending case_scrap semi → terminator [T = backup outdent E sspace C S]	**end case**;
27 ending if_scrap semi → terminator [T = E sspace outdent I S]	**end if**;
28 ending loop_scrap semi → terminator [T = E sspace L S]	**end loop**;
29 ending loop_scrap math semi → terminator [T = E sspace L pdollar sspace M pdollar S]	**end loop** *name*;
30 ending math semi → terminator [T = E pdollar sspace M pdollar S]	**end** *name*;
31 ending record_scrap semi → terminator [T = outdent E sspace R S]	**end record**;
32 ending semi → terminator	**end**;
33 exception_scrap → beginning [B = backup E]	beginning of **exception** part
34 exp digit_scrap → math* [M = E D }]	unsigned exponent
35 exp math digit_scrap → math* [M_2 = E D M_1 }]	signed exponent
36 generic_scrap decl → generic_scrap [G_2 = G_1 force D]	generic formal part grows
37 generic_scrap proc → proc [P_2 = indent G outdent force P_1]	generic formal part ends
38 gproc math semi → decl [D = G indent pdollar M pdollar outdent S]	generic formal subprogram
39 gproc math follows math semi → decl [D = G indent pdollar M_1 F M_2 pdollar outdent S]	*38* with default
40 if_scrap math then_scrap stmt → stmt [S_2 = force I indent pdollar sspace M pdollar outdent T force S_1]	part of **if** statement
41 in_scrap → math	**in** as '∈'

42 label label $\rightarrow$ *label* $[\![L_3 = L_1\ sspace\ opt\ 7\ L_2]\!]$	multiple label
43 label stmt $\rightarrow$ *stmt* $[\![S_2 = force\ backup\ L\ sspace\ cancel\ S_1]\!]$	labeled statement
44 loop_scrap $\rightarrow$ *beginning*	beginning of **loop** statement
45 math and_scrap math $\rightarrow$ *math*	**and** as '$\wedge$'
46 math colon beginning $\rightarrow$ *beginning* $[\![B_2 = backup\ pdollar\ M\ sspace\ pdollar\ C\ sspace\ B]\!]$	block or loop with name
47 math colon decl $\rightarrow$ *decl* $[\![D_2 = force\ backup\ pdollar\ M\ sspace\ pdollar\ C\ sspace\ cancel\ D_1]\!]$	block with name and **declare**
48 math colon math semi $\rightarrow$ *decl* $[\![D = pdollar\ M_1\ C\ M_2\ pdollar\ S]\!]$	declaration
49 math else_scrap math $\rightarrow$ *math* $[\![M_3 = M_1\ \backslash\mathtt{V}\ M_2]\!]$	**or** as '$\vee$'
50 math follows math semi $\rightarrow$ *decl* $[\![D = pdollar\ M_1\ pdollar\ F\ pdollar\ M_2\ pdollar\ S]\!]$	**type** declaration etc.
51 math follows record_scrap $\rightarrow$ *decl* $[\![D = pdollar\ M\ pdollar\ F\ indent\ force\ R\ indent\ force]\!]$	**record** type declaration
52 math loop_scrap $\rightarrow$ *beginning* $[\![B = pdollar\ M\ pdollar\ sspace\ L]\!]$	**loop** with **for** or **while**
53 math math $\rightarrow$ *math*	simple concatenation
54 math semi $\rightarrow$ *stmt* $[\![S_2 = pdollar\ M\ pdollar\ S_1]\!]$	simple statement
55 math stmt $\rightarrow$ *stmt* $[\![S_2 = pdollar\ M\ pdollar\ S_1]\!]$	simple concatenation
56 math separate_scrap $\rightarrow$ *separate_scrap* $[\![S_2 = pdollar\ M\ pdollar\ S_1]\!]$	in emergencies
57 math when_scrap math $\rightarrow$ *math*	**exit when** x;
58 math with_scrap $\rightarrow$ *with_scrap* $[\![W_2 = pdollar\ M\ pspace\ pdollar\ W_1]\!]$	**for** x **use** ...
59 mod_scrap $\rightarrow$ *math*	module unlike a statement
60 mod_scrap semi $\rightarrow$ *stmt* $[\![S_2 = M\ S_1\ force]\!]$	module like a statement
61 new_scrap $\rightarrow$ *math*	**new** in allocator
62 not_scrap $\rightarrow$ *math*	**not** as '$\neg$'
63 not_scrap in_scrap $\rightarrow$ *math* $[\![M = \backslash\mathtt{notin}]\!]$	**not in** as '$\notin$'
64 open_scrap decl $\rightarrow$ *open_scrap* $[\![O_2 = O_1\ pdollar\ D\ pdollar\ \backslash\mathtt{,}\ opt\ 5]\!]$	declaration in parentheses
65 open_scrap math close_scrap $\rightarrow$ *math* $[\![M_2 = indent\ O\ M_1\ C\ outdent]\!]$	parenthesized quantity
66 open_scrap math colon math close_scrap $\rightarrow$ *math* $[\![M_3 = O\ indent\ M_1\ C_1\ M_2\ outdent\ C_2]\!]$	formal part ends
67 period $\rightarrow$ *math*	period in selected component
68 private_scrap $\rightarrow$ *decl* $[\![D = force\ backup\ P\ force]\!]$	beginning of private part
69 private_scrap semi $\rightarrow$ *math*	private type declaration
70 proc math follows $\rightarrow$ *decl* $[\![D = P\ indent\ pdollar\ M\ pdollar\ outdent\ F\ cancel\ indent]\!]$	package declaration etc.
71 proc math follows new_scrap $\rightarrow$ *proc math math*	instantiation
72 proc math follows separate_scrap $\rightarrow$ *proc math math*	body stub
73 proc math semi $\rightarrow$ *decl* $[\![D = P\ indent\ pdollar\ M\ pdollar\ outdent\ S]\!]$	subprogram specification
74 return_scrap $\rightarrow$ *math* $[\![M = pspace\ R\ sspace]\!]$	**return** with expression
75 return_scrap semi $\rightarrow$ *stmt*	**return**;
76 separate_scrap math $\rightarrow$ *decl* $[\![D = S\ pdollar\ sspace\ M\ pdollar]\!]$	**separate**(...)
77 stmt else_scrap stmt $\rightarrow$ *stmt* $[\![S_3 = S_1\ force\ backup\ E\ force\ S_2]\!]$	**else** part
78 stmt stmt $\rightarrow$ *stmt* $[\![S_3 = S_1\ break_space\ S_2]\!]$	concatenation
79 stmt terminator $\rightarrow$ *stmt* $[\![S_2 = S_1\ cancel\ outdent\ force\ T\ force]\!]$	end of block etc.
80 when_scrap math arrow $\rightarrow$ *clause* $[\![C = W\ pdollar\ M\ pdollar\ A]\!]$	clause in **case**
81 with_scrap colon $\rightarrow$ *with_scrap*	**use at**
82 with_scrap math semi $\rightarrow$ *decl* $[\![D = W\ pdollar\ M\ pdollar\ S]\!]$	context clause
83 with_scrap proc $\rightarrow$ *gproc*	generic formal subprogram
84 with_scrap record_scrap $\rightarrow$ *decl* $[\![D = W\ indent\ force\ indent\ R]\!]$	**record** representation
85 with_scrap record_scrap colon $\rightarrow$ *math colon* $[\![M = pdollar\ W\ indent\ force\ indent\ R\ pdollar]\!]$	*84* with alignment

Translations are not specified here when they are simple concatenations of the scraps that change. For

example, the full translation of '*colon exception_scrap* $\rightarrow$ *colon math*' is $C_2 = C_1$, $M = E$. The notation '*digit_scrap**' which occurs in the productions related to *exp* (numbers 34 and 35) means a *digit_scrap* scrap that isn't followed by another *digit_scrap* scrap.

171. Implementing the productions. When ADA text is to be processed with the grammar above, we put its initial scraps $s_1 \ldots s_n$ into two arrays $cat(1 \mathinner{..} n)$ and $trans(1 \mathinner{..} n)$. The value of $cat(k)$ is simply a category code from the list above; the value of $trans(k)$ is a text pointer, i.e., an index into *tok_start*. Our production rules have the nice property that the right-hand side is never longer than the left-hand side. Therefore it is convenient to use sequential allocation for the current sequence of scraps. Five pointers are used to manage the parsing:

pp (the parsing pointer) is such that we are trying to match the category codes $cat(pp)\, cat(pp+1) \ldots$ to the left-hand sides of productions.

scrap_base, *lo_ptr*, *hi_ptr*, and *scrap_ptr* are such that the current sequence of scraps appears in positions *scrap_base* through *lo_ptr* and *hi_ptr* through *scrap_ptr*, inclusive, in the *cat* and *trans* arrays. Scraps located between *scrap_base* and *lo_ptr* have been examined, while those in positions $\geq$ *hi_ptr* have not yet been looked at by the parsing process.

Initially *scrap_ptr* is set to the position of the final scrap to be parsed, and it doesn't change its value. The parsing process makes sure that $lo_ptr \geq pp + 4$, since productions have as many as five terms, by moving scraps from *hi_ptr* to *lo_ptr*. If there are fewer than $pp + 4$ scraps left, the positions up to $pp + 4$ are filled with blanks that will not match in any productions. Parsing stops when $pp = lo_ptr + 1$ and $hi_ptr = scrap_ptr + 1$.

The *trans* array elements are declared to be of type 0 .. 10_239 instead of type *text_pointer*, because the final sorting phase of AWEAVE uses this array to contain elements of type *name_pointer*. Both of these types are subranges of 0 .. 10_239.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
```
subtype scrap_index is INTEGER range 0 .. max_scraps;
type cat_type is array (scrap_index) of eight_bits;
type trans_type is array (scrap_index) of INTEGER range 0 .. 10_239;
cat : cat_type;  { category codes of scraps }
trans : trans_type;  { translation texts of scraps }
pp : scrap_index;  { current position for reducing productions }
scrap_base : scrap_index ← 1;  { beginning of the current scrap sequence }
scrap_ptr : scrap_index ← 0;  { ending of the current scrap sequence }
lo_ptr : scrap_index;  { last scrap that has been examined }
hi_ptr : scrap_index;  { first scrap that has not been examined }
stat
  max_scr_ptr : scrap_index ← 0;  { largest value assumed by scrap_ptr }
tats
```

172. Token lists in *tok_mem* are composed of the following kinds of items for TEX output.

- ASCII codes and special codes like *force* and *math_rel* represent themselves;
- $id_flag + p$ represents \\{identifier p};
- $res_flag + p$ represents \&{identifier p};
- $mod_flag + p$ represents module name p;
- $tok_flag + p$ represents token list number p;
- $inner_tok_flag + p$ represents token list number p, to be translated without line-break controls.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
```
id_flag : constant sixteen_bits ← 10_240;  { signifies an identifier }
res_flag : constant sixteen_bits ← 20_480;  { signifies a reserved word }
mod_flag : constant sixteen_bits ← 30_720;  { signifies a module name }
tok_flag : constant sixteen_bits ← 40_960;  { signifies a token list }
inner_tok_flag : constant sixteen_bits ← 51_200;  { signifies a token list in '| ... |' }
```

173. The production rules listed above are embedded directly into the `AWEAVE` program, since it is easier to do this than to write an interpretive system that would handle production systems in general. Several macros are defined here so that the program for each production is fairly short.

All of our productions conform to the general notion that some k consecutive scraps starting at some position j are to be replaced by a single scrap of some category c whose translations is composed from the translations of the disappearing scraps. After this production has been applied, the production pointer *pp* should change by an amount d. Such a production can be represented by the quadruple (j, k, c, d). For example, the production '*math math* $\rightarrow$ *math*' would be represented by '$(pp, 2, math, -3)$'; in this case the pointer *pp* should decrease by 3 after the production has been applied, because some productions with *math* in their fourth positions might now match, but no productions have *math* in the fifth position of their left-hand sides. Note that the value of d is determined by the whole collection of productions, not by an individual one. The determination of d has been done by hand in each case, based on the full set of productions but not on the grammar of ADA or on the rules for constructing the initial scraps.

Before calling *reduce*, the program should have appended the tokens of the new translation to the *tok_mem* array. We commonly want to append copies of several existing translations, and macros are defined to simplify these common cases. For example, *app2*(*pp*) will append the translations of two consecutive scraps, *trans*(*pp*) and *trans*(*pp* + 1), to the current token list. If the entire new translation is formed in this way, we write '*squash*(j, k, c, d)' instead of '*reduce*(j, k, c, d)'. For example, '*squash*$(pp, 2, math, -3)$' is an abbreviation for '*app2*(*pp*); *reduce*$(pp, 2, math, -3)$'.

The code below is an exact translation of the production rules into ADA, using such macros, and the reader should have no difficulty understanding the format by comparing the code with the symbolic productions as they were listed earlier.

define *reduce*(#) $\equiv$ *red*(#); **goto** *found*
define *squash*(#) $\equiv$ *sq*(#); **goto** *found*
define *app*(#) $\equiv$ *tok_mem*(*tok_ptr*) $\leftarrow$ #; *incr*(*tok_ptr*)
{ this is like *app_tok*, but it doesn't test for overflow }
define *app1*(#) $\equiv$ *tok_mem*(*tok_ptr*) $\leftarrow$ *tok_flag* + *trans*(#); *incr*(*tok_ptr*)
define *app2*(#) $\equiv$ *app1*(#); *app1*(# + 1)
define *app3*(#) $\equiv$ *app2*(#); *app1*(# + 2)

174. Let us consider the big case statement for productions now, before looking at its context. We want to design the program so that this case statement works, so we might as well not keep ourselves in suspense about exactly what code needs to be provided with a proper environment.

The code here is more complicated than it need be, since some compilers are unable to deal with procedures that contain a lot of program text. The *translate* procedure, which incorporates the **case** statement here, would become too long for those compilers if we did not do something to split the cases into parts. Therefore a separate procedure called *do_cases* has been introduced.

```
⟨ Match a production at pp, or increase pp if there is no match 174 ⟩ ≡
  if cat(pp) < if_scrap then
    do_cases;
  else
    case cat(pp) is
      when if_scrap ⇒ ⟨ Cases for if_scrap 194 ⟩;
      when in_scrap ⇒ ⟨ Cases for in_scrap 195 ⟩;
      when label ⇒ ⟨ Cases for label 196 ⟩;
      when loop_scrap ⇒ ⟨ Cases for loop_scrap 197 ⟩;
      when math ⇒ ⟨ Cases for math 198 ⟩;
      when mod_scrap ⇒ ⟨ Cases for mod_scrap 199 ⟩;
      when new_scrap ⇒ ⟨ Cases for new_scrap 200 ⟩;
      when not_scrap ⇒ ⟨ Cases for not_scrap 201 ⟩;
      when open_scrap ⇒ ⟨ Cases for open_scrap 202 ⟩;
      when period ⇒ ⟨ Cases for period 203 ⟩;
      when private_scrap ⇒ ⟨ Cases for private_scrap 204 ⟩;
      when proc ⇒ ⟨ Cases for proc 205 ⟩;
      when return_scrap ⇒ ⟨ Cases for return_scrap 206 ⟩;
      when separate_scrap ⇒ ⟨ Cases for separate_scrap 207 ⟩;
      when stmt ⇒ ⟨ Cases for stmt 208 ⟩;
      when when_scrap ⇒ ⟨ Cases for when_scrap 209 ⟩;
      when with_scrap ⇒ ⟨ Cases for with_scrap 210 ⟩;
      when others ⇒ null;
    end case;
    incr(pp);  { if no match was found, we move to the right }
  end if;
⟪found⟫ null
```

This code is used in section 215.

175. Here is the procedure that need to be present for the reason just explained.

```
⟨ Declaration of subprocedure for translate 175 ⟩ ≡
  procedure do_cases is separate;
```

This code is used in section 218.

176.*

```
output to file parsing-do_cases.adb
separate (parsing)
procedure do_cases is
begin
  case cat(pp) is
    when accept_scrap ⇒ ⟨ Cases for accept_scrap 177 ⟩;
    when and_scrap ⇒ ⟨ Cases for and_scrap 178 ⟩;
    when arrow ⇒ ⟨ Cases for arrow 179 ⟩;
    when base ⇒ ⟨ Cases for base 180 ⟩;
    when begin_label_scrap ⇒ ⟨ Cases for begin_label_scrap 181 ⟩;
    when beginning ⇒ ⟨ Cases for beginning 182 ⟩;
    when case_scrap ⇒ ⟨ Cases for case_scrap 183 ⟩;
    when clause ⇒ ⟨ Cases for clause 184 ⟩;
    when colon ⇒ ⟨ Cases for colon 185 ⟩;
    when decl ⇒ ⟨ Cases for decl 186 ⟩;
    when digit_scrap ⇒ ⟨ Cases for digit_scrap 187 ⟩;
    when else_scrap ⇒ ⟨ Cases for else_scrap 188 ⟩;
    when ending ⇒ ⟨ Cases for ending 189 ⟩;
    when exception_scrap ⇒ ⟨ Cases for exception_scrap 190 ⟩;
    when exp ⇒ ⟨ Cases for exp 191 ⟩;
    when generic_scrap ⇒ ⟨ Cases for generic_scrap 192 ⟩;
    when gproc ⇒ ⟨ Cases for gproc 193 ⟩;
    when others ⇒ null;
  end case;
  incr(pp);
⟪found⟫ null;
end do_cases;
```

177. Now comes the code that tries to match each production that starts with a particular type of scrap. Whenever a match is discovered, the *squash* or *reduce* macro will cause the appropriate action to be performed, followed by **goto** *found*.

```
⟨ Cases for accept_scrap 177 ⟩ ≡
  if cat(pp + 1 .. pp + 2) = (math, follows) then
    app1(pp); app(indent); app(pdollar); app1(pp + 1); app(pdollar); app(outdent); app1(pp + 2);
    reduce(pp, 3, beginning, −2);
  elsif cat(pp + 1 .. pp + 2) = (math, semi) then
    app1(pp); app(pdollar); app1(pp + 1); app(pdollar); app1(pp + 2);
    reduce(pp, 3, stmt, −3);
  end if
```

This code is used in section 176*.

178.

```
⟨ Cases for and_scrap 178 ⟩ ≡
  if cat(pp + 1) = then_scrap then
    app([\]); app([A]); app([N]); app([T]);
    reduce(pp, 2, math, −3);
  end if
```

This code is used in section 176*.

179. ⟨ Cases for *arrow* 179 ⟩ ≡
$squash(pp, 1, math, -3)$

This code is used in section 176*.

180. ⟨ Cases for *base* 180 ⟩ ≡
if $cat(pp + 1) = base$ **then**
$app([\backslash])$; $app([\mathtt{b}])$; $app([\mathtt{n}])$; $app([\{])$; $app1(pp)$; $app([\}])$; $app([\{])$; $app1(pp + 1)$; $app([\}])$;
$reduce(pp, 2, math, -3)$;
end if

This code is used in section 176*.

181. ⟨ Cases for *begin_label_scrap* 181 ⟩ ≡
if $cat(pp + 1 \mathinner{..} pp + 2) = (math, end_label_scrap)$ **then**
$app1(pp)$; $app(pdollar)$; $app1(pp + 1)$; $app(pdollar)$; $app1(pp + 2)$;
$reduce(pp, 3, label, -1)$;
end if

This code is used in section 176*.

182. ⟨ Cases for *beginning* 182 ⟩ ≡
if $cat(pp + 1) = stmt$ **then**
$app(force)$; $app(indent)$; $app1(pp)$; $app(force)$; $app1(pp + 1)$;
$reduce(pp, 2, stmt, -3)$;
end if

This code is used in section 176*.

183. ⟨ Cases for *case_scrap* 183 ⟩ ≡
if $cat(pp + 1 \mathinner{..} pp + 3) = (math, follows, decl)$ **then**
$app1(pp)$; $app(indent)$; $app(pdollar)$; $app(sspace)$; $app1(pp + 1)$; $app(pdollar)$; $app(indent)$;
$app1(pp + 2)$; $app(force)$; $app1(pp + 3)$;
$reduce(pp, 4, decl, -3)$;
elsif $cat(pp + 1 \mathinner{..} pp + 3) = (math, follows, stmt)$ **then**
$app(force)$; $app1(pp)$; $app(indent)$; $app(pdollar)$; $app(sspace)$; $app1(pp + 1)$; $app(pdollar)$;
$app(indent)$; $app1(pp + 2)$; $app(force)$; $app1(pp + 3)$;
$reduce(pp, 4, stmt, -3)$;
end if

This code is used in section 176*.

184. ⟨ Cases for *clause* 184 ⟩ ≡
if $cat(pp + 1) = decl$ **then**
$app(force)$; $app(backup)$; $app1(pp)$; $app(force)$; $app1(pp + 1)$;
$reduce(pp, 2, decl, -3)$;
elsif $cat(pp + 1) = stmt$ **then**
$app(force)$; $app(backup)$; $app1(pp)$; $app(force)$; $app1(pp + 1)$;
$reduce(pp, 2, stmt, -3)$;
end if

This code is used in section 176*.

185. ⟨ Cases for *colon* 185 ⟩ ≡
if $cat(pp+1) = exception_scrap$ **then**
$squash(pp+1, 1, math, -1)$;
elsif $cat(pp+1) = in_scrap$ **then**
$app1(pp)$; $app([\backslash])$; $app([\&])$; $app([\{])$; $app([\mathtt{i}])$; $app([\mathtt{n}])$; $app([\}])$; $app(sspace)$;
$reduce(pp, 2, colon, -2)$;
end if

This code is used in section 176*.

186. ⟨ Cases for *decl* 186 ⟩ ≡
if $cat(pp+1) = beginning$ **then**
$app1(pp)$; $app(force)$; $app(backup)$; $app1(pp+1)$; $app(force)$;
$reduce(pp, 2, stmt, -3)$;
elsif $cat(pp+1) = decl$ **then**
$app1(pp)$; $app(force)$; $app1(pp+1)$;
$reduce(pp, 2, decl, -3)$;
elsif $cat(pp+1 \mathinner{..} pp+2) = (mod_scrap, semi)$ **then**
$app1(pp)$; $app(force)$; $app2(pp+1)$;
$reduce(pp, 3, decl, -3)$;
elsif $cat(pp+1) = terminator$ **then**
$app1(pp)$; $app(cancel)$; $app(outdent)$; $app(force)$; $app1(pp+1)$; $app(force)$;
$reduce(pp, 2, decl, -3)$;
elsif $cat(pp+1) = with_scrap$ **then**
$app1(pp)$; $app(force)$; $app1(pp+1)$;
$reduce(pp, 2, with_scrap, -1)$;
end if

This code is used in section 176*.

187. ⟨ Cases for *digit_scrap* 187 ⟩ ≡
if $cat(pp+1) = base$ **then**
$app1(pp)$;
$reduce(pp, 2, base, -1)$;
elsif $cat(pp+1) = digit_scrap$ **then**
$squash(pp, 2, digit_scrap, -2)$;
elsif $cat(pp+1) = period$ **then**
$squash(pp, 2, digit_scrap, -2)$;
else
$squash(pp, 1, math, -3)$;
end if

This code is used in section 176*.

188. ⟨ Cases for *else_scrap* 188 ⟩ ≡
if $cat(pp+1) = else_scrap$ **then**
$app([\backslash])$; $app([\mathtt{O}])$; $app([\mathtt{R}])$; $app([\mathtt{E}])$;
$reduce(pp, 2, math, -3)$;
elsif $cat(pp+1) = when_scrap$ **then**
$app([\backslash])$; $app([\mathtt{O}])$; $app([\mathtt{R}])$; $app([\mathtt{W}])$;
$reduce(pp, 2, when_scrap, -1)$;
end if

This code is used in section 176*.

189.

define *compress_end* ≡ *app1*(*pp*); *app*(*sspace*); *app2*(*pp* + 1)

⟨ Cases for *ending* 189 ⟩ ≡
  if *cat*(*pp* + 1 .. *pp* + 2) = (*beginning*, *semi*) **then**
    compress_end;
    reduce(*pp*, 3, *terminator*, −1);
  elsif *cat*(*pp* + 1 .. *pp* + 2) = (*case_scrap*, *semi*) **then**
    app(*backup*); *app*(*outdent*); *compress_end*;
    reduce(*pp*, 3, *terminator*, −1);
  elsif *cat*(*pp* + 1 .. *pp* + 2) = (*if_scrap*, *semi*) **then**
    app1(*pp*); *app*(*sspace*); *app*(*outdent*); *app2*(*pp* + 1);
    reduce(*pp*, 3, *terminator*, −1);
  elsif *cat*(*pp* + 1 .. *pp* + 2) = (*loop_scrap*, *semi*) **then**
    compress_end;
    reduce(*pp*, 3, *terminator*, −1);
  elsif *cat*(*pp* + 1 .. *pp* + 3) = (*loop_scrap*, *math*, *semi*) **then**
    app1(*pp*); *app*(*sspace*); *app1*(*pp* + 1); *app*(*pdollar*); *app*(*sspace*); *app1*(*pp* + 2); *app*(*pdollar*);
    app1(*pp* + 3);
    reduce(*pp*, 4, *terminator*, −1);
  elsif *cat*(*pp* + 1 .. *pp* + 2) = (*math*, *semi*) **then**
    app1(*pp*); *app*(*pdollar*); *app*(*sspace*); *app1*(*pp* + 1); *app*(*pdollar*); *app1*(*pp* + 2);
    reduce(*pp*, 3, *terminator*, −1);
  elsif *cat*(*pp* + 1 .. *pp* + 2) = (*record_scrap*, *semi*) **then**
    app(*outdent*); *compress_end*;
    reduce(*pp*, 3, *terminator*, −1);
  elsif *cat*(*pp* + 1) = *semi* **then**
    squash(*pp*, 2, *terminator*, −1);
  end if

This code is used in section 176*.

190. ⟨ Cases for *exception_scrap* 190 ⟩ ≡
  app(*backup*); *app1*(*pp*);
  reduce(*pp*, 1, *beginning*, −2)

This code is used in section 176*.

191. ⟨ Cases for *exp* 191 ⟩ ≡
  if (*cat*(*pp* + 1) = *digit_scrap*) ∧ (*cat*(*pp* + 2) ≠ *digit_scrap*) **then**
    app2(*pp*); *app*([}]);
    reduce(*pp*, 2, *math*, −3);
  elsif (*cat*(*pp* + 1) = *math*) ∧ ((*cat*(*pp* + 2) = *digit_scrap*) ∧ (*cat*(*pp* + 3) ≠ *digit_scrap*)) **then**
    app3(*pp*); *app*([}]);
    reduce(*pp*, 3, *math*, −3);
  end if

This code is used in section 176*.

192. ⟨ Cases for *generic_scrap* 192 ⟩ ≡

```
if cat(pp + 1) = decl then
    app1(pp); app(force); app1(pp + 1);
    reduce(pp, 2, generic_scrap, 0);
elsif cat(pp + 1) = proc then
    app(indent); app1(pp); app(outdent); app(force); app1(pp + 1);
    reduce(pp, 2, proc, −1);
end if
```

This code is used in section 176*.

193. ⟨ Cases for *gproc* 193 ⟩ ≡

```
if cat(pp + 1 .. pp + 2) = (math, semi) then
    app1(pp); app(indent); app(pdollar); app1(pp + 1); app(pdollar); app(outdent); app1(pp + 2);
    reduce(pp, 3, decl, −3);
elsif cat(pp + 1 .. pp + 4) = (math, follows, math, semi) then
    app1(pp); app(indent); app(pdollar); app3(pp + 1); app(pdollar); app(outdent); app1(pp + 4);
    reduce(pp, 5, decl, −3);
end if
```

This code is used in section 176*.

194. ⟨ Cases for *if_scrap* 194 ⟩ ≡

```
if cat(pp + 1 .. pp + 3) = (math, then_scrap, stmt) then
    app(force); app1(pp); app(indent); app(pdollar); app(sspace); app1(pp + 1); app(pdollar);
    app(outdent); app1(pp + 2); app(force); app1(pp + 3);
    reduce(pp, 4, stmt, −3);
end if
```

This code is used in section 174.

195. ⟨ Cases for *in_scrap* 195 ⟩ ≡

```
squash(pp, 1, math, −3)
```

This code is used in section 174.

196. ⟨ Cases for *label* 196 ⟩ ≡

```
if cat(pp + 1) = label then
    app1(pp); app(sspace); app(opt); app([7]); app(pp + 1);
    reduce(pp, 2, label, −1);
elsif cat(pp + 1) = stmt then
    app(force); app(backup); app1(pp); app(sspace); app(cancel); app1(pp + 1);
    reduce(pp, 2, stmt, −3);
end if
```

This code is used in section 174.

197. ⟨ Cases for *loop_scrap* 197 ⟩ ≡

```
squash(pp, 1, beginning, −2)
```

This code is used in section 174.

198. ⟨ Cases for *math* 198 ⟩ ≡

```
if cat(pp + 1 .. pp + 2) = (and_scrap, math) then
  squash(pp, 3, math, −3);
elsif cat(pp + 1 .. pp + 2) = (colon, beginning) then
  app(backup); app(pdollar); app1(pp); app(sspace); app(pdollar); app1(pp + 1); app(sspace);
  app1(pp + 2);
  reduce(pp, 3, beginning, −3);
elsif cat(pp + 1 .. pp + 2) = (colon, decl) then
  app(force); app(backup); app(pdollar); app1(pp); app(sspace); app(pdollar); app1(pp + 1);
  app(sspace); app(cancel); app1(pp + 2);
  reduce(pp, 3, decl, −3);
elsif cat(pp + 1 .. pp + 3) = (colon, math, semi) then
  app(pdollar); app3(pp); app(pdollar); app1(pp + 3);
  reduce(pp, 4, decl, −3);
elsif cat(pp + 1 .. pp + 2) = (else_scrap, math) then
  app1(pp); app([\]); app([V]); app1(pp + 2);
  reduce(pp, 3, math, −3);
elsif cat(pp + 1 .. pp + 3) = (follows, math, semi) then
  app(pdollar); app1(pp); app(pdollar); app1(pp + 1); app(pdollar); app1(pp + 2); app(pdollar);
  app1(pp + 3);
  reduce(pp, 4, decl, −3);
elsif cat(pp + 1 .. pp + 2) = (follows, record_scrap) then
  app(pdollar); app1(pp); app(pdollar); app1(pp + 1); app(indent); app(force); app1(pp + 2);
  app(indent); app(force);
  reduce(pp, 3, decl, −3);
elsif cat(pp + 1) = loop_scrap then
  app(pdollar); app1(pp); app(pdollar); app(sspace); app1(pp + 1);
  reduce(pp, 2, beginning, −2);
elsif cat(pp + 1) = math then
  squash(pp, 2, math, −3);
elsif cat(pp + 1) = semi then
  app(pdollar); app1(pp); app(pdollar); app1(pp + 1);
  reduce(pp, 2, stmt, −3);
elsif cat(pp + 1) = stmt then
  app(pdollar); app1(pp); app(pdollar); app1(pp + 1);
  reduce(pp, 2, stmt, −3);
elsif cat(pp + 1) = separate_scrap then
  app(pdollar); app1(pp); app(pdollar); app1(pp + 1);
  reduce(pp, 2, separate_scrap, −1);
elsif cat(pp + 1 .. pp + 2) = (when_scrap, math) then
  squash(pp, 3, math, −3);
elsif cat(pp + 1) = with_scrap then
  app(pdollar); app1(pp); app(pspace); app(pdollar); app1(pp + 1);
  reduce(pp, 2, with_scrap, −1);
end if
```

This code is used in section 174.

199. ⟨ Cases for *mod_scrap* 199 ⟩ ≡
if $cat(pp+1) = semi$ **then**
$app2(pp)$; $app(force)$;
$reduce(pp, 2, stmt, -3)$;
else
$squash(pp, 1, math, -3)$;
end if

This code is used in section 174.

200. ⟨ Cases for *new_scrap* 200 ⟩ ≡
$squash(pp, 1, math, -3)$

This code is used in section 174.

201. ⟨ Cases for *not_scrap* 201 ⟩ ≡
if $cat(pp+1) = in_scrap$ **then**
$app([\backslash])$; $app([\mathtt{n}])$; $app([\mathtt{o}])$; $app([\mathtt{t}])$; $app([\mathtt{i}])$; $app([\mathtt{n}])$;
$reduce(pp, 2, math, -3)$;
else
$squash(pp, 1, math, -3)$;
end if

This code is used in section 174.

202. ⟨ Cases for *open_scrap* 202 ⟩ ≡
if $cat(pp+1) = decl$ **then**
$app1(pp)$; $app(pdollar)$; $app1(pp+1)$; $app(pdollar)$; $app([\backslash])$; $app([\mathtt{,}])$; $app(opt)$; $app([\mathtt{5}])$;
$reduce(pp, 2, open_scrap, 0)$;
elsif $cat(pp+1 \mathinner{..} pp+2) = (math, close_scrap)$ **then**
$app(indent)$; $app2(pp)$; $app(outdent)$; $app1(pp+2)$;
$reduce(pp, 3, math, -3)$;
elsif $cat(pp+1 \mathinner{..} pp+4) = (math, colon, math, close_scrap)$ **then**
$app1(pp)$; $app(indent)$; $app3(pp+1)$; $app(outdent)$; $app1(pp+4)$;
$reduce(pp, 5, math, -3)$;
end if

This code is used in section 174.

203. ⟨ Cases for *period* 203 ⟩ ≡
$squash(pp, 1, math, -3)$

This code is used in section 174.

204. ⟨ Cases for *private_scrap* 204 ⟩ ≡
if $cat(pp+1) = semi$ **then**
$squash(pp, 1, math, -2)$;
else
$app(force)$; $app(backup)$; $app1(pp)$; $app(force)$;
$reduce(pp, 1, decl, -3)$;
end if

This code is used in section 174.

205. ⟨ Cases for *proc* 205 ⟩ ≡

```
if cat(pp + 1 .. pp + 3) = (math, follows, new_scrap) then
    squash(pp + 2, 2, math, −3);
elsif cat(pp + 1 .. pp + 3) = (math, follows, separate_scrap) then
    squash(pp + 2, 2, math, −3);
elsif cat(pp + 1 .. pp + 2) = (math, follows) then
    app1(pp); app(indent); app(pdollar); app1(pp + 1); app(pdollar); app(outdent); app1(pp + 2);
    app(cancel); app(indent);
    reduce(pp, 3, decl, −3);
elsif cat(pp + 1 .. pp + 2) = (math, semi) then
    app1(pp); app(indent); app(pdollar); app1(pp + 1); app(pdollar); app(outdent); app1(pp + 2);
    reduce(pp, 3, decl, −3);
end if
```

This code is used in section 174.

206. ⟨ Cases for *return_scrap* 206 ⟩ ≡

```
if cat(pp + 1) = semi then
    squash(pp, 2, stmt, −3);
else
    app(pspace); app1(pp); app(sspace);
    reduce(pp, 1, math, −3);
end if
```

This code is used in section 174.

207. ⟨ Cases for *separate_scrap* 207 ⟩ ≡

```
if cat(pp + 1) = math then
    app1(pp); app(pdollar); app(sspace); app1(pp + 1); app(pdollar);
    reduce(pp, 2, decl, −3);
end if
```

This code is used in section 174.

208. ⟨ Cases for *stmt* 208 ⟩ ≡

```
if cat(pp + 1 .. pp + 2) = (else_scrap, stmt) then
    app1(pp); app(force); app(backup); app1(pp + 1); app(force); app1(pp + 2);
    reduce(pp, 3, stmt, −3);
elsif cat(pp + 1) = stmt then
    app1(pp); app(break_space); app1(pp + 1);
    reduce(pp, 2, stmt, −3);
elsif cat(pp + 1) = terminator then
    app1(pp); app(cancel); app(outdent); app(force); app1(pp + 1); app(force);
    reduce(pp, 2, stmt, −3);
end if
```

This code is used in section 174.

209. ⟨ Cases for *when_scrap* 209 ⟩ ≡

```
if cat(pp + 1 .. pp + 2) = (math, arrow) then
    app1(pp); app(pdollar); app1(pp + 1); app(pdollar); app1(pp + 2);
    reduce(pp, 3, clause, −1);
end if
```

This code is used in section 174.

210. ⟨ Cases for *with_scrap* 210 ⟩ ≡

```
if cat(pp + 1) = colon then
  squash(pp, 2, with_scrap, −1);
elsif cat(pp + 1 .. pp + 2) = (math, semi) then
  app1(pp); app(pdollar); app1(pp + 1); app(pdollar); app1(pp + 2);
  reduce(pp, 3, decl, −3);
elsif cat(pp + 1) = proc then
  squash(pp, 2, gproc, 0);
elsif cat(pp + 1 .. pp + 2) = (record_scrap, colon) then
  app(pdollar); app1(pp); app(indent); app(force); app(indent); app1(pp + 1); app(pdollar);
  reduce(pp, 2, math, −1);
elsif cat(pp + 1) = record_scrap then
  app1(pp); app(indent); app(force); app(indent); app1(pp + 1);
  reduce(pp, 2, decl, −3);
end if
```

This code is used in section 174.

211. The '*freeze_text*' macro is used to give official status to a token list. Before saying *freeze_text*, items are appended to the current token list, and we know that the eventual number of this token list will be the current value of *text_ptr*. But no list of that number really exists as yet, because no ending point for the current list has been stored in the *tok_start* array. After saying *freeze_text*, the old current token list becomes legitimate, and its number is the current value of *text_ptr* − 1 since *text_ptr* has been increased. The new current token list is empty and ready to be appended to. Note that *freeze_text* does not check to see that *text_ptr* hasn't gotten too large, since it is assumed that this test was done beforehand.

define *freeze_text* ≡ *incr*(*text_ptr*); *tok_start*(*text_ptr*) ← *tok_ptr*

212. The '*reduce*' macro used in our code for productions actually calls on a procedure named '*red*', which makes the appropriate changes to the scrap list.

⟨ Procedures in *parsing* 212 ⟩ ≡

```
procedure red(j : sixteen_bits; k : eight_bits; c : eight_bits; d : INTEGER) is
begin
  cat(j) ← c; trans(j) ← text_ptr; freeze_text;
  if k > 1 then
    for i ∈ j + k .. lo_ptr loop
      cat(i − k + 1) ← cat(i); trans(i − k + 1) ← trans(i);
    end loop;
    lo_ptr ← lo_ptr − k + 1;
  end if;
  ⟨ Change pp to max(scrap_base, pp+d) 213 ⟩;
end red;
```

See also sections 214, 218, 222, 238, and 240.

This code is used in section 9*.

213. ⟨ Change *pp* to max(*scrap_base*, *pp*+*d*) 213 ⟩ ≡

```
if pp + d ≥ scrap_base then
  pp ← pp + d;
else
  pp ← scrap_base;
end if
```

This code is used in sections 212 and 214.

214. Similarly, the '*squash*' macro invokes a procedure called '*sq*'. This procedure takes advantage of the simplification that occurs when $k = 1$.

```
⟨Procedures in parsing 212⟩ +≡
  procedure sq(j : sixteen_bits; k : eight_bits; c : eight_bits; d : INTEGER) is
  begin
    if k = 1 then
      cat(j) ← c;
      ⟨Change pp to max(scrap_base, pp+d) 213⟩;
    else
      for i ∈ j .. j + k − 1 loop
        app1(i);
      end loop;
      red(j, k, c, d);
    end if;
  end sq;
```

215. Here now is the code that applies productions as long as possible.

```
⟨Reduce the scraps using the productions until no more rules apply 215⟩ ≡
  loop
    ⟨Make sure the entries cat(pp .. pp + 4) are defined 216⟩;
    if(tok_ptr + 11 > max_toks) ∨ (text_ptr + 4 > max_texts) then
      stat
        if tok_ptr > max_tok_ptr then
          max_tok_ptr ← tok_ptr;
        end if;
        if text_ptr > max_txt_ptr then
          max_txt_ptr ← text_ptr;
        end if;
      tats
      overflow("token/text");
    end if;
    exit when pp > lo_ptr;
    ⟨Match a production at pp, or increase pp if there is no match 174⟩;
  end loop
```

This code is used in section 219*.

216. If we get to the end of the scrap list, category codes equal to zero are stored, since zero does not match anything in a production.

```
⟨ Make sure the entries cat(pp .. pp + 4) are defined 216 ⟩ ≡
  if lo_ptr < pp + 4 then
    loop
      if hi_ptr ≤ scrap_ptr then
        incr(lo_ptr);
        cat(lo_ptr) ← cat(hi_ptr); trans(lo_ptr) ← trans(hi_ptr);
        incr(hi_ptr);
      end if;
      exit when (hi_ptr > scrap_ptr) ∨ (lo_ptr = pp + 4);
    end loop;
    for i ∈ lo_ptr + 1 .. pp + 4 loop
      cat(i) ← 0;
    end loop;
  end if
```

This code is used in section 215.

217. The *translate* function assumes that scraps have been stored in positions *scrap_base* through *scrap_ptr* of *cat* and *trans*. It appends a *comment_scrap* and begins to apply productions as much as possible. The result is a token list containing the translation of the given sequence of scraps.

After calling *translate*, we will have $text_ptr + 3 \le max_texts$ and $tok_ptr + 6 \le max_toks$, so it will be possible to create up to three token lists with up to six tokens without checking for overflow. Before calling *translate*, we should have $text_ptr < max_texts$ and $scrap_ptr < max_scraps$, since *translate* might add a new text and a new scrap before it checks for overflow.

```
⟨ Specification of procedures visible from parsing 217 ⟩ ≡
  function translate return text_pointer;
```

See also sections 221, 237, and 239.

This code is used in section 9*.

218.

```
⟨ Procedures in parsing 212 ⟩ +≡
  ⟨ Declaration of subprocedure for translate 175 ⟩
  function translate return text_pointer is separate;
```

219*

```
  output to file parsing-translate.adb
  separate (parsing)
  function translate return text_pointer is    { converts a sequence of scraps }
    k : long_buffer_index;   { index into buffer }
  begin
    pp ← scrap_base; lo_ptr ← pp − 1; hi_ptr ← pp;
    ⟨ Reduce the scraps using the productions until no more rules apply 215 ⟩;
    if (lo_ptr = scrap_base) ∧ (cat(lo_ptr) ≠ math) then
      return trans(lo_ptr);
    else
      ⟨ Combine the irreducible scraps that remain 220 ⟩;
    end if;
  end translate;
```

220. If the initial sequence of scraps does not reduce to a single scrap, we concatenate the translations of all remaining scraps, separated by blank spaces, with dollar signs surrounding the translations of *math* scraps.

```
⟨ Combine the irreducible scraps that remain 220 ⟩ ≡
  for j ∈ scrap_base .. lo_ptr loop
    if j ≠ scrap_base then
      app([␣]);
    end if;
    if cat(j) = math ∨ cat(j) = open_scrap ∨ cat(j) = close_scrap then
      app(pdollar);
    end if;
    app1(j);
    if cat(j) = math ∨ cat(j) = open_scrap ∨ cat(j) = close_scrap then
      app(pdollar);
    end if;
    if tok_ptr + 6 > max_toks then
      overflow("token");
    end if;
  end loop;
  freeze_text; return text_ptr − 1
```

This code is used in section 219*.

221. Initializing the scraps. If we are going to use the powerful production mechanism just developed, we must get the scraps set up in the first place, given a ADA text. A table of the initial scraps corresponding to ADA tokens appeared above in the section on parsing; our goal now is to implement that table. We shall do this by implementing a subroutine called *ADA_parse* that is analogous to the *ADA_xref* routine used during phase one.

Like *ADA_xref*, the *ADA_parse* procedure starts with the current value of *next_control* and it uses the operation *next_control* ← *get_next* repeatedly to read ADA text until encountering the next '|' or '{', or until *next_control* ≥ *format*. The scraps corresponding to what it reads are appended into the *cat* and *trans* arrays, and *scrap_ptr* is advanced.

Like *prod*, this procedure has to split into pieces so that each part is short enough to be handled by ADA compilers that discriminate against long subroutines. This time there is one split-off routine, called *easy_cases*.

After studying *ADA_parse*, we will look at the sub-procedure *app_comment*, that is used in some of its branches.

```
⟨ Specification of procedures visible from parsing 217 ⟩ +≡
  procedure app_comment;
```

222.
```
⟨ Procedures in parsing 212 ⟩ +≡
  ⟨ Declaration of the app_comment procedure 236 ⟩
  ⟨ Declaration of the easy_cases procedure 226 ⟩
  procedure ADA_parse is separate;
```

223*
```
output to file parsing-ada_parse.adb
separate (parsing)
procedure ADA_parse is    { creates scraps from ADA tokens }
  j : long_buffer_index;   { index into buffer }
  p : name_pointer;   { identifier designator }
begin
  while next_control < format loop
    ⟨ Append the scrap appropriate to next_control 225 ⟩;
    next_control ← get_next;
    if (next_control = [|]) ∨ (next_control = [{]) then
      return;
    end if;
  end loop;
end ADA_parse;
```

224. The macros defined here are helpful abbreviations for the operations needed when generating the scraps. A scrap of category c whose translation has three tokens t_1, t_2, t_3 is generated by $sc3(t_1)(t_2)(t_3)(c)$, etc.

```
define s0(#) ≡ incr(scrap_ptr); cat(scrap_ptr) ← #; trans(scrap_ptr) ← text_ptr; freeze_text
define sc1(#) ≡ app(#); s0
define sc2(#) ≡ app(#); sc1
define sc3(#) ≡ app(#); sc2
define sc4(#) ≡ app(#); sc3
define sc5(#) ≡ app(#); sc4
define sc6(#) ≡ app(#); sc5
define sc0(#) ≡ incr(scrap_ptr); cat(scrap_ptr) ← #; trans(scrap_ptr) ← 0
define comment_scrap(#) ≡ app(#); app_comment
```

225. ⟨ Append the scrap appropriate to *next_control* 225 ⟩ ≡
⟨ Make sure that there is room for at least four more scraps, six more tokens, and four more texts 229 ⟩;
⟪*reswitch*⟫ **case** *next_control* **is**
when *str* | *verbatim* | *output_tok* | *character_tok* ⇒ ⟨ Append a string scrap 231 ⟩;
when *ascii_tok* ⇒ ⟨ Append an ASCII-token scrap 233 ⟩;
when *identifier* ⇒ ⟨ Append an identifier scrap 235 ⟩;
when *TeX_string* ⇒ ⟨ Append a TEX string scrap 234 ⟩;
when others ⇒ *easy_cases*;
end case

This code is used in section 223*.

226. The *easy_cases* each result in straightforward scraps.

⟨ Declaration of the *easy_cases* procedure 226 ⟩ ≡
procedure *easy_cases* **is** { a subprocedure of *ADA_parse* }
begin
case *next_control* **is**
when *set_element_sign* ⇒ *sc3*([\])([i])([n])(*in_scrap*);
when *double_dot* ⇒ *sc3*([\])([t])([o])(*math*);
when [#] | [$] | [%] | [^] | [_] ⇒ *sc4*(*pdollar*)([\])(*next_control*)(*pdollar*)(*math*);
when [&] ⇒ *sc6*(*math_bin*)([\])([.])([\])([&])([}])(*math*);
when [!] ⇒ *sc5*(*math_rel*)([\])([.])([|])([}])(*math*);
when *ignore* | [|] | *xref_roman* .. *xref_reserved* ⇒ **null**;
when [(] ⇒ *sc1*(*next_control*)(*open_scrap*);
when [)] ⇒ *sc1*(*next_control*)(*close_scrap*);
when [*] ⇒ *sc4*([\])([a])([s])([t])(*math*);
when [,] ⇒ *sc3*([,])(*opt*)([9])(*math*);
when [0] .. [9] ⇒
if (*buffer*(*loc*) = [#]) ∨ (*buffer*(*loc* + 1) = [#]) **then**
⟨ Get a based number 227 ⟩;
else
sc3(*pdollar*)(*next_control*)(*pdollar*)(*digit_scrap*);
end if;
when [.] ⇒ *sc1*([.])(*period*);
when [;] ⇒ *sc1*([;])(*semi*);
when [:] ⇒ *sc1*([:])(*colon*);
⟨ Cases involving nonstandard ASCII characters 230 ⟩
when *exponent* ⇒ *sc3*([\])([E])([{])(*exp*);
when *begin_comment* ⇒ *sc2*([\])([B])(*math*);
when *end_comment* ⇒ *sc2*([\])([T])(*math*);
when *force_line* ⇒ *sc2*([\])([)])(*math*);
when *thin_space* ⇒ *sc2*([\])([,])(*math*);
when *math_break* ⇒ *sc2*(*opt*)([0])(*math*);
when *line_break* ⇒ *comment_scrap*(*force*);
when *big_line_break* ⇒ *comment_scrap*(*big_force*);
when *no_line_break* ⇒ *app*(*big_cancel*); *app*([\]); *app*([8]); *comment_scrap*(*big_cancel*);
when *pseudo_semi* ⇒ *sc0*(*semi*);
when *join* ⇒ *sc2*([\])([J])(*math*);
when others ⇒ *sc1*(*next_control*)(*math*);
end case;
end *easy_cases*;

This code is used in section 222.

227. ⟨ Get a based number 227 ⟩ ≡
⟨ Append *digit_scrap* scraps until a '#' occurs 228 ⟩;
sc1(*next_control*)(*base*); *next_control* ← *get_next*;
⟨ Append *digit_scrap* scraps until a '#' occurs 228 ⟩;
sc1(*next_control*)(*base*)

This code is used in section 226.

228. ⟨ Append *digit_scrap* scraps until a '#' occurs 228 ⟩ ≡

```
loop
  if next_control = [_] then
    sc2([\])([_])(digit_scrap);
  else
    sc1(next_control)(digit_scrap);
  end if;
  next_control ← get_next;
  exit when next_control = [#];
end loop
```

This code is used in section 227(2).

229. ⟨ Make sure that there is room for at least four more scraps, six more tokens, and four more texts 229 ⟩ ≡

```
if(scrap_ptr + 4 > max_scraps) ∨ (tok_ptr + 6 > max_toks) ∨ (text_ptr + 4 > max_texts) then
  stat
    if scrap_ptr > max_scr_ptr then
      max_scr_ptr ← scrap_ptr;
    end if;
    if tok_ptr > max_tok_ptr then
      max_tok_ptr ← tok_ptr;
    end if;
    if text_ptr > max_txt_ptr then
      max_txt_ptr ← text_ptr;
    end if;
  tats
  overflow("scrap/token/text");
end if
```

This code is used in section 225.

230. Some nonstandard ASCII characters may have entered AWEAVE by means of standard ones. They are converted to TEX control sequences so that it is possible to keep AWEAVE from stepping beyond standard ASCII.

⟨ Cases involving nonstandard ASCII characters 230 ⟩ ≡
when *not_equal* ⇒ *sc2*([\])([I])(*math*);
when *less_or_equal* ⇒ *sc2*([\])([L])(*math*);
when *greater_or_equal* ⇒ *sc2*([\])([G])(*math*);
when *equivalence_sign* ⇒ *sc2*([\])([S])(*math*);
when *and_sign* ⇒ *sc2*([\])([W])(*and_scrap*);
when *or_sign* ⇒ *sc2*([\])([V])(*else_scrap*);
when *not_sign* ⇒ *sc2*([\])([R])(*not_scrap*);
when *left_arrow* ⇒ *sc2*([\])([K])(*math*);
when *right_arrow* ⇒ *sc3*([\])([K])([R])(*arrow*);
when *double_star* ⇒ *sc4*([\])([d])([s])([t])(*math*);
when *left_label_bracket* ⇒ *sc3*([\])([B])([L])(*begin_label_scrap*);
when *right_label_bracket* ⇒ *sc3*([\])([E])([L])(*end_label_scrap*);
when *box_sign* ⇒ *sc4*([\])([B])([O])([X])(*math*);

This code is used in section 226.

231. The following code must use *app_tok* instead of *app* in order to protect against overflow. Note that $tok_ptr + 1 \le max_toks$ after *app_tok* has been used, so another *app* is legitimate before testing again.

Many of the special characters in a string must be prefixed by '\' so that TEX will print them properly.

⟨ Append a string scrap 231 ⟩ ≡
  app(*pdollar*); *app*([\]);
  case *next_control* **is**
    when *verbatim* ⇒ *app*([=]);
    when *output_tok* ⇒ *app*([O]);
    when others ⇒ *app*([.]);
  end case;
  app([{]); $j \leftarrow id_first$;
  while $j < id_loc$ **loop**
    ⟨ Append the character appropriate to *buffer*(j) 232 ⟩;
    incr(j);
  end loop;
  if *next_control* = *output_tok* **then**
    sc3([}])(*pdollar*)(*force*)(*math*);
  else
    sc2([}])(*pdollar*)(*math*);
  end if

This code is used in section 225.

232. ⟨ Append the character appropriate to *buffer*(*j*) 232 ⟩ ≡

```
case buffer(j) is
  when [␣] | [\] | [%] | [$] | [^] | [´] | [`] | [{] | [#] | [}] | [~] | [&] | [_] ⇒
    app([\]);
  when [@] ⇒
    if buffer(j + 1) = [@] then
      incr(j);
    else
      err_print("!␣Double␣@␣should␣be␣used␣in␣strings␣and␣character␣literals");
    end if;
  when others ⇒
    null;
end case;
app_tok(buffer(j))
```

This code is used in sections 231 and 233.

233. ⟨ Append an ASCII-token scrap 233 ⟩ ≡

```
app(pdollar); app([\]); app([H]); app([{]); j ← id_first + 1;
⟨ Append the character appropriate to buffer(j) 232 ⟩;
sc2([}])(pdollar)(math)
```

This code is used in section 225.

234. ⟨ Append a TEX string scrap 234 ⟩ ≡

```
app(pdollar); app([\]); app([h]); app([b]); app([o]); app([x]); app([{]);
for l ∈ id_first .. id_loc − 1 loop
  app_tok(buffer(l));
end loop;
sc2([}])(pdollar)(math)
```

This code is used in section 225.

235. ⟨ Append an identifier scrap 235 ⟩ ≡

```
p ← id_lookup(normal);
case ilk(p) is
  when normal | predefined ⇒ sc5(pspace)(pdollar)(id_flag + p)(pdollar)(pspace)(math);
        { not a reserved word }
  when accept_like ⇒ sc2(res_flag + p)(sspace)(accept_scrap);
  when all_like ⇒ sc5(pdollar)(pspace)(res_flag + p)(pdollar)(pspace)(math);
  when and_like ⇒ sc2([\])([W])(and_scrap);
  when array_like ⇒ sc2(res_flag + p)(sspace)(open_scrap);
  when at_like ⇒ sc3(sspace)(res_flag + p)(sspace)(colon);
  when begin_like ⇒ sc1(res_flag + p)(beginning);
  when body_like ⇒ sc4(pdollar)(res_flag + p)(pdollar)(pspace)(math);
  when case_like ⇒ sc1(res_flag + p)(case_scrap);
  when declare_like ⇒ sc4(force)(res_flag + p)(indent)(force)(decl);
  when goto_like ⇒ sc5(pspace)(pdollar)(res_flag + p)(pdollar)(sspace)(math);
  when is_like ⇒ sc3(sspace)(res_flag + p)(sspace)(follows);
  when else_like ⇒ sc1(res_flag + p)(else_scrap);
  when elsif_like ⇒ sc2(backup)(res_flag + p)(if_scrap);
  when end_like ⇒ sc1(res_flag + p)(ending);
  when exception_like ⇒ sc1(res_flag + p)(exception_scrap);
  when proc_like ⇒ sc2(res_flag + p)(sspace)(proc);
  when new_like ⇒ sc3(pspace)(res_flag + p)(pspace)(new_scrap);
  when generic_like ⇒ sc1(res_flag + p)(generic_scrap);
  when if_like ⇒ sc2(res_flag + p)(indent)(if_scrap);
  when in_like ⇒ sc3([\])([i])([n])(in_scrap);
  when loop_like ⇒ sc1(res_flag + p)(loop_scrap);
  when mod_like ⇒ sc3(math_bin)(res_flag + p)([}])(math);
  when not_like ⇒ sc2([\])([R])(not_scrap);
  when of_like ⇒ sc3(sspace)(res_flag + p)(sspace)(close_scrap);
  when or_like ⇒ sc1(res_flag + p)(else_scrap);
  when pragma_like ⇒ sc2(res_flag + p)(sspace)(proc);
  when private_like ⇒ sc1(res_flag + p)(private_scrap);
  when range_like ⇒ sc5(sspace)(pdollar)(res_flag + p)(pdollar)(sspace)(math);
  when record_like ⇒ sc1(res_flag + p)(record_scrap);
  when return_like ⇒ sc1(res_flag + p)(return_scrap);
  when separate_like ⇒ sc1(res_flag + p)(separate_scrap);
  when type_like ⇒ sc4(pdollar)(res_flag + p)(pdollar)(sspace)(math);
  when then_like ⇒ sc2(sspace)(res_flag + p)(then_scrap);
  when with_like ⇒ sc3(pspace)(res_flag + p)(sspace)(with_scrap);
  when when_like ⇒ sc2(res_flag + p)(sspace)(when_scrap);
  when others ⇒ null;
end case
```

This code is used in section 225.

236. A comment is incorporated into the previous scrap. The *app_comment* procedure takes care of placing a comment at the end of the current scrap list. When *app_comment* is called, we assume that the current token list is the translation of the comment involved.

⟨ Declaration of the *app_comment* procedure 236 ⟩ ≡
```
procedure app_comment is   { append a comment to the scrap list }
begin
  freeze_text; app1(scrap_ptr); app(text_ptr − 1 + tok_flag); trans(scrap_ptr) ← text_ptr; freeze_text;
end app_comment;
```
This code is used in section 222.

237. When the '|' that introduces ADA text is sensed, a call on *ADA_translate* will return a pointer to the TEX translation of that text. If scraps exist in the *cat* and *trans* arrays, they are unaffected by this translation process.

⟨ Specification of procedures visible from *parsing* 217 ⟩ +≡
```
function ADA_translate return text_pointer;
```

238. ⟨ Procedures in *parsing* 212 ⟩ +≡
```
function ADA_translate return text_pointer is
  p : text_pointer;   { points to the translation }
  save_base : scrap_index;   { holds original value of scrap_base }
begin
  save_base ← scrap_base; scrap_base ← scrap_ptr + 1; ADA_parse;   { get the scraps together }
  if next_control ≠ [|] then
    err_print("!␣Missing␣`|´␣after␣ADA␣text");
  end if;
  app_tok(cancel); app_comment;   { place a cancel token as a final "comment" }
  p ← translate;   { make the translation }
  stat
    if scrap_ptr > max_scr_ptr then
      max_scr_ptr ← scrap_ptr;
    end if;
  tats
  scrap_ptr ← scrap_base − 1; scrap_base ← save_base;   { scrap the scraps }
  return p;
end ADA_translate;
```

239. The *outer_parse* routine is to *ADA_parse* as *outer_xref* is to *ADA_xref*: It constructs a sequence of scraps for ADA text until *next_control* ≥ *format*. Thus, it takes care of embedded comments.

⟨ Specification of procedures visible from *parsing* 217 ⟩ +≡
```
procedure outer_parse;
```

240. ⟨ Procedures in *parsing* 212 ⟩ +≡

```
procedure outer_parse is   { makes scraps from ADA tokens and comments }
  bal : eight_bits;   { brace level in comment }
  p, q : text_pointer;   { partial comments }
begin
  while next_control < format loop
    if next_control ≠ [{] then
      ADA_parse;
    else
      ⟨ Make sure that there is room for at least seven more tokens, three more texts, and one more
          scrap 241 ⟩;
      app([\]); app([C]); app([{]); bal ← copy_comment(1); next_control ← [|];
      while bal > 0 loop
        p ← text_ptr; freeze_text; q ← ADA_translate;   { at this point we have tok_ptr + 6 ≤ max_toks }
        app(tok_flag + p); app(inner_tok_flag + q);
        if next_control = [|] then
          bal ← copy_comment(bal);
        else
          bal ← 0;   { an error has been reported }
        end if;
      end loop;
      app(force); app_comment;   { the full comment becomes a scrap }
    end if;
  end loop;
end outer_parse;
```

241. ⟨ Make sure that there is room for at least seven more tokens, three more texts, and one more scrap 241 ⟩ ≡

```
if(tok_ptr + 7 > max_toks) ∨ (text_ptr + 3 > max_texts) ∨ (scrap_ptr ≥ max_scraps) then
  stat
    if scrap_ptr > max_scr_ptr then
      max_scr_ptr ← scrap_ptr;
    end if;
    if tok_ptr > max_tok_ptr then
      max_tok_ptr ← tok_ptr;
    end if;
    if text_ptr > max_txt_ptr then
      max_txt_ptr ← text_ptr;
    end if;
  tats
  overflow("token/text/scrap");
end if
```

This code is used in section 240.

242. Output of tokens. So far our programs have only built up multi-layered token lists in AWEAVE's internal memory; we have to figure out how to get them into the desired final form. The job of converting token lists to characters in the TEX output file is not difficult, although it is an implicitly recursive process. Three main considerations had to be kept in mind when this part of AWEAVE was designed: (a) There are two modes of output, *outer* mode that translates tokens like *force* into line-breaking control sequences, and *inner* mode that ignores them except that blank spaces take the place of line breaks. (b) The *cancel* instruction applies to adjacent token or tokens that are output, and this cuts across levels of recursion since '*cancel*' occurs at the beginning or end of a token list on one level. (c) The TEX output file will be semi-readable if line breaks are inserted after the result of tokens like *break_space* and *force*. (d) The final line break should be suppressed, and there should be no *force* token output immediately after '\Y\P'.

243. The output process uses a stack to keep track of what is going on at different "levels" as the token lists are being written out. Entries on this stack have three parts:

end_field is the *tok_mem* location where the token list of a particular level will end;

tok_field is the *tok_mem* location from which the next token on a particular level will be read;

mode_field is the current mode, either *inner* or *outer*.

The current values of these quantities are referred to quite frequently, so they are stored in a separate place instead of in the *stack* array. We call the current values *cur_end*, *cur_tok*, and *cur_mode*.

The global variable *stack_ptr* tells how many levels of output are currently in progress. The end of output occurs when an *end_translation* token is found, so the stack is never empty except when we first begin the output process.

define *init_stack* ≡ *stack_ptr* ← 0; *cur_mode* ← *outer* { do this to initialize the stack }

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
```
type output_mode is (inner, outer);
type output_state is
  record
    end_field : sixteen_bits;  { ending location of token list }
    tok_field : sixteen_bits;  { present location within token list }
    mode_field : output_mode;  { interpretation of control tokens }
  end record;
cur_state : output_state;  { cur_end, cur_tok, cur_mode }
stack : array (INTEGER range 1 .. stack_size) of output_state;  { info for non-current levels }
stack_ptr : INTEGER range 0 .. stack_size;  { first unused location in the output state stack }
stat
  max_stack_ptr : INTEGER range 0 .. stack_size ← 0;  { largest value assumed by stack_ptr }
tats
cur_end : sixteen_bits renames cur_state.end_field;  { current ending location in tok_mem }
cur_tok : sixteen_bits renames cur_state.tok_field;  { location of next output token in tok_mem }
cur_mode : output_mode renames cur_state.mode_field;  { current mode of interpretation }
```

244. To insert token-list p into the output, the *push_level* subroutine is called; it saves the old level of output and gets a new one going. The value of *cur_mode* is not changed.

⟨ Specification of procedures visible from *second_phase* 244 ⟩ ≡
```
procedure push_level(p : text_pointer);
```
See also sections 249, 278, and 303.

This code is used in section 10*.

245. ⟨ Procedures in *second_phase* 245 ⟩ ≡

```
procedure push_level(p : text_pointer) is   { suspends the current level }
begin
  if stack_ptr = stack_size then
    overflow("stack");
  else
    if stack_ptr > 0 then
      stack(stack_ptr) ← cur_state;   { save cur_end ... cur_mode }
    end if;
    incr(stack_ptr);
    stat
      if stack_ptr > max_stack_ptr then
        max_stack_ptr ← stack_ptr;
      end if;
    tats
    cur_tok ← tok_start(p);  cur_end ← tok_start(p + 1);
  end if;
end push_level;
```

See also sections 248, 250, 251, 268, 279, and 304.

This code is used in section 10*.

246. Conversely, the *pop_level* routine restores the conditions that were in force when the current level was begun. This subroutine will never be called when $stack_ptr = 1$. It is so simple, we declare it as a macro:

define *pop_level* ≡ *decr*(*stack_ptr*); *cur_state* ← *stack*(*stack_ptr*) { do this when *cur_tok* reaches *cur_end* }

247. The *get_output* function returns the next byte of output that is not a reference to a token list. It returns the values *identifier* or *res_word* or *mod_name* if the next token is to be an identifier (typeset in italics), a reserved word (typeset in boldface) or a module name (typeset by a complex routine that might generate additional levels of output). In these cases *cur_name* points to the identifier or module name in question.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡

```
res_word : constant eight_bits ← 201_8;   { returned by get_output for reserved words }
mod_name : constant eight_bits ← 200_8;   { returned by get_output for module names }
```

248. ⟨ Procedures in *second_phase* 245 ⟩ +≡

```
function get_output return eight_bits is   { returns the next token of output }
  a : sixteen_bits;  { current item read from tok_mem }
begin
⟪restart⟫ while cur_tok = cur_end loop
    pop_level;
  end loop;
  a ← tok_mem(cur_tok); incr(cur_tok);
  if a ≥ 400₈ then
    cur_name ← a mod id_flag;
    case a/id_flag is
      when 2 ⇒ a ← res_word;  { a = res_flag + cur_name }
      when 3 ⇒ a ← mod_name;  { a = mod_flag + cur_name }
      when 4 ⇒ push_level(cur_name); goto restart;  { a = tok_flag + cur_name }
      when 5 ⇒ push_level(cur_name); cur_mode ← inner; goto restart;
            { a = inner_tok_flag + cur_name }
      when others ⇒ a ← identifier;  { a = id_flag + cur_name }
    end case;
  end if;
  return a;
end get_output;
```

249. The real work associated with token output is done by *make_output*. This procedure appends an *end_translation* token to the current token list, and then it repeatedly calls *get_output* and feeds characters to the output buffer until reaching the *end_translation* sentinel. It is possible for *make_output* to be called recursively, since a module name may include embedded ADA text; however, the depth of recursion never exceeds one level, since module names cannot be inside of module names.

A procedure called *output_ADA* does the scanning, translation, and output of ADA text within '| . . . |' brackets, and this procedure uses *make_output* to output the current token list. Thus, the recursive call of *make_output* actually occurs when *make_output* calls *output_ADA* while outputting the name of a module.

⟨ Specification of procedures visible from *second_phase* 244 ⟩ +≡

```
procedure make_output;
```

250. ⟨ Procedures in *second_phase* 245 ⟩ +≡

```
procedure output_ADA is   { outputs the current token list }
  save_tok_ptr, save_text_ptr, save_next_control : sixteen_bits;  { values to be restored }
  p : text_pointer;  { translation of the ADA text }
begin
  save_tok_ptr ← tok_ptr; save_text_ptr ← text_ptr; save_next_control ← next_control;
  next_control ← [|]; p ← ADA_translate; app(p + inner_tok_flag); make_output;  { output the list }
  stat
    if text_ptr > max_txt_ptr then
      max_txt_ptr ← text_ptr;
    end if;
    if tok_ptr > max_tok_ptr then
      max_tok_ptr ← tok_ptr;
    end if;
  tats
  text_ptr ← save_text_ptr; tok_ptr ← save_tok_ptr;  { forget the tokens }
  next_control ← save_next_control;  { restore next_control to original state }
end output_ADA;
```

251. Here is AWEAVE's major output handler.

```
⟨ Procedures in second_phase 245 ⟩ +≡
  procedure make_output is   { outputs the equivalents of tokens }
    a : eight_bits;   { current output byte }
    b : eight_bits;   { next output byte }
    k, k_limit : byte_index;   { indices into byte_mem }
    w : w_range;   { row of byte_mem }
    j : long_buffer_index;   { index into buffer }
    d : INTEGER;   { counts consecutive [$] signs or spaces }
    string_delimiter : ASCII_code;   { first and last character of string being copied }
    save_loc, save_limit : long_buffer_index;   { loc and limit to be restored }
    cur_mod_name : name_pointer;   { name of module being output }
    save_mode : output_mode;   { value of cur_mode before a sequence of breaks }
    ignore_spaces : BOOLEAN;   { should spaces be ignored? }
  begin
    app(end_translation);   { append a sentinel }
    freeze_text; push_level(text_ptr − 1);
    loop
      a ← get_output; ignore_spaces ← FALSE;
    ⟪reswitch⟫ case a is
        when end_translation ⇒
          return;
        when identifier | res_word ⇒
          ⟨ Output an identifier 253 ⟩;
        when mod_name ⇒
          ⟨ Output a module name 257 ⟩;
        when math_bin | math_op | math_rel ⇒
          ⟨ Output a \math operator 254 ⟩;
        when cancel ⇒
          loop
            a ← get_output;
            exit when (a < backup) ∨ (a > big_force);
          end loop;
          ignore_spaces ← FALSE; goto reswitch;
        when big_cancel ⇒
          loop
            a ← get_output;
            exit when ((a < backup) ∧ (a ≠ [␣]) ∧ (a ∉ pspace .. sspace)) ∨ (a > big_force);
          end loop;
          ignore_spaces ← FALSE; goto reswitch;
        when pdollar pspace .. sspace ⇒
          ⟨ Count the pdollar and space tokens; output the appropriate control sequences and goto
              reswitch 252 ⟩;
        when indent .. big_force ⇒
          ⟨ Output a control, look ahead in case of line breaks, possibly goto reswitch 255 ⟩;
        when others ⇒
          out1(a);   { otherwise a is an ASCII character }
      end case;
    end loop;
  end make_output;
```

252. Here we look ahead to see whether we have to output a begin-math control sequence and/or a space control sequence

```
⟨ Count the pdollar and space tokens; output the appropriate control sequences and goto reswitch 252 ⟩ ≡
  d ← 0; b ← 0;
  while (a = pdollar) ∨ (a ∈ pspace .. sspace) loop
    if a = pdollar then
      incr(d);
    else
      b ← b + a − pspace + 1;
    end if;
    a ← get_output;
    if cur_mode = inner then
      while a ∈ big_cancel .. backup loop
        if a = opt then
          a ← get_output;   { opt is followed by digit }
        end if;
        a ← get_output;
      end loop;
    end if;
  end loop;
  if (d mod 2) = 1 then
    out2([\])([?]);
  end if;
  if ¬ignore_spaces ∧ b > 1 then
    out2([\])([8]);
  end if;
  ignore_spaces ← FALSE; goto reswitch
```

This code is used in section 251.

253. An identifier of length one does not have to be enclosed in braces, and it looks slightly better if set in a math-italic font instead of a (slightly narrower) text-italic font. Thus we output '`\|a`' but '`\\{aa}`'.

```
⟨ Output an identifier 253 ⟩ ≡
  out1([\]);
  if a = identifier then
    if ilk(cur_name) = predefined then
      out1([p]);
    elsif length(cur_name) = 1 then
      out1([|]);
    else
      out1([\]);
    end if;
  else
    out1([&]);   { a = res_word }
  end if;
  if length(cur_name) = 1 then
    out1(byte_mem(cur_name mod ww, byte_start(cur_name)));
  else
    out_name(cur_name);
  end if
```

This code is used in section 251.

254. ⟨ Output a `\math` operator 254 ⟩ ≡

```
out5([\])([m])([a])([t])([h]);
if a = math_bin then
  out3([b])([i])([n]);
elsif a = math_rel then
  out3([r])([e])([l]);
else
  out2([o])([p]);
end if;
out1([{])
```

This code is used in section 251.

255. The current mode does not affect the behavior of `AWEAVE`'s output routine except when we are outputting control tokens.

⟨ Output a control, look ahead in case of line breaks, possibly **goto** *reswitch* 255 ⟩ ≡

```
if a < break_space then
  if cur_mode = outer then
    out2([\])(a − cancel + [0]);
    if a = opt then
      out1(get_output);  { opt is followed by a digit }
    end if;
  elsif a = opt then
    b ← get_output;  { ignore digit following opt }
  end if;
else
  ⟨ Look ahead for strongest line break, goto reswitch 256 ⟩;
end if
```

This code is used in section 251.

256. If several of the tokens *break_space*, *force*, *big_force* occur in a row, possibly mixed with blank spaces (which are ignored), the largest one is used. A line break also occurs in the output file, except at the very end of the translation. The very first line break is suppressed (i.e., a line break that follows '\Y\P').

```
⟨ Look ahead for strongest line break, goto reswitch 256 ⟩ ≡
  b ← a; save_mode ← cur_mode;
  loop
    a ← get_output;
    if (a = cancel) ∨ (a = big_cancel) then
      goto reswitch;  { cancel overrides everything }
    end if;
    if ((a ≠ [␣]) ∧ (a < break_space) ∧ (a ∉ pspace .. sspace)) ∨ (a > big_force) then
      if save_mode = outer then
        if (out_ptr > 3) and then (out_buf(out_ptr − 3 .. out_ptr) = ([\], [Y], [\], [P])) then
          goto reswitch;
        end if;
        out2([\])(b − cancel + [0]); ignore_spaces ← b ∈ force .. big_force;
        if a ≠ end_translation then
          finish_line;
        end if;
      elsif (a ≠ end_translation) ∧ (cur_mode = inner) then
        out1([␣]);
      end if;
      goto reswitch;
    end if;
    if a > b then
      b ← a;
    end if;  { if a = [␣] we have a < b }
  end loop
```

This code is used in section 255.

257. The remaining part of *make_output* is somewhat more complicated. When we output a module name, we may need to enter the parsing and translation routines, since the name may contain ADA code embedded in | ... | constructions. This ADA code is placed at the end of the active input buffer and the translation process uses the end of the active *tok_mem* area.

```
⟨ Output a module name 257 ⟩ ≡
  out2([\])([X]); cur_xref ← xref(cur_name);
  if num(cur_xref) ≥ def_flag then
    out_mod(num(cur_xref) − def_flag);
    if phase_three then
      cur_xref ← xlink(cur_xref);
      while num(cur_xref) ≥ def_flag loop
        out2([,])([␣]); out_mod(num(cur_xref) − def_flag); cur_xref ← xlink(cur_xref);
      end loop;
    end if;
  else
    out1([0]);  { output the module number, or zero if it was undefined }
  end if;
  out1([:]); ⟨ Output the text of the module name 258 ⟩;
  out2([\])([X])
```

This code is used in section 251.

258. ⟨ Output the text of the module name 258 ⟩ ≡

```
k ← byte_start(cur_name); w ← cur_name mod ww; k_limit ← byte_start(cur_name + ww);
cur_mod_name ← cur_name;
while k < k_limit loop
  b ← byte_mem(w, k); incr(k);
  if b = [@] then
    ⟨ Skip next character, give error if not '@' 259 ⟩;
  end if;
  if b ≠ [|] then
    out1(b);
  else
    ⟨ Copy the ADA text into buffer(limit + 1 .. j) 260 ⟩;
    save_loc ← loc; save_limit ← limit; loc ← limit + 2; limit ← j + 1; buffer(limit) ← [|];
    output_ADA; loc ← save_loc; limit ← save_limit;
  end if;
end loop
```

This code is used in section 257.

259. ⟨ Skip next character, give error if not '@' 259 ⟩ ≡

```
if byte_mem(w, k) ≠ [@] then
  print_nl("!␣Illegal␣control␣code␣in␣section␣name:"); print_nl("<"); print_id(cur_mod_name);
  print(">␣"); mark_error;
end if;
incr(k)
```

This code is used in section 258.

260. The ADA text enclosed in `|...|` should not contain '`|`' characters, except within strings. We put a '`|`' at the front of the buffer, so that an error message that displays the whole buffer will look a little bit sensible. The variable *string_delimiter* is zero outside of strings, otherwise it equals the delimiter that began the string being copied.

```
⟨ Copy the ADA text into buffer(limit + 1 .. j) 260 ⟩ ≡
  j ← limit + 1; buffer(j) ← [|]; string_delimiter ← 0;
  loop
    if k ≥ k_limit then
      print_nl("!␣ADA␣text␣in␣section␣name␣didn´t␣end:"); print_nl("<"); print_id(cur_mod_name);
      print(">␣"); mark_error; exit;
    end if;
    b ← byte_mem(w, k); incr(k);
    if b = [@] then
      ⟨ Copy a control code into the buffer 261 ⟩;
    else
      if b = ["] then
        if string_delimiter = 0 then
          string_delimiter ← b;
        elsif string_delimiter = b then
          string_delimiter ← 0;
        end if;
      elsif (b = [´]) ∧ (byte_mem(w, k + 1) = [´]) ∧ (string_delimiter = 0) then
        j ← j + 2; buffer(j − 1 .. j) ← ([´], byte_mem(w, k)); k ← k + 3; b ← [´];
      end if;
      if (b ≠ [|]) ∨ (string_delimiter ≠ 0) then
        if j > long_buf_size − 3 then
          overflow("buffer");
        end if;
        incr(j); buffer(j) ← b;
      else
        exit;
      end if;
    end if;
  end loop
```

This code is used in section 258.

261. ⟨ Copy a control code into the buffer 261 ⟩ ≡

```
  if j > long_buf_size − 4 then
    overflow("buffer");
  end if;
  buffer(j + 1) ← [@]; buffer(j + 2) ← byte_mem(w, k); j ← j + 2; incr(k)
```

This code is used in section 260.

262. Phase two processing. We have assembled enough pieces of the puzzle in order to be ready to specify the processing in AWEAVE's main pass over the source file. Phase two is analogous to phase one, except that more work is involved because we must actually output the TEX material instead of merely looking at the AWEB specifications.

⟨ Phase II: Read all the text again and translate it to TEX form 262 ⟩ ≡
  reset_input; *print_nl*("Writing␣the␣output␣file..."); *module_count* ← 0; *copy_limbo*; *finish_line*;
  flush_buffer(0, FALSE); { insert a blank line, it looks nice }
  while ¬*input_has_ended* **loop**
    ⟨ Translate the current module 264 ⟩;
  end loop

This code is used in section 304.

263. The output file will contain the control sequence \Y between non-null sections of a module, e.g., between the TEX and definition parts if both are nonempty. This puts a little white space between the parts when they are printed. However, we don't want \Y to occur between two definitions within a single module. The variables *out_line* or *out_ptr* will change if a section is non-null, so the following macros '*save_position*' and '*emit_space_if_needed*' are able to handle the situation:

define *save_position* ≡ *save_line* ← *out_line*; *save_place* ← *out_ptr*
define *emit_space_if_needed* ≡
    if (*save_line* ≠ *out_line*) ∨ (*save_place* ≠ *out_ptr*) **then**
      out2([\])([Y]);
    end if

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
  save_line : INTEGER; { former value of *out_line* }
  save_place : *sixteen_bits*; { former value of *out_ptr* }

264. ⟨ Translate the current module 264 ⟩ ≡
  incr(*module_count*);
  ⟨ Output the code for the beginning of a new module 265 ⟩;
  save_position;
  ⟨ Translate the TEX part of the current module 266 ⟩;
  ⟨ Translate the definition part of the current module 267 ⟩;
  ⟨ Translate the ADA part of the current module 272 ⟩;
  ⟨ Show cross references to this module 275 ⟩;
  ⟨ Output the code for the end of a module 281 ⟩

This code is used in section 262.

265. Modules beginning with the AWEB control sequence '@␣' start in the output with the TEX control sequence '\M', followed by the module number. Similarly, '@*' modules lead to the control sequence '\N'. If this is a changed module, we put * just before the module number.

⟨ Output the code for the beginning of a new module 265 ⟩ ≡
  out1([\]);
  if *buffer*(*loc* − 1) ≠ [*] **then**
    out1([M]);
  else
    out1([N]); *print*("*"); *print_int*(*module_count*, 1); *update_terminal*; { print a progress report }
  end if;
  out_mod(*module_count*); *out2*([.])([␣])

This code is used in section 264.

266. In the TEX part of a module, we simply copy the source text, except that index entries are not copied and ADA text within | . . . | is translated.

```
⟨ Translate the TEX part of the current module 266 ⟩ ≡
  loop
    next_control ← copy_TeX;
    case next_control is
      when [|] ⇒
        init_stack; output_ADA;
      when [@] ⇒
        out1([@]);
      when TeX_string | xref_roman .. xref_reserved | module_name ⇒
        loc ← loc − 2; next_control ← get_next;  { skip to @> }
        if next_control = TeX_string then
          err_print("!␣TeX␣string␣should␣be␣in␣ADA␣text␣only");
        end if;
      when begin_comment | end_comment | thin_space | math_break | line_break | big_line_break |
            no_line_break | join | pseudo_semi ⇒
        err_print("!␣You␣can´t␣do␣that␣in␣TeX␣text");
      when others ⇒
        null;
    end case;
    exit when next_control ≥ format;
  end loop
```

This code is used in section 264.

267. When we get to the following code we have *next_control* ≥ *format*, and the token memory is in its initial empty state.

```
⟨ Translate the definition part of the current module 267 ⟩ ≡
  if next_control ≤ definition then  { definition part non-empty }
    emit_space_if_needed; save_position;
  end if;
  while next_control ≤ definition loop  { format or definition }
    init_stack;
    if next_control = definition then
      ⟨ Start a macro definition 269 ⟩;
    else
      ⟨ Start a format definition 270 ⟩;
    end if;
    outer_parse; finish_ADA;
  end loop
```

This code is used in section 264.

268. The *finish_ADA* procedure outputs the translation of the current scraps, preceded by the control sequence '\P' and followed by the control sequence '\par'. It also restores the token and scrap memories to their initial empty state.

A *force* token is appended to the current scraps before translation takes place, so that the translation will normally end with \6 or \7 (the TEX macros for *force* and *big_force*). This \6 or \7 is replaced by the concluding \par or by \Y\par.

```
⟨ Procedures in second_phase 245 ⟩ +≡
  procedure finish_ADA is   { finishes a definition or a ADA part }
    p : text_pointer;   { translation of the scraps }
  begin
    out2([\])([P]); app_tok(force); app_comment; p ← translate; app(p + tok_flag); make_output;
        { output the list }
    if (out_ptr > 1) and then (out_buf(out_ptr − 1) = [\]) then
      if out_buf(out_ptr) = [6] then
        out_ptr ← out_ptr − 2;
      elsif out_buf(out_ptr) = [7] then
        out_buf(out_ptr) ← [Y];
      end if;
    end if;
    out4([\])([p])([a])([r]); finish_line;
    stat
      if text_ptr > max_txt_ptr then
        max_txt_ptr ← text_ptr;
      end if;
      if tok_ptr > max_tok_ptr then
        max_tok_ptr ← tok_ptr;
      end if;
      if scrap_ptr > max_scr_ptr then
        max_scr_ptr ← scrap_ptr;
      end if;
    tats
    tok_ptr ← 1; text_ptr ← 1; scrap_ptr ← 0;   { forget the tokens and the scraps }
  end finish_ADA;
```

269.

```
⟨ Start a macro definition 269 ⟩ ≡
  sc5(pdollar)(backup)([\])([D])(pdollar)(math);   { this will produce 'define ' }
  next_control ← get_next;
  if next_control ≠ identifier then
    err_print("!␣Improper␣macro␣definition");
  else
    sc3(pdollar)(id_flag + id_lookup(normal))(pdollar)(math);
  end if;
  next_control ← get_next
```

This code is used in section 267.

270. ⟨ Start a format definition 270 ⟩ ≡

```
sc5(pdollar)(backup)([\])([F])(pdollar)(math);  { this will produce 'format ' }
next_control ← get_next;
if next_control = identifier then
  sc3(pdollar)(id_flag + id_lookup(normal))(pdollar)(math); next_control ← get_next;
  if next_control = equivalence_sign then
    sc2([\])([S])(math);  { output an equivalence sign }
    next_control ← get_next;
    if next_control = identifier then
      sc3(pdollar)(id_flag + id_lookup(normal))(pdollar)(math); sc0(semi);
          { insert an invisible semicolon }
      next_control ← get_next;
    end if;
  end if;
end if;
if scrap_ptr ≠ 5 then
  err_print("!␣Improper␣format␣definition");
end if
```

This code is used in section 267.

271. Finally, when the TEX and definition parts have been treated, we have *next_control* ≥ *begin_ada*. We will make the global variable *this_module* point to the current module name, if it has a name.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡

```
this_module : name_pointer;  { the current module name, or zero }
```

272. ⟨ Translate the ADA part of the current module 272 ⟩ ≡

```
this_module ← 0;
if next_control ≤ module_name then
  emit_space_if_needed; init_stack;
  if next_control = begin_ada then
    next_control ← get_next;
  else
    this_module ← cur_module;
    ⟨ Check that = or ≡ follows this module name, and emit the scraps to start the module
        definition 273 ⟩;
  end if;
  while next_control ≤ module_name loop
    outer_parse;
    ⟨ Emit the scrap for a module name if present 274 ⟩;
  end loop;
  finish_ADA;
end if
```

This code is used in section 264.

273. ⟨ Check that = or ≡ follows this module name, and emit the scraps to start the module definition 273 ⟩ ≡

```
loop
  next_control ← get_next;
  exit when next_control ≠ [+];  { allow optional '+=' }
end loop;
if (next_control ≠ [=]) ∧ (next_control ≠ equivalence_sign) then
  err_print("!␣You␣need␣an␣=␣sign␣after␣the␣section␣name");
else
  next_control ← get_next;
end if;
if (out_ptr > 1) and then (out_buf(out_ptr − 1 .. out_ptr) = ([\], [Y])) then
  app(backup);  { the module name will be flush left }
end if;
sc1(mod_flag + this_module)(mod_scrap); cur_xref ← xref(this_module);
if num(cur_xref) ≠ module_count + def_flag then
  sc3(math_rel)([+])([}])(math);  { module name is multiply defined }
  this_module ← 0;  { so we won't give cross-reference info here }
end if;
sc2([\])([S])(math);  { output an equivalence sign }
sc1(force)(semi)  { this forces a line break unless '@+' follows }
```

This code is used in section 272.

274. ⟨ Emit the scrap for a module name if present 274 ⟩ ≡

```
if next_control < module_name then
  err_print("!␣You␣can´t␣do␣that␣in␣ADA␣text");
  next_control ← get_next;
elsif next_control = module_name then
  sc1(mod_flag + cur_module)(mod_scrap); next_control ← get_next;
end if
```

This code is used in section 272.

275. Cross references relating to a named module are given after the module ends.

⟨ Show cross references to this module 275 ⟩ ≡

```
if this_module > 0 then
  ⟨ Rearrange the list pointed to by cur_xref 277 ⟩;
  footnote(def_flag); footnote(0);
end if
```

This code is used in section 264.

276. To rearrange the order of the linked list of cross references, we need four more variables that point to cross reference entries. We'll end up with a list pointed to by *cur_xref*.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡

```
next_xref, this_xref, first_xref, mid_xref : xref_number;  { pointer variables for rearranging a list }
```

277. We want to rearrange the cross reference list so that all the entries with *def_flag* come first, in ascending order; then come all the other entries, in ascending order. There may be no entries in either one or both of these categories.

⟨ Rearrange the list pointed to by *cur_xref* 277 ⟩ ≡

```
first_xref ← xref(this_module); this_xref ← xlink(first_xref);  { bypass current module number }
if num(this_xref) > def_flag then
  mid_xref ← this_xref; cur_xref ← 0;  { this value doesn't matter }
  loop
    next_xref ← xlink(this_xref); xlink(this_xref) ← cur_xref; cur_xref ← this_xref;
    this_xref ← next_xref;
    exit when num(this_xref) ≤ def_flag;
  end loop;
  xlink(first_xref) ← cur_xref;
else
  mid_xref ← 0;  { first list null }
end if;
cur_xref ← 0;
while this_xref ≠ 0 loop
  next_xref ← xlink(this_xref); xlink(this_xref) ← cur_xref; cur_xref ← this_xref;
  this_xref ← next_xref;
end loop;
if mid_xref > 0 then
  xlink(mid_xref) ← cur_xref;
else
  xlink(first_xref) ← cur_xref;
end if;
cur_xref ← xlink(first_xref)
```

This code is used in section 275.

278. The *footnote* procedure gives cross reference information about multiply defined module names (if the *flag* parameter is *def_flag*), or about the uses of a module name (if the *flag* parameter is zero). It assumes that *cur_xref* points to the first cross-reference entry of interest, and it leaves *cur_xref* pointing to the first element not printed. Typical outputs: '`\A␣section 101.`'; '`\U␣sections 370 and 1009.`'; '`\U␣section 227(2)`'; '`\A␣sections 8, 27\*, and 64.`'.

⟨ Specification of procedures visible from *second_phase* 244 ⟩ +≡

```
procedure footnote(flag : sixteen_bits);
```

279. ⟨ Procedures in *second_phase* 245 ⟩ +≡

```
procedure footnote(flag : sixteen_bits) is   { outputs module cross-references }
  count : INTEGER;  { counts the number of occurences of a cross-reference }
begin
  if num(cur_xref) ≤ flag then
    return;
  end if;
  finish_line; out1([\]);
  if flag = 0 then
    out1([U]);
  else
    out1([A]);
  end if;
  out4([␣])([s])([e])([c]); out4([t])([i])([o])([n]);
  ⟨ Output all the module numbers on the reference list cur_xref 280 ⟩;
  out1([.]);
end footnote;
```

280. The following code distinguishes three cases, according as the number of cross references is one, two, or more than two.

```
define count_xref ≡ count ← 1;
        loop
          exit when (num(xlink(cur_xref)) ≤ flag) ∨ (num(xlink(cur_xref)) ≠ num(cur_xref));
          cur_xref ← xlink(cur_xref); incr(count);
        end loop
```

⟨ Output all the module numbers on the reference list *cur_xref* 280 ⟩ ≡

```
count_xref; first_xref ← xlink(cur_xref);  { first_xref is the first entry of the second xref }
if num(first_xref) > flag then
  out1([s]);  { plural }
end if;
out1([~]);
loop
  out_mod(num(cur_xref) − flag, count); cur_xref ← xlink(cur_xref);
      { point to the next cross reference to output }
  exit when num(cur_xref) ≤ flag; next_xref ← cur_xref;  { save cur_xref before counting }
  count_xref;
  if (num(xlink(cur_xref)) > flag) ∨ (next_xref ≠ first_xref) then
    out1([,]);  { not the last of two }
  end if;
  out1([␣]);
  if num(xlink(cur_xref)) ≤ flag then
    out4([a])([n])([d])([~]);  { the last }
  end if;
end loop
```

This code is used in section 279.

281. ⟨ Output the code for the end of a module 281 ⟩ ≡

```
out3([\])([f])([i]); finish_line; flush_buffer(0, FALSE)  { insert a blank line, it looks nice }
```

This code is used in section 264.

282. Phase three processing. We are nearly finished! AWEAVE's only remaining task is to write out the index, after sorting the identifiers and index entries.

```
⟨ Phase III: Output the cross-reference index 282 ⟩ ≡
  phase_three ← TRUE; print_nl("Writing␣the␣index...");
  if change_exists then
    finish_line;
    ⟨ Tell about changed modules 284 ⟩;
  end if;
  finish_line; out4([\])([i])([n])([x]); finish_line;
  ⟨ Do the first pass of sorting 286 ⟩;
  ⟨ Sort and output the index 292 ⟩;
  out4([\])([f])([i])([n]); finish_line;
  ⟨ Output all the module names 300 ⟩;
  out4([\])([c])([o])([n]); finish_line; print("Done.")
```

This code is used in section 306.

283. Just before the index comes a list of all the changed modules, including the index module itself.

```
⟨ Types and objects visible from data_structures 14 ⟩ +≡
  k_module : module_index; { runs through the modules }
```

284.

```
⟨ Tell about changed modules 284 ⟩ ≡
  k_module ← 1;
  while ¬changed_module(k_module) loop
    incr(k_module);
  end loop;
  out4([\])([c])([h])([␣]); out_mod(k_module);
  loop
    loop
      incr(k_module);
      exit when changed_module(k_module);
    end loop;
    out2([,])([␣]); out_mod(k_module);
    exit when k_module = module_count;
  end loop;
  out1([.])
```

This code is used in section 282.

285. A left-to-right radix sorting method is used, since this makes it easy to adjust the collating sequence and since the running time will be at worst proportional to the total length of all entries in the index. We put the identifiers into 102 different lists based on their first characters. (Uppercase letters are put into the same list as the corresponding lowercase letters, since we want to have '$t < TeX <$ **then**'.) The list for character c begins at location $bucket(c)$ and continues through the *blink* array.

```
⟨ Types and objects visible from data_structures 14 ⟩ +≡
  bucket : array (ASCII_code) of name_pointer;
  next_name : name_pointer; { successor of cur_name when sorting }
  blink : name_info; { links in the buckets }
  c : ASCII_code; { index into bucket }
```

286. To begin the sorting, we go through all the hash lists and put each entry having a nonempty cross-reference list into the proper bucket.

```
⟨ Do the first pass of sorting 286 ⟩ ≡
  for d ∈ INTEGER range 0 .. 127 loop
    bucket(d) ← 0;
  end loop;
  for h ∈ INTEGER range 0 .. hash_size − 1 loop
    next_name ← hash(h);
    while next_name ≠ 0 loop
      cur_name ← next_name; next_name ← link(cur_name);
      if xref(cur_name) ≠ 0 then
        c ← byte_mem(cur_name mod ww, byte_start(cur_name));
        if (c ≤ [Z]) ∧ (c ≥ [A]) then
          c ← c + 40₈;
        end if;
        blink(cur_name) ← bucket(c); bucket(c) ← cur_name;
      end if;
    end loop;
  end loop
```

This code is used in section 282.

287. During the sorting phase we shall use the *cat* and *trans* arrays from AWEAVE's parsing algorithm and rename them *depth* and *head*. They now represent a stack of identifier lists for all the index entries that have not yet been output. The variable *sort_ptr* tells how many such lists are present; the lists are output in reverse order (first *sort_ptr*, then *sort_ptr* − 1, etc.). The jth list starts at $head(j)$, and if the first k characters of all entries on this list are known to be equal we have $depth(j) = k$.

```
⟨ Types and objects visible from data_structures 14 ⟩ +≡
  depth : cat_type renames cat;  { reclaims memory that is no longer needed for parsing }
  head : trans_type renames trans;  { ditto }
  sort_ptr : scrap_index renames scrap_ptr;  { ditto }
  max_sorts : constant ← max_scraps;  { ditto }

  cur_depth : eight_bits;  { depth of current buckets }
  cur_byte : byte_index;  { index into byte_mem }
  cur_bank : w_range;  { row of byte_mem }
  cur_val : sixteen_bits;  { current cross reference number }
  stat
    max_sort_ptr : INTEGER range 0 .. max_sorts ← 0;
  tats  { largest value of sort_ptr }
```

288. The desired alphabetic order is specified by the *collate* array; namely, $collate(0) < collate(1) < \cdots < collate(100)$.

```
⟨ Types and objects visible from data_structures 14 ⟩ +≡
  collate : array (0 .. 100) of ASCII_code;  { collation order }
```

289. We use the order null $<$ ␣ $<$ other characters $<$ _ $<$ A $=$ a $< \cdots <$ Z $=$ z $<$ 0 $< \cdots <$ 9.

⟨ Set initial values 19 ⟩ +≡
```
collate(0) ← 0; collate(1) ← [␣];
for d ∈ INTEGER range 1 .. [␣] − 1 loop
  collate(d + 1) ← d;
end loop;
for d ∈ INTEGER range [␣] + 1 .. [0] − 1 loop
  collate(d) ← d;
end loop;
for d ∈ INTEGER range [9] + 1 .. [A] − 1 loop
  collate(d − 10) ← d;
end loop;
for d ∈ INTEGER range [Z] + 1 .. [_] − 1 loop
  collate(d − 36) ← d;
end loop;
collate([_] − 36) ← [_] + 1;
for d ∈ INTEGER range [z] + 1 .. 126 loop
  collate(d − 63) ← d;
end loop;
collate(64) ← [_];
for d ∈ INTEGER range [a] .. [z] loop
  collate(d − [a] + 65) ← d;
end loop;
for d ∈ INTEGER range [0] .. [9] loop
  collate(d − [0] + 91) ← d;
end loop;
```

290. Procedure *unbucket* goes through the buckets and adds nonempty lists to the stack, using the collating sequence specified in the *collate* array. The parameter to *unbucket* tells the current depth in the buckets. Any two sequences that agree in their first 255 character positions are regarded as identical.

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
```
infinity : constant eight_bits ← 255;   { ∞ (approximately) }
```

291. ⟨ Procedures in *third_phase* 291 ⟩ ≡

```
procedure unbucket(d : eight_bits) is   { empties buckets having depth d }
begin
  for f ∈ reverse 0 .. 100 loop
    if bucket(collate(f)) > 0 then
      if sort_ptr > max_sorts then
        overflow("sorting");
      end if;
      incr(sort_ptr);
      stat
        if sort_ptr > max_sort_ptr then
          max_sort_ptr ← sort_ptr;
        end if;
      tats
      if f = 0 then
        depth(sort_ptr) ← infinity;
      else
        depth(sort_ptr) ← d;
      end if;
      head(sort_ptr) ← bucket(collate(f)); bucket(collate(f)) ← 0;
    end if;
  end loop;
end unbucket;
```

See also sections 299 and 306.

This code is used in section 11*.

292. ⟨ Sort and output the index 292 ⟩ ≡

```
sort_ptr ← 0; unbucket(1);
while sort_ptr > 0 loop
  cur_depth ← depth(sort_ptr);
  if (blink(head(sort_ptr)) = 0) ∨ (cur_depth = infinity) then
    ⟨ Output index entries for the list at sort_ptr 294 ⟩;
  else
    ⟨ Split the list at sort_ptr into further lists 293 ⟩;
  end if;
end loop
```

This code is used in section 282.

293. ⟨ Split the list at *sort_ptr* into further lists 293 ⟩ ≡

```
next_name ← head(sort_ptr);
loop
  cur_name ← next_name; next_name ← blink(cur_name);
  cur_byte ← byte_start(cur_name) + cur_depth; cur_bank ← cur_name mod ww;
  if cur_byte = byte_start(cur_name + ww) then
    c ← 0;  { we hit the end of the name }
  else
    c ← byte_mem(cur_bank, cur_byte);
    if (c ≤ [Z]) ∧ (c ≥ [A]) then
      c ← c + 40₈;
    end if;
  end if;
  blink(cur_name) ← bucket(c); bucket(c) ← cur_name;
  exit when next_name = 0;
end loop;
decr(sort_ptr); unbucket(cur_depth + 1)
```

This code is used in section 292.

294. ⟨ Output index entries for the list at *sort_ptr* 294 ⟩ ≡

```
cur_name ← head(sort_ptr);
loop
  out2([\])([:]);
  ⟨ Output the name at cur_name 295 ⟩;
  ⟨ Output the cross-references at cur_name 296 ⟩;
  cur_name ← blink(cur_name);
  exit when cur_name = 0;
end loop;
decr(sort_ptr)
```

This code is used in section 292.

295. ⟨ Output the name at *cur_name* 295 ⟩ ≡

```
case ilk(cur_name) is
  when normal ⇒
    if length(cur_name) = 1 then
      out2([\])([|]);
    else
      out2([\])([\]);
    end if;
  when roman ⇒ null;
  when wildcard ⇒ out2([\])([9]);
  when italic ⇒ out2([\])([\]);
  when slanted ⇒ out2([\])([s]);
  when predefined ⇒ out2([\])([p]);
  when typewriter ⇒ out2([\])([.]);
  when others ⇒ out2([\])([&]);  { bold_face or reserved word }
end case;
out_name(cur_name)
```

This code is used in section 294.

296. Section numbers that are to be underlined are enclosed in '`\[...]`', wheras those to be output in italics are enclosed in '`\\{...}`'. Here we have to consider a special case: if section numbers $s_1, \ldots, s_n$ have to be output and there are indices $1 \le i < j \le n$ such that $s_i + 1 = s_{i+1}, s_{i+1} + 1 = s_{i+2}, \ldots, s_{j-1} + 1 = s_j$ and $j = n$ or $s_j + 1 \ne s_{j+1}$, we output 's_i--s_j'.

```
⟨ Output the cross-references at cur_name 296 ⟩ ≡
  ⟨ Invert the cross-reference list at cur_name, making cur_xref the head 297 ⟩;
  while cur_xref ≠ 0 loop
    out2([,])([␣]); cur_val ← num(cur_xref); this_xref ← cur_xref;
    if cur_val < def_flag then
      next_xref ← xlink(this_xref);
      while (next_xref ≠ 0) and then ((num(next_xref) < def_flag) ∧
              (num(this_xref) = (num(next_xref) − 1))) loop
        this_xref ← next_xref; next_xref ← xlink(this_xref);
      end loop;
      out_mod(cur_val);
      if cur_xref ≠ this_xref then
        out2([-])([-]); out_mod(num(this_xref));
      end if;
    elsif cur_val < body_flag then
      out2([\])([[]); out_mod(cur_val − def_flag); out1([]]);
    else
      out3([\])([\])([{]); out_mod(cur_val − body_flag); out1([}]);
    end if;
    cur_xref ← xlink(this_xref);
  end loop;
  out1([.]); finish_line
```

This code is used in section 294.

297. List inversion is best thought of as popping elements off one stack and pushing them onto another. In this case *cur_xref* will be the head of the stack that we push things onto.

```
⟨ Invert the cross-reference list at cur_name, making cur_xref the head 297 ⟩ ≡
  this_xref ← xref(cur_name); cur_xref ← 0;
  loop
    next_xref ← xlink(this_xref); xlink(this_xref) ← cur_xref; cur_xref ← this_xref;
    this_xref ← next_xref;
    exit when this_xref = 0;
  end loop
```

This code is used in section 296.

298. The following recursive procedure walks through the tree of module names and prints them.

```
⟨ Specification of procedures visible from third_phase 298 ⟩ ≡
  procedure mod_print(p : name_pointer);
```

See also section 305.

This code is used in section 11*.

299. ⟨ Procedures in *third_phase* 291 ⟩ +≡

```
procedure mod_print(p : name_pointer) is    { print all module names in subtree p }
begin
  if p > 0 then
    mod_print(llink(p));
    out2([\])([:]);
    tok_ptr ← 1; text_ptr ← 1; scrap_ptr ← 0; init_stack; app(p + mod_flag); make_output;
    footnote(0);   { cur_xref was set by make_output }
    finish_line;
    mod_print(rlink(p));
  end if;
end mod_print;
```

300. ⟨ Output all the module names 300 ⟩ ≡ *mod_print*(*root*)

This code is used in section 282.

301. The main program. Let's put it all together now: `AWEAVE` starts and ends here.

The main procedure has been split into three sub-procedures in order to keep certain ADA compilers from overflowing their capacity.

⟨ Specification of procedures visible from *first_phase* 128 ⟩ +≡
procedure *Phase_I*;

302. ⟨ Procedures in *first_phase* 129 ⟩ +≡
procedure *Phase_I* **is**
begin
⟨ Phase I: Read all the user's text and store the cross references 126 ⟩;
end *Phase_I*;

303. ⟨ Specification of procedures visible from *second_phase* 244 ⟩ +≡
procedure *Phase_II*;

304. ⟨ Procedures in *second_phase* 245 ⟩ +≡
procedure *Phase_II* **is**
begin
⟨ Phase II: Read all the text again and translate it to TEX form 262 ⟩;
end *Phase_II*;

305. ⟨ Specification of procedures visible from *third_phase* 298 ⟩ +≡
procedure *Phase_III*;

306. ⟨ Procedures in *third_phase* 291 ⟩ +≡
procedure *Phase_III* **is**
begin
⟨ Phase III: Output the cross-reference index 282 ⟩;
end *Phase_III*;

307.
define *aweb_input_file_name* ≡ `"input.web"`
define *change_input_file_name* ≡ `"input.ch"`
define *TeX_output_file_name* ≡ `"aweb_output.tex"`

⟨ Types and objects visible from *data_structures* 14 ⟩ +≡
opened : BOOLEAN ← FALSE;

308.*

```
output to file aweave.adb
with Ada.Command_Line;
use Ada.Command_Line;
with Ustrings;
use Ustrings;
with TEXT_IO, int_number_io;
with data_structures, searching, input_output, first_phase, second_phase, third_phase;
use data_structures, searching, input_output, first_phase, second_phase, third_phase;
with Ada.Strings.Unbounded;
use Ada.Strings.Unbounded;
procedure AWEAVE is
  ac : INTEGER ← 0;
  ic : INTEGER ← 0;
  yr_output_file : Ustring;
begin
  print_ln("Trying␣to␣open␣input␣file."); ac ← Argument_Count;
  if ac < 1 then
    fatal_error("No␣file␣on␣command␣line");
  end if;
  open_a_file(aweb_file, TEXT_IO.IN_FILE, Argument(1), opened);
  if ¬opened then
    fatal_error("AWEB␣file␣not␣open");
  end if;
  opened ← FALSE;
  if ac ≥ 2 then
    open_a_file(change_file, TEXT_IO.IN_FILE, Argument(2), opened);
  end if;
  if ¬opened then
    new_ln; print_ln("!␣No␣change␣file.");
  end if;
  ic ← Index(U(Argument(1)), ".");
  if ic = 0 then
    yr_output_file ← U(Argument(1)) & ".tex";
  else
    yr_output_file ← U(Argument(1)); Overwrite(yr_output_file, ic, ".tex");
    Ada.Strings.Unbounded.head(yr_output_file, ic + 3);
  end if;
  −−open_a_file(TeX_file, TEXT_IO.OUT_FILE, TeX_output_file_name, opened);
  open_a_file(TeX_file, TEXT_IO.OUT_FILE, S(yr_output_file), opened);
  if (¬opened) and then (¬create_TeX_file(S(yr_output_file))) then
    fatal_error("TeX␣file␣not␣open");
  end if;
  initialize; print_ln(banner);  { print a "banner line" }
  ⟨ Store all the reserved words 67 ⟩;
  ⟨ Store all the predefined identifiers 69 ⟩;
  Phase_I; Phase_II; Phase_III;
  ⟨ Check that all changes have been read 98 ⟩;
  raise end_of_AWEAVE; exception
when end_of_AWEAVE ⇒
  stat
    ⟨ Print statistics about memory usage 309 ⟩;
```

```
    tats
  { here files should be closed if the operating system requires it }
    ⟨ Print the job history 310 ⟩;
    return;
end AWEAVE;
```

309. ⟨ Print statistics about memory usage 309 ⟩ ≡

```
  print_nl("Memory␣usage␣statistics:␣"); print_int(name_ptr, 1); print("␣names,␣");
  print_int(xref_ptr, 1); print("␣cross␣references,␣"); print_int(byte_ptr(0), 1);
  for i ∈ 1 .. ww − 1 loop
    print("+"); print_int(byte_ptr(i), 1);
  end loop;
  print("␣bytes;"); print_nl("parsing␣required␣"); print_int(max_scr_ptr, 1); print("␣scraps,␣");
  print_int(max_txt_ptr, 1); print("␣texts,␣"); print_int(max_tok_ptr, 1); print("␣tokens,␣");
  print_int(max_stack_ptr, 1); print("␣levels;"); print_nl("sorting␣required␣");
  print_int(max_sort_ptr, 1); print("␣levels.")
```

This code is used in section 308*.

310. Some implementations may wish to pass the *history* value to the operating system so that it can be used to govern whether or not other programs are started. Here we simply report the history to the user.

⟨ Print the job *history* 310 ⟩ ≡

```
  case history is
    when spotless ⇒ print_nl("(No␣errors␣were␣found.)");
    when harmless_message ⇒ print_nl("(Did␣you␣see␣the␣warning␣message␣above?)");
    when error_message ⇒ print_nl("(Pardon␣me,␣but␣I␣think␣I␣spotted␣something␣wrong.)");
    when fatal_message ⇒ print_nl("(That␣was␣a␣fatal␣error,␣my␣friend.)");
  end case; { there are no other cases }
  new_ln
```

This code is used in section 308*.

311. System-dependent changes. This module should be replaced, if necessary, by changes to the program that are necessary to make `AWEAVE` work at a particular installation. It is usually best to design your change file so that all changes to previous modules preserve the module numbering; then everybody's version will be consistent with the printed program. More extensive changes, which introduce new modules, can be inserted here; then only the index itself will get a new module number.

312* Index. If you have read and understood the code for Phase III above, you know what is in this index and how it got here. All modules in which an identifier is used are listed with that identifier, except that reserved words are indexed only when they appear in format definitions, and the appearances of identifiers in module names are not indexed. Underlined entries correspond to where the identifier was declared. Error messages, control sequences put into the output, and a few other things like "recursion" are indexed here too.

The following sections were changed by the change file: 3, 4, 5, 6, 7, 8, 9, 10, 11, 176, 219, 223, 308, 312.

`(...)`: 152.
`**`: 20, 112, 169.
`+=`: 273.
`--`: 296.
`..`: 112, 169.
`/=`: 20, 112, 169.
`:=`: 20, 112, 169.
`<<`: 20, 112, 169.
`<=`: 20, 112, 169.
`<>`: 20, 112, 169.
`==`: 20, 112, 169.
`=>`: 20, 112, 169.
`>=`: 20, 112, 169.
`>>`: 20, 112, 169.
`$`: 169.
`%`: 169.
`&`: 169, 226.
`\'`: 232.
`\)`: 226.
`\*`: 152.
`\,`: 202, 226.
`\.`: 168–169, 231, 295.
`\:`: 294, 299.
`\=`: 169, 231.
`\?`: 168, 252.
`\[`: 296.
`\␣`: 232.
`\#`: 169, 226, 232.
`\$`: 169, 226, 232.
`\%`: 169, 226, 232.
`\&`: 168–169, 185, 232, 253, 295.
`\\`: 168–169, 232, 253, 295–296.
`\^`: 169, 226, 232.
`\{`: 232.
`\}`: 232.
`\~`: 232.
`\‘`: 232.
`\|`: 253, 295.
`\_`: 155, 169, 226, 228, 232.
`\A`: 279.
`\ANT`: 178.
`\ast`: 226.
`\B`: 226.
`\BL`: 169, 230.
`\bn`: 180.
`\BOX`: 169, 230.
`\C`: 168, 240.
`\ch`: 284.
`\con`: 282.
`\D`: 269.
`\dst`: 169, 230.
`\E`: 169, 226.
`\EL`: 169, 230.
`\F`: 270.
`\fi`: 281.
`\fin`: 282.
`\G`: 169, 230.
`\H`: 233.
`\hbox`: 234.
`\I`: 169, 230.
`\in`: 226, 235.
`\input awebmac`: 144.
`\inx`: 282.
`\J`: 226.
`\K`: 169, 230.
`\KR`: 169, 230.
`\L`: 169, 230.
`\M`: 265.
`\mathbin`: 168, 254.
`\mathop`: 168, 254.
`\mathrel`: 168, 254.
`\N`: 265.
`\notin`: 201.
`\O`: 169, 231.
`\ORE`: 188.
`\ORW`: 188.
`\P`: 256, 268.
`\p`: 168–169, 253, 295.
`\par`: 268.
`\R`: 230, 235.
`\S`: 169, 230, 270, 273.
`\s`: 295.
`\T`: 226.
`\to`: 226.
`\U`: 279.
`\V`: 198, 230.
`\W`: 230, 235.
`\X`: 168, 257.
`\Y`: 256, 263, 268, 273.
`\1`: 168, 255–256.

\2: 168, 255–256.
\3: 168, 255–256.
\4: 168, 255–256.
\5: 168, 255–256.
\6: 168, 255–256, 268.
\7: 168, 255–256, 268.
\8: 168–169, 226, 252.
\9: 295.
^: 169.
@!: 51, 101, 109, 133.
@': 101, 169.
@*: 81, 99, 101, 265.
@+: 99, 101, 169, 273.
@,: 99, 101, 169.
@.: 45, 99, 101, 132.
@/: 99, 101, 169.
@:: 45, 99, 101, 132.
@;: 99, 101, 169.
@<: 99, 101, 169.
@=: 101, 169.
@>: 101.
@?: 101, 109.
@␣: 81, 99, 101, 265.
@#: 99, 101, 169.
@&: 99, 101, 169.
@\: 101, 169.
@^: 45, 99, 101, 132.
@{: 101, 169.
@}: 101, 169.
@~: 101, 169.
@@: 101, 156.
@|: 99, 101, 169.
@a: 99, 101, 132.
@b: 45, 99, 101, 132.
@d: 51, 99, 101.
@f: 51, 99, 101.
@i: 45, 99, 101, 132.
@p: 99, 101, 132.
@r: 99, 101, 132.
@s: 45, 99, 101.
@t: 99, 101, 169.
@x: 87–88.
@y: 87, 90–91.
@z: 87, 97.
a: 152, 248, 251.
abort: 169.
abort: 45, 67, 167.
abs: 169.
abs: 45, 67, 167.
ac: 308*.
accept: 45, 67, 128, 167.
accept: 169.
accept_like: 45, 67, 129, 235.
accept_scrap: 167, 169–170, 176*, 235.
access: 45, 67, 167.
access: 169.
Ada: 308*.
ADA attributes: 18.
ADA text...didn't end: 260.
ADA_parse: 221, 222, *223*, 226, 238–240.
ADA_translate: 237, *238*, 240, 250.
ADA_xref: 128, *129*, 130, 132, 221, 239.
ADDRESS: 69.
AFT: 69.
all: 169.
all: 45, 67, 167.
all_like: 45, 67–68, 235.
Ambiguous prefix: 80.
and: 20, 169.
and: 45, 67, 167.
and_like: 45, 67, 235.
and_scrap: 167, 169–170, 176*, 198, 230, 235.
and_sign: 20, 230.
app: 173, 177–178, 180–186, 188–194, 196, 198–199, 201–202, 204–210, 220, 224, 226, 231–234, 236, 240, 250–251, 268, 273, 299.
app_comment: 221, 224, *236*, 238, 240, 268.
app_tok: 162, 163–165, 173, 231–232, 234, 238, 268.
append_xref: 53, 54–55.
app1: 173, 177, 180–187, 189–190, 192–194, 196, 198, 202, 204–210, 214, 220, 236.
app2: 173, 186, 189, 191, 199, 202.
app3: 173, 191, 193, 198, 202.
Argument: 308*.
Argument_Count: 308*.
array: 169.
array: 45, 67, 167.
array_like: 45, 67, 235.
arrow: 167, 170, 176*, 209, 230.
ASCII: 69.
ASCII code: 16, 99.
ASCII_code: 16, 18, 20, 31–32, 42, 73, 100–101, 103, 107, 139, 148, 157, 163, 251, 285, 288.
ascii_tok: 99, 101, 109, 115, 225.
at: 45, 67, 167.
at: 169.
at_like: 45, 67, 235.
AWEAVE: *308**.
aweave.adb: 308*.
AWEB file ended...: 90.
AWEB file not open: 308*.
aweb_file: 26, 37, 82, 84, 90, 93, 96, 98, 308*.
aweb_input_file_name: 307.

aweb_output.tex: 307.
b: 107, 141, 163, 251.
backup: 168, 169–170, 184, 186, 189–190, 196, 198, 204, 208, 235, 251–252, 269–270, 273.
bal: 106, 107, 130, 162, 163, 240.
banner: 1, 308*.
BASE: 69.
base: 167, 169–170, 176*, 180, 187, 227.
begin: 169.
begin: 45, 67, 167.
begin_ada: 99, 101, 135, 271–272.
begin_comment: 99, 101, 226, 266.
begin_label_scrap: 167, 170, 176*, 230.
begin_like: 45, 67–68, 235.
beginning: 166, 167, 169–170, 176*177, 186, 189–190, 197–198, 235.
big_cancel: 168, 169, 226, 251–252, 256.
big_force: 168, 169, 226, 251, 256, 268.
big_line_break: 99, 101, 226, 266.
blink: 285, 286, 292–294.
body: 45, 67, 128, 167.
body: 169.
body_flag: 51, 54, 109, 115, 128–129, 132, 296.
body_like: 45, 67, 129, 235.
bold_face: 45, 295.
BOOLEAN: 27–28, 30, 32–34, 50, 69, 82, 85, 109, 140–141, 251, 307.
box_sign: 20, 112, 230.
break_out: 145–146, 147, *148*.
break_space: 168, 170, 208, 242, 255–256.
bucket: 285, 286, 291, 293.
buf_size: 14, 31–33.
buffer: 31, 32–33, 36–37, 59, 61–62, 64–66, 85, 87, 89–91, 93, 95–98, 103, 105, 107–109, 111–115, 118–119, 121–124, 127, 143, 157–158, 160–161, 163–164, 219*, 223*, 226, 232, 234, 251, 258, 260–261, 265.
buffer_index: 31, 33, 36, 59, 61, 82, 84.
buffer_type: 31, 84.
byte_index: 42, 43, 61, 76, 80, 251, 287.
byte_mem: 41, 42, 43, 46, 49, 56, 61, 64–65, 76–78, 80, 155, 251, 253, 258–261, 286–287, 293.
byte_memory: 42.
byte_ptr: 43, 65, 77, 309.
byte_start: 41, 42, 43–44, 49, 54, 59, 64–65, 77–78, 109, 131, 155, 253, 258, 286, 293.
c: 33, 61, 76, 80, 101, 103, 105, 107, 111, 148, 152, 157, 160, 163, 212, 214, 285.
CALENDAR: 69.
CALLABLE: 69.
cancel: 168, 169–170, 186, 196, 198, 205, 208, 238, 242, 251, 255–256.
carriage_return: 20, 21, 32.
case: 45, 67, 167.
case: 169.
case_like: 45, 67, 235.
case_scrap: 167, 169–170, 176*, 189, 235.
cat: 171, 174, 176*178, 180–189, 191–194, 196, 198–199, 201–202, 204–210, 212, 214, 216–217, 219*221, 224, 237, 287.
cat_type: 171, 287.
Change file ended...: 88, 90, 97.
Change file entry did not match: 98.
change_buffer: 84, 85–86, 89–90, 96, 98.
change_changing: 83, 90, 93, 97.
change_exists: 50, 126–127, 282.
change_file: 26, 37, 82, 84, 87–88, 90, 93, 97, 308*.
change_input_file_name: 307.
change_limit: 84, 85–86, 89–90, 96, 98.
changed_module: 50, 95, 126–127, 152, 284.
changing: 37, 82, 83–84, 86, 90, 93, 95, 98.
CHARACTER: 17, 19, 69.
character_tok: 99, 109, 123, 225.
check_change: *90*, 96.
clause: 167, 170, 176*, 209.
CLOCK: 69.
close_scrap: 167, 169–170, 202, 220, 226, 235.
COL: 69.
collate: 288, 289–291.
colon: 167, 169–170, 176*, 185, 198, 202, 210, 226, 235.
Command_Line: 308*.
comment: 169.
comment_scrap: 217, 224, 226.
compare: 74, 76, 80.
compress: 112.
compress_end: 189.
confusion: 39.
constant: 45, 67, 167.
constant: 169.
CONSTRAINED: 69.
CONSTRAINT_ERROR: 69.
continue: 87.
Control codes are forbidden...: 121.
Control text didn't end: 121.
control_code: 100, *101*, 105, 109, 115, 161.
CONTROLLED: 69.
copy_comment: 156, 162, *163*, 240.
copy_limbo: 156, *157*, 262.
copy_TeX: 156, 159, *160*, 266.
count: 80, 279, 280.
COUNT: 69.
count_xref: 280.
CREATE: 30, 69.

create_TeX_file: 27, *30*, 308*.
cur_bank: 287, 293.
cur_byte: 287, 293.
cur_depth: 287, 292–293.
cur_end: 243, 245–246, 248.
cur_mod_name: 251, 258–260.
cur_mode: 243, 244–245, 248, 251–252, 255–256.
cur_module: 109, 116, 135, 272, 274.
cur_name: 66, 247–248, 253, 257–258, 285–286, 293–295, 297.
cur_state: 243, 245–246.
cur_tok: 243, 245–246, 248.
cur_val: 287, 296.
cur_xref: 136, 137, 257, 273, 276–280, 296–297, 299.
CURRENT_INPUT: 69.
CURRENT_OUTPUT: 69.
d: 111, 148, 212, 214, 251, 286, 289, 291.
DATA: 70.
DATA_ERROR: 70.
data_structures: *3*,* 4*,* 11*,* 308*.
`data_structures.adb`: 3*.
`data_structures.ads`: 3*.
DATE: 70.
DAY: 70.
DAY_DURATION: 70.
decl: 166, 167, 169–170, 176*, 183–184, 186, 192–193, 198, 202, 204–205, 207, 210, 235.
`declare`: 169.
declare: 12, 45, 67, 167.
declare_like: 45, 67, 235.
decr: 13, 33, 108, 113, 118, 141, 148, 153, 161, 164–165, 246, 293–294.
def_flag: 51, 54–55, 109, 115, 128–129, 132–133, 135, 137, 151, 257, 273, 275, 277–278, 296.
DEFAULT_AFT: 70.
DEFAULT_BASE: 70.
DEFAULT_EXP: 70.
DEFAULT_FORE: 70.
default_name: 27, 28.
DEFAULT_SETTING: 70.
DEFAULT_WIDTH: 70.
defined: 45, 63.
definition: 99, 101, 133, 267.
`delay`: 169.
delay: 45, 67, 167.
DELETE: 70.
DELTA: 70.
`delta`: 169.
delta: 45, 67, 167.
depth: 287, 291–292.
DEVICE: 70.
DEVICE_ERROR: 70.
dig: 150, 152–153.
digit_scrap: 167, 169–170, 176*, 187, 191, 226, 228.
DIGITS: 70.
digits: 45, 67, 167.
`digits`: 169.
DIRECT_IO: 70.
do: 45, 67, 167.
`do`: 169.
do_cases: 174, 175, *176*.
`Done.`: 282.
`Double @ required...`: 158.
`Double @ should be used...`: 232.
double_dot: 99, 112, 226.
double_star: 20, 112, 230.
DURATION: 70.
easy_cases: 221, 225, *226*.
eight_bits: 41, 60–61, 99–101, 104–107, 109–111, 125, 130, 140–141, 159–160, 162–163, 167–168, 171, 212, 214, 240, 247–248, 251, 287, 290–291.
ELABORATE: 70.
ELEMENT_TYPE: 70.
else: 45, 67, 167.
`else`: 169.
else_like: 45, 67, 235.
else_scrap: 167, 169–170, 176*, 188, 198, 208, 230, 235.
`elsif`: 169.
elsif: 45, 67, 167.
elsif_like: 45, 67, 235.
EMAX: 70.
emit_space_if_needed: 263, 267, 272.
`end`: 169.
end: 12, 45, 67, 167.
end_comment: 99, 101, 226, 266.
END_ERROR: 70.
end_field: 243.
end_label_scrap: 167, 170, 181, 230.
end_like: 45, 67, 235.
end_of_AWEAVE: 38, 308*.
END_OF_FILE: 33, 70.
END_OF_LINE: 33, 70.
END_OF_PAGE: 70.
end_translation: 168, 243, 249, 251, 256.
ending: 167, 169–170, 176*, 235.
`entry`: 169.
entry: 45, 67, 128, 167.
ENUM: 70.
ENUMERATION_IO: 70.
EPSILON: 70.
equal: 74, 76–78.
equivalence_sign: 20, 112, 134, 230, 270, 273.

err_print: 35, 76, 80, 83, 87–88, 90–91, 97–98, 101, 111, 114, 118–119, 121–122, 124, 158, 163–164, 232, 238, 266, 269–270, 273–274.
error: 33, 35, *36*, 38, 91.
error_message: 15, 310.
`exception`: 169.
exception: 45, 67, 167.
exception_like: 45, 67, 235.
exception_scrap: 167, 169–170, 176,* 185, 235.
exit: 45, 67, 167.
`exit`: 169.
exp: 167, 169–170, 176,* 226.
EXP: 70.
exponent: 109, 113, 226.
extension: 74, 78.
`Extra @>`: 101.
f: 28, 33, 291.
FALSE: 28, 30, 32–34, 70, 83–85, 90, 93, 98, 109, 111, 126–127, 140, 143, 148, 251–252, 262, 281, 307–308.*
fatal_error: 38, 39–40, 308.*
fatal_message: 15, 310.
FIELD: 70.
FILE: 28, 30, 70.
FILE_MODE: 27–28, 70.
FILE_TYPE: 17, 70.
final_limit: 33.
FINE_DELTA: 70.
finish_ADA: 267, *268*, 272.
finish_line: 142, *143*, 144, 157, 160–161, 256, 262, 268, 279, 281–282, 296, 299.
FIRST: 17, 70.
FIRST_BIT: 70.
first_phase: *7*,* 308.*
`first_phase.adb`: 7.*
`first_phase_.ads`: 7.*
first_xref: 276, 277, 280.
FIXED_IO: 70.
flag: 278, 279, 280.
FLOAT: 70.
FLOAT_IO: 70.
flush_buffer: 140, *141*, 143, 148–149, 262, 281.
follows: 167, 169–170, 177, 183, 193, 198, 205, 235.
footnote: 275, 278, *279*, 299.
`for`: 169.
for: 45, 67, 128, 167.
force: 168, 169–170, 172, 182–184, 186, 192, 194, 196, 198–199, 204, 208, 210, 226, 231, 235, 240, 242, 256, 268, 273.
force_line: 99, 101, 226.
FORE: 70.
FORM: 70.
form_feed: 20, 32.
format: 99, 101, 128–130, 132–133, 221, 223,* 239–240, 266–267.
found: 61, 64, 76, 173–174, 176,*177.
freeze_text: 211, 212, 220, 224, 236, 240, 251.
FROM: 70.
function: 45, 67, 128, 167.
`function`: 169.
`generic`: 169.
generic: 45, 67, 167.
generic_like: 45, 67, 235.
generic_scrap: 167, 169–170, 176,* 192, 235.
GET: 33, 70.
GET_LINE: 70.
get_line: 82, 94, *95*, 103, 105, 107, 111, 118, 142, 157, 160, 163.
get_next: 109, 110, *111*, 125, 129, 132–135, 221, 223,* 227–228, 266, 269–270, 272–274.
get_output: 247, *248*, 249, 251–252, 255–256.
goto: 45, 67, 167.
`goto`: 169.
goto_like: 45, 67–68, 235.
gproc: 167, 170, 176,* 210.
greater: 74, 76, 78, 80.
greater_or_equal: 20, 112, 230.
h: 61, 286.
harmless_message: 15, 310.
hash: 43, 59, 63, 286.
hash_index: 59, 61.
hash_size: 14, 59, 62, 286.
head: 287, 291–294, 308.*
hi_ptr: 171, 216, 219.*
history: 15, 310.
history_type: 15.
`Hmm... n of the preceding...`: 91.
i: 22, 61, 120, 129, 212, 214, 216, 309.
ic: 308.*
id: 66, 67–68.
id_first: 59, 61–62, 64–65, 67, 109, 113–114, 121–124, 231, 233–234.
id_flag: 172, 235, 248, 269–270.
id_loc: 59, 61–62, 64–66, 109, 113–114, 121–124, 231, 234.
id_lookup: 59, 60, *61*, 66, 109, 129, 132, 134, 235, 269–270.
identifier: 109, 113, 128–129, 132, 134, 225, 247–248, 251, 253, 269–270.
if: 45, 67, 167.
`if`: 169.
if_like: 45, 67, 235.
if_scrap: 167, 169–170, 174, 189, 235.
ignore: 99, 101, 226.

ignore_spaces: 251, 252, 256.
ilk: 41, 42, 45–46, 59–60, 63, 65, 129, 131–132, 134, 235, 253, 295.
`Illegal control code...`: 259.
`Illegal use of @...`: 164.
IMAGE: 70.
`Improper ASCII-code sequence...`: 124.
`Improper based number...`: 111.
`Improper format definition`: 270.
`Improper macro definition`: 269.
in: 45, 67, 167.
`in`: 20, 169.
IN_FILE: 70, 308*.
in_like: 45, 67, 235.
in_scrap: 167, 169–170, 174, 185, 201, 226, 235.
`Incompatible module names`: 76.
incr: 13, 33, 53, 62, 64–65, 77–78, 80, 87–88, 90, 96–97, 103, 105, 107–108, 111–115, 118–119, 121–122, 124, 127, 141, 145, 153, 158, 161–164, 173–174, 176*, 211, 216, 224, 231–232, 245, 248, 252, 258–261, 264, 280, 284, 291.
indent: 168, 169–170, 177, 182–183, 192–194, 198, 202, 205, 210, 235, 251.
INDEX: 70.
Index: 308*.
infinity: 290, 291–292.
init_stack: 243, 266–267, 272, 299.
initialize: *3*, 308*.
INLINE: 70.
inner: 242, 243, 248, 252, 256.
inner_tok_flag: 172, 240, 248, 250.
INOUT_FILE: 70.
`Input ended in mid-comment`: 163.
`Input ended in section name`: 118.
`Input line too long`: 33.
`input.ch`: 307.
`input.web`: 307.
input_has_ended: 82, 90, 93, 96, 102–103, 105, 107, 111, 118, 126, 157, 160, 163, 262.
input_ln: 32, *33*, 87–88, 90, 96–97.
input_output: 3*, *4*, 5*11*, 308*.
`input_output.adb`: 4*.
`input_output.ads`: 4*.
int_number_io: 3*4*, 6*11*, 24, 308*.
INTEGER: 16, 19, 31, 41–42, 45, 50–51, 56–57, 59, 70, 73, 82, 90, 129, 139, 150–152, 171, 212, 214, 243, 251, 263, 279, 286–287, 289, 308*.
INTEGER_IO: 70.
IO_EXCEPTIONS: 70.
is: 45, 67, 167.
`is`: 169.
is_like: 45, 67, 235.
IS_OPEN: 28, 30, 33, 70, 93.
italic: 45, 295.
ITEM: 70.
j: 76, 80, 111, 141, 149, 212, 214, 220, 223*, 251.
join: 99, 101, 226, 266.
jump_out: 38.
k: 37, 49, 61, 76, 80, 111, 141, 143, 148, 152, 155, 212, 214, 219*, 251.
k_limit: 251, 258, 260.
k_module: 283, 284.
l: 36, 61, 76, 80, 234.
label: 167, 170, 174, 181, 196.
LARGE: 71.
LAST: 17, 71.
LAST_BIT: 71.
LAYOUT_ERROR: 71.
left_arrow: 20, 112, 230.
left_label_bracket: 20, 112, 230.
length: 43, 63, 253, 295.
LENGTH: 71.
length_index: 139, 141, 148.
less: 74, 76–78, 80.
less_or_equal: 20, 112, 230.
lexical_scanning: *6*, 7*10*.
`lexical_scanning.adb`: 6*.
`lexical_scannning.ads`: 6*.
lhs: 131, 132, 134.
limit: 32–33, 37, 82, 85, 87–89, 91, 93–98, 103, 105, 107, 111–112, 114, 118, 121–122, 143, 157–158, 160–161, 163, 251, 258, 260.
`limited`: 169.
limited: 45, 67, 167.
line: 37, 82, 83, 87–88, 90, 93, 96–98.
LINE: 71.
`Line had to be broken`: 149.
line_break: 99, 101, 226, 266.
line_feed: 20, 32.
line_length: 14, 139–140, 145.
LINE_LENGTH: 71.
lines_dont_match: *85*, 90.
link: 41, 42, 43, 46, 63, 286.
LIST: 71.
llink: 46, 76–77, 80, 137, 299.
lo_ptr: 171, 212, 215–216, 219*220.
loc: 33, 37, 82, 87, 91, 93–95, 97–98, 103, 105, 107–108, 111–115, 118–119, 121–124, 127, 132, 157–158, 160–161, 163–164, 226, 251, 258, 265–266.
long_buf_size: 14, 31, 260–261.
long_buffer_index: 31, 219*, 223*, 251.
long_name_index: 73, 76, 80, 111.
longest_name: 14, 73, 118, 120.

loop: 45, 67, 167.
`loop`: 169.
loop_like: 45, 67, 235.
loop_scrap: 166, 167, 169–170, 174, 189, 198, 235.
LOW_LEVEL_IO: 71.
LOWER_CASE: 71.
m: 54, 77, 152.
MACHINE_EMAX: 71.
MACHINE_EMIN: 71.
MACHINE_MANTISSA: 71.
MACHINE_OVERFLOWS: 71.
MACHINE_RADIX: 71.
MACHINE_ROUNDS: 71.
make_output: 249, 250, *251*, 257, 268, 299.
MANTISSA: 71.
mark_error: 15, 36, 259–260.
mark_fatal: 15, 38.
mark_harmless: 15, 120, 137, 149.
math: 167, 169–170, 173–174, 177–181, 183, 185, 187–189, 191, 193–195, 198–207, 209–210, 219*220, 226, 230–231, 233–235, 269–270, 273.
math_bin: 168, 169, 226, 235, 251, 254.
math_break: 99, 101, 226, 266.
math_op: 168, 251.
math_rel: 168, 169, 172, 226, 251, 254, 273.
max_bytes: 14, 42, 65, 77.
MAX_DIGITS: 71.
MAX_INT: 71.
MAX_MANTISSA: 71.
max_modules: 14, 50–51, 127.
max_names: 14, 42, 65, 77.
max_refs: 14, 51, 53.
max_scr_ptr: 171, 229, 238, 241, 268, 309.
max_scraps: 14, 171, 217, 229, 241, 287.
max_sort_ptr: 287, 291, 309.
max_sorts: 287, 291.
max_stack_ptr: 243, 245, 309.
max_texts: 14, 56, 215, 217, 229, 241.
max_tok_ptr: 57, 215, 229, 241, 250, 268, 309.
max_toks: 14, 57, 162, 215, 217, 220, 229, 231, 240–241.
max_txt_ptr: 57, 215, 229, 241, 250, 268, 309.
MEMORY_SIZE: 71.
mid_xref: 276, 277.
MIN_INT: 71.
`Missing '|'...`: 238.
mitpo: 12.
`mod`: 169.
mod: 45, 67, 167.
mod_check: *137*, 138.
mod_flag: 172, 248, 273–274, 299.
mod_like: 45, 67–68, 235.
mod_lookup: 73–74, 75, *76*, 116–117.
mod_name: 247, 248, 251.
mod_print: 298, *299*, 300.
mod_scrap: 167, 169–170, 174, 186, 273–274.
mod_text: 73, 76–80, 111, 116–120.
mod_xref_switch: 51, 55, 135.
MODE: 28, 30, 71.
MODE_ERROR: 71.
mode_field: 243.
mode_of_file: 27, 28.
module_count: 50, 54–55, 95, 126–127, 262, 264–265, 273, 284.
module_index: 50, 283.
module_name: 99, 101, 109, 115, 132, 135, 266, 272, 274.
MONTH: 71.
MONTH_NUMBER: 71.
n: 54, 90.
NAME: 28, 30, 71.
`Name does not match`: 80.
NAME_ERROR: 28, 30, 71.
name_index: 42, 43, 80.
name_info: 42, 46, 285.
name_pointer: 43, 48–49, 53–55, 60–61, 66, 75–76, 79–80, 109, 129, 131, 137, 154–155, 171, 223*, 251, 271, 285, 298–299.
name_ptr: 43, 49, 61, 63, 65, 77, 309.
NATURAL: 71.
`Never defined: <section name>`: 137.
`Never used: <section name>`: 137.
`new`: 169.
new: 45, 67, 167.
new_like: 45, 67, 235.
NEW_LINE: 24, 71, 141.
new_ln: 24, 37, 91, 149, 308*, 310.
new_mod_xref: 53, *55*, 135.
new_module: 99, 101, 104–105, 111, 160.
NEW_PAGE: 71.
new_scrap: 167, 169–170, 174, 205, 235.
new_xref: 53, *54*, 129, 131–132, 134.
next_control: 125, 128–130, 132–135, 221, 223*, 225–228, 231, 238–240, 250, 266–267, 269–274.
next_name: 285, 286, 293.
next_xref: 276, 277, 280, 296–297.
`No change file`: 308*.
no_line_break: 99, 101, 226, 266.
no_underline: 99, 101, 115, 132.
normal: 45, 60, 63, 128–129, 131–132, 134, 235, 269–270, 295.
`not`: 20, 169.
not: 45, 67, 167.
not_equal: 20, 112, 230.

not_like: 45, 67, 235.
not_scrap: 167, 169–170, 174, 230, 235.
not_sign: 20, 230.
null: 45, 67, 167.
`null`: 169.
null_array: 43, 44, 59.
num: 51, 52–55, 137, 257, 273, 277, 279–280, 296.
NUM: 71.
num_field: 51.
NUMBER_BASE: 71.
NUMERIC_ERROR: 71.
of: 45, 67, 167.
`of`: 169.
of_like: 45, 67, 235.
OFF: 71.
ON: 71.
OPEN: 28, 71.
open_a_file: 27, *28*, 308*.
open_scrap: 166, 167, 169–170, 174, 202, 220, 226, 235.
opened: 307, 308*.
opt: 166, 168, 169–170, 196, 202, 226, 252, 255.
optim: 12.
OPTIMIZE: 71.
or: 45, 67, 167.
`or`: 20, 169.
or_like: 45, 67, 235.
or_sign: 20, 230.
other_line: 82, 83, 93, 98.
others: 45, 67, 167.
`others`: 169.
`out`: 169.
out: 45, 68, 167.
out_buf: 139, 141, 144–146, 148–149, 256, 268, 273.
OUT_FILE: 30, 71, 308*.
out_line: 139, 141, 144, 149, 263.
out_mod: 151, *152*, 257, 265, 280, 284, 296.
out_name: 154, *155*, 253, 295.
out_ptr: 139, 141, 143–145, 148–149, 161, 256, 263, 268, 273.
outdent: 168, 170, 177, 186, 189, 192–194, 202, 205, 208.
outer: 242, 243, 255–256.
outer_parse: 239, *240*, 267, 272.
outer_xref: *130*, 133, 135, 239.
output: *8*,* 9*11*.
`output.adb`: 8*.
`output.ads`: 8*.
output_ADA: 249, *250*, 258, 266.
output_mode: 243, 251.
output_state: 243.
output_tok: 99, 101, 109, 115, 122, 225, 231.
out1: 145, 152–153, 155, 158, 161, 251, 253–258, 265–266, 279–280, 284, 296.
out2: 145, 152, 252, 254–257, 263, 265, 268, 284, 294–296, 299.
out3: 145, 254, 281, 296.
out4: 145, 268, 279–280, 282, 284.
out5: 145, 254.
overflow: 40, 53, 65, 77, 127, 162, 215, 220, 229, 241, 245, 260–261, 291.
Overwrite: 308*.
p: 49, 54, 55, 61, 76, 80, 129, 137, 155, 223*, 238, 240, 245, 268.
PACK: 42, 71.
`package`: 169.
package: 45, 68, 128, 167.
PAGE: 71.
PAGE_LENGTH: 71.
param: 99.
parsing: *9*,* 10*, 176*, 219*, 223*.
`parsing-ada_parse.adb`: 223*.
`parsing-do_cases.adb`: 176*.
`parsing-translate.adb`: 219*.
`parsing.adb`: 9*.
`parsing.ads`: 9*.
pdollar: 166, 168, 169–170, 177, 181, 183, 189, 193–194, 198, 202, 205, 207, 209–210, 220, 226, 231, 233–235, 251–252, 269–270.
per_cent: 140, 141.
period: 167, 169–170, 174, 187, 226.
Phase_I: 301, *302*, 308*.
Phase_II: 303, *304*, 308*.
Phase_III: 305, *306*, 308*.
phase_one: 34, 35, 126.
phase_three: 34, 126, 257, 282.
pid: 66, 69–72.
pop_level: 246, 248.
POS: 18, 71.
POSITION: 71.
POSITIVE: 71.
POSITIVE_COUNT: 71.
pp: 171, 173–174, 176*210, 213, 215–216, 219*.
pragma: 45, 68, 167.
`pragma`: 169.
pragma_like: 45, 68, 235.
PRED: 71.
predefined: 45, 66, 235, 253, 295.
prefix: 74, 78.
prefix_lookup: 79, *80*, 116.
prime_the_change_buffer: *86*, 93, 97.
print: 24, 37, 49, 91, 120, 127, 137, 149, 259–260, 265, 282, 309.

print_id: 48, *49*, 137, 259–260.
print_int: 24, 37, 91, 127, 149, 265, 309.
print_ln: 24, 37, 149, 308*.
print_nl: 24, 33, 35, 38, 120, 137, 149, 259–260, 262, 282, 309–310.
PRIORITY: 71.
`private`: 169.
private: 45, 68, 167.
private_like: 45, 68, 235.
private_scrap: 167, 169–170, 174, 235.
proc: 167, 169–170, 174, 192, 210, 235.
proc_like: 45, 67–68, 129, 235.
`procedure`: 169.
procedure: 45, 68, 128, 167.
prod: 221.
productions, table of: 170.
PROGRAM_ERROR: 71.
pseudo_semi: 99, 101, 226, 266.
psid: 66.
pspace: 168, 169–170, 198, 206, 235, 251–252, 256.
push_level: 244, *245*, 248, 251.
PUT: 24, 71, 141, 144.
PUT_LINE: 71.
q: 54, 55, 61, 76, 80, 240.
r: 55, 80.
`raise`: 169.
raise: 45, 68, 167.
`range`: 169.
range: 45, 68, 167.
RANGE: 71.
range_like: 45, 68, 235.
READ: 71.
RECEIVE_CONTROL: 71.
record: 45, 68, 167.
`record`: 169.
record_like: 45, 68, 235.
record_scrap: 167, 169–170, 189, 198, 210, 235.
recursion: 137, 249, 298.
red: 173, *212*, 214.
reduce: 173, 177–178, 180–194, 196, 198–199, 201–202, 204–210, 212.
`rem`: 169.
rem: 45, 68, 167.
`renames`: 169.
renames: 45, 68, 167.
res_flag: 172, 235, 248.
res_word: 247, 248, 251, 253.
reserved: 45, 54.
RESET: 71, 93.
reset_input: 92, *93*, 126, 262.
restart: 95, 111, 115, 248.
reswitch: 225, 251–252, 256.
return: 45, 68, 167.
`return`: 169.
return_like: 45, 67–68, 235.
return_scrap: 167, 169–170, 174, 235.
`reverse`: 169.
reverse: 45, 68, 167.
rhs: 131, 134.
right_arrow: 20, 112, 230.
right_label_bracket: 20, 112, 230.
rlink: 46, 76–77, 80, 137, 299.
roman: 45, 128, 295.
root: 46, 47, 76, 80, 138, 300.
SAFE_EMAX: 72.
SAFE_LARGE: 72.
SAFE_SMALL: 72.
save_base: 238.
save_limit: 251, 258.
save_line: 263.
save_loc: 251, 258.
save_mode: 251, 256.
save_next_control: 250.
save_place: 263.
save_position: 263, 264, 267.
save_text_ptr: 250.
save_tok_ptr: 250.
scanning_based_number: 109, 111.
scrap_base: 171, 213, 217, 219*220, 238.
scrap_index: 171, 238, 287.
scrap_ptr: 171, 216–217, 221, 224, 229, 236, 238, 241, 268, 270, 287, 299.
sc0: 224, 226, 270.
sc1: 224, 226–228, 235, 273–274.
sc2: 224, 226, 228, 230–231, 233–235, 270, 273.
sc3: 224, 226, 230–231, 235, 269–270, 273.
sc4: 224, 226, 230, 235.
sc5: 224, 226, 235, 269–270.
sc6: 224, 226.
searching: *5*,* 6*7*, 9*10*, 308*.
`searching.adb`: 5*.
`searching.ads`: 5*.
second_phase: *10*,* 11*, 308*.
`second_phase.adb`: 10*.
`second_phase.ads`: 10*.
SECONDS: 72.
`Section name didn't end`: 119.
`Section name too long`: 120.
select: 45, 68, 167.
`select`: 169.
semi: 167, 169–170, 177, 186, 189, 193, 198–199, 204–206, 210, 226, 270, 273.
SEND_CONTROL: 72.
separate: 45, 68, 167.

`separate`: 169.
separate_like: 45, 68, 235.
separate_scrap: 167, 169–170, 174, 198, 205, 235.
SEQUENTIAL_IO: 72.
SET: 72.
SET_COL: 72.
set_element_sign: 20, 226.
SET_INDEX: 72.
SET_INPUT: 72.
SET_LINE: 72.
SET_LINE_LENGTH: 72.
SET_OUTPUT: 72.
SET_PAGE_LENGTH: 72.
SHARED: 72.
sid: 66.
sixteen_bits: 41, 42, 46, 51, 54, 57, 59, 75–76, 79–80, 131, 172, 212, 214, 243, 248, 250, 263, 278–279, 287.
SIZE: 72.
skip_comment: 106, *107*, 130, 156, 162.
skip_limbo: 102, *103*, 126, 156.
SKIP_LINE: 33, 72.
SKIP_PAGE: 72.
skip_TeX: 104, *105*, 132, 156.
slanted: 45, 295.
SMALL: 72.
`Sorry, x capacity exceeded`: 40.
sort_ptr: 287, 291–294.
SPACE: 72.
SPACING: 72.
special string characters: 231.
SPLIT: 72.
split procedures: 174, 221, 301.
spotless: 15, 310.
sq: 173, *214*.
squash: 173, 177, 179, 185, 187, 189, 195, 197–201, 203–206, 210, 214.
sspace: 168, 169–170, 183, 185, 189, 194, 196, 198, 206–207, 235, 251–252, 256.
stack: 243, 245–246.
stack_ptr: 243, 245–246.
stack_size: 14, 243, 245.
STANDARD: 72.
STANDARD_INPUT: 72.
STANDARD_OUTPUT: 24, 72.
stat: 12.
STATUS_ERROR: 28, 30, 72.
stmt: 167, 170, 174, 177, 182–184, 186, 194, 196, 198–199, 206, 208.
STORAGE_ERROR: 72.
STORAGE_SIZE: 72.
STORAGE_UNIT: 72.
str: 109, 114, 225.
STRING: 27–28, 30, 72.
`String constant didn't end`: 114.
string_delimiter: 251, 260.
Strings: 308*.
`subtype`: 169.
subtype: 45, 68, 128, 167.
SUCC: 72.
success: 27, 28.
SUPPRESS: 72.
SYSTEM: 72.
system dependencies: 17, 21, 24–25, 28, 30, 32, 37, 301, 310–311.
SYSTEM_NAME: 72.
s0: 224.
t: 61.
tab_mark: 20, 37, 101, 103, 108, 111, 118–119, 143, 158, 161.
task: 45, 68, 128, 167.
`task`: 169.
TASKING_ERROR: 72.
tats: 12.
temp_ilk: 131, 132.
temp_line: 82, 83.
`terminate`: 169.
terminate: 45, 68, 167.
TERMINATED: 72.
terminator: 167, 170, 186, 189, 208.
`TeX file not open`: 308*.
`TeX string should be...`: 266.
TeX_file: 29, 30, 141, 144, 308*.
TeX_file_name: 27, 30.
TeX_output_file_name: 307, 308*.
TeX_string: 99, 101, 109, 115, 225, 266.
text_char: 17, 18–19, 24, 33.
text_file: 17, 26–29, 32–33.
TEXT_IO: 3*, 11*, 17, 24, 27–28, 30, 33, 72, 93, 141, 144, 308*.
text_pointer: 56, 57, 171, 217–219*, 237–238, 240, 244–245, 250, 268.
text_ptr: 57, 211–212, 215, 217, 220, 224, 229, 236, 240–241, 250–251, 268, 299.
then: 45, 68, 167.
`then`: 169.
then_like: 45, 68, 235.
then_scrap: 167, 169–170, 178, 194, 235.
thin_space: 99, 101, 226, 266.
third_phase: *11*, 308*.
`third_phase.adb`: 11*.
`third_phase.ads`: 11*.
`This can't happen`: 39.
this_module: 271, 272–273, 275, 277.

this_xref: 276, 277, 296–297.
TICK: 72.
TIME: 72.
TIME_ERROR: 72.
TIME_OF: 72.
TO: 72.
tok_field: 243.
tok_flag: 172, 173, 236, 240, 248, 268.
tok_mem: 57, 162, 172, 173, 243, 248, 257.
tok_ptr: 57, 162, 164, 173, 211, 215, 217, 220, 229, 231, 240–241, 250, 268, 299.
tok_start: 56, 57, 58, 171, 211, 245.
token_index: 57.
trans: 171, 173, 212, 216–217, 219,* 221, 224, 236–237, 287.
trans_type: 171, 287.
translate: 174, 217, 218, *219*,* 238, 268.
TRUE: 28, 30, 32–34, 72, 82–83, 85, 90, 93, 95–96, 98, 111, 126–127, 140, 148–149, 282.
type: 45, 68, 128, 167.
`type`: 169.
type_like: 45, 67–68, 129, 235.
TYPE_SET: 72.
typewriter: 45, 128, 295.
UNBOUNDED: 72.
Unbounded: 308.*
unbucket: 290, *291*, 292–293.
UNCHECKED_CONVERSION: 72.
UNCHECKED_DEALLOCATION: 72.
underline: 99, 101, 115, 132.
`Unknown control code`: 101.
update_terminal: 25, 36, 127, 265.
UPPER_CASE: 72.
`use`: 169.
use: 45, 68, 167.
USE_ERROR: 28, 30, 72.
Ustring: 308.*
Ustrings: 308.*
VAL: 18, 72.
VALUE: 72.
verbatim: 99, 101, 115, 122, 225, 231.
`Verbatim string didn't end`: 122.
w: 49, 61, 76, 80, 155, 251.
w_range: 42, 43, 49, 61, 76, 80, 155, 251, 287.
when: 45, 68, 167.
`when`: 169.
when_like: 45, 68, 235.
when_scrap: 167, 169–170, 174, 188, 198, 235.
`Where is the match...`: 87, 91, 97.
`while`: 169.
while: 45, 68, 167.
WIDTH: 72.
wildcard: 45, 128, 295.
with: 45, 68, 167.
`with`: 169.
with_like: 45, 68, 235.
with_scrap: 167, 169–170, 174, 186, 198, 210, 235.
WRITE: 72.
ww: 14, 42, 43–44, 49, 54, 64–65, 77–78, 155, 253, 258, 286, 293, 309.
xchr: 18, 19, 21–22, 37, 49, 120, 141, 149.
xlink: 51, 54–55, 137, 257, 277, 280, 296–297.
xlink_field: 51.
xmem: 51.
xmem_field: 51.
`xor`: 169.
xor: 45, 68, 167.
xord: 18, 22, 33.
xref: 41, 42, 51–52, 54–55, 65, 137, 257, 273, 277, 286, 297.
xref_bold_face: 99, 101.
xref_italic: 99, 101.
xref_number: 51, 54–55, 136, 276.
xref_predefined: 99, 101, 129, 132.
xref_ptr: 51, 53–55, 309.
xref_reserved: 99, 101, 129, 132, 226, 266.
xref_roman: 99, 101, 109, 115, 128, 132, 226, 266.
xref_slanted: 99, 101.
xref_switch: 51, 54, 109, 115–116, 128–129, 132–133.
xref_typewriter: 99, 101, 128.
xref_wildcard: 99, 101, 128.
YEAR: 72.
YEAR_NUMBER: 72.
`You can't do that...`: 266, 274.
`You need an = sign...`: 273.
yr_output_file: 308.*

⟨ Append a string scrap 231 ⟩ Used in section 225.
⟨ Append a TEX string scrap 234 ⟩ Used in section 225.
⟨ Append an ASCII-token scrap 233 ⟩ Used in section 225.
⟨ Append an identifier scrap 235 ⟩ Used in section 225.
⟨ Append *digit_scrap* scraps until a '#' occurs 228 ⟩ Used in section 227(2).
⟨ Append the character appropriate to *buffer*(*j*) 232 ⟩ Used in sections 231 and 233.
⟨ Append the scrap appropriate to *next_control* 225 ⟩ Used in section 223*.
⟨ Cases for *accept_scrap* 177 ⟩ Used in section 176*.
⟨ Cases for *and_scrap* 178 ⟩ Used in section 176*.
⟨ Cases for *arrow* 179 ⟩ Used in section 176*.
⟨ Cases for *base* 180 ⟩ Used in section 176*.
⟨ Cases for *begin_label_scrap* 181 ⟩ Used in section 176*.
⟨ Cases for *beginning* 182 ⟩ Used in section 176*.
⟨ Cases for *case_scrap* 183 ⟩ Used in section 176*.
⟨ Cases for *clause* 184 ⟩ Used in section 176*.
⟨ Cases for *colon* 185 ⟩ Used in section 176*.
⟨ Cases for *decl* 186 ⟩ Used in section 176*.
⟨ Cases for *digit_scrap* 187 ⟩ Used in section 176*.
⟨ Cases for *else_scrap* 188 ⟩ Used in section 176*.
⟨ Cases for *ending* 189 ⟩ Used in section 176*.
⟨ Cases for *exception_scrap* 190 ⟩ Used in section 176*.
⟨ Cases for *exp* 191 ⟩ Used in section 176*.
⟨ Cases for *generic_scrap* 192 ⟩ Used in section 176*.
⟨ Cases for *gproc* 193 ⟩ Used in section 176*.
⟨ Cases for *if_scrap* 194 ⟩ Used in section 174.
⟨ Cases for *in_scrap* 195 ⟩ Used in section 174.
⟨ Cases for *label* 196 ⟩ Used in section 174.
⟨ Cases for *loop_scrap* 197 ⟩ Used in section 174.
⟨ Cases for *math* 198 ⟩ Used in section 174.
⟨ Cases for *mod_scrap* 199 ⟩ Used in section 174.
⟨ Cases for *new_scrap* 200 ⟩ Used in section 174.
⟨ Cases for *not_scrap* 201 ⟩ Used in section 174.
⟨ Cases for *open_scrap* 202 ⟩ Used in section 174.
⟨ Cases for *period* 203 ⟩ Used in section 174.
⟨ Cases for *private_scrap* 204 ⟩ Used in section 174.
⟨ Cases for *proc* 205 ⟩ Used in section 174.
⟨ Cases for *return_scrap* 206 ⟩ Used in section 174.
⟨ Cases for *separate_scrap* 207 ⟩ Used in section 174.
⟨ Cases for *stmt* 208 ⟩ Used in section 174.
⟨ Cases for *when_scrap* 209 ⟩ Used in section 174.
⟨ Cases for *with_scrap* 210 ⟩ Used in section 174.
⟨ Cases involving nonstandard ASCII characters 230 ⟩ Used in section 226.
⟨ Change *pp* to max(*scrap_base*, *pp*+*d*) 213 ⟩ Used in sections 212 and 214.
⟨ Check for overlong name 120 ⟩ Used in section 118.
⟨ Check that all changes have been read 98 ⟩ Used in section 308*.
⟨ Check that = or ≡ follows this module name, and emit the scraps to start the module definition 273 ⟩ Used in section 272.
⟨ Clear *b* and **return** 0 165 ⟩ Used in sections 163 and 164.
⟨ Combine the irreducible scraps that remain 220 ⟩ Used in section 219*.
⟨ Compare name *p* with current identifier, **goto** *found* if equal 64 ⟩ Used in section 63.
⟨ Compress two-symbol combinations like ':=' 112 ⟩ Used in section 111.
⟨ Compute the hash code *h* 62 ⟩ Used in section 61.

⟨ Compute the name location p 63 ⟩ Used in section 61.
⟨ Copy a control code into the buffer 261 ⟩ Used in section 260.
⟨ Copy special things when $c = [@], [\backslash], [\{], [\}]$; **return** at end 164 ⟩ Used in section 163.
⟨ Copy the ADA text into *buffer*(*limit* + 1 . . *j*) 260 ⟩ Used in section 258.
⟨ Copy up to '|' or control code, **return** c if finished 161 ⟩ Used in section 160.
⟨ Copy up to control code, **return** if finished 158 ⟩ Used in section 157.
⟨ Count the *pdollar* and space tokens; output the appropriate control sequences and **goto** *reswitch* 252 ⟩ Used in section 251.
⟨ Declaration of subprocedure for *translate* 175 ⟩ Used in section 218.
⟨ Declaration of the *app_comment* procedure 236 ⟩ Used in section 222.
⟨ Declaration of the *easy_cases* procedure 226 ⟩ Used in section 222.
⟨ Do special things when $c = [@], [\backslash], [\{], [\}]$; **return** at end 108 ⟩ Used in section 107.
⟨ Do the first pass of sorting 286 ⟩ Used in section 282.
⟨ Emit the scrap for a module name if present 274 ⟩ Used in section 272.
⟨ Enter a new module name into the tree 77 ⟩ Used in section 76.
⟨ Enter a new name into the table at position p 65 ⟩ Used in section 61.
⟨ Get a based number 227 ⟩ Used in section 226.
⟨ Get a character token 123 ⟩ Used in section 111.
⟨ Get a string 114 ⟩ Used in section 111.
⟨ Get an identifier 113 ⟩ Used in section 111.
⟨ Get control code and possible module name 115 ⟩ Used in section 111.
⟨ If end of name, **exit** 119 ⟩ Used in section 118.
⟨ If the current line starts with `@y`, report any discrepancies and **return** 91 ⟩ Used in section 90.
⟨ Invert the cross-reference list at *cur_name*, making *cur_xref* the head 297 ⟩ Used in section 296.
⟨ Look ahead for strongest line break, **goto** *reswitch* 256 ⟩ Used in section 255.
⟨ Make sure that there is room for at least four more scraps, six more tokens, and four more texts 229 ⟩ Used in section 225.
⟨ Make sure that there is room for at least seven more tokens, three more texts, and one more scrap 241 ⟩ Used in section 240.
⟨ Make sure the entries *cat*(*pp* . . *pp* + 4) are defined 216 ⟩ Used in section 215.
⟨ Match a production at *pp*, or increase *pp* if there is no match 174 ⟩ Used in section 215.
⟨ Move *buffer* and *limit* to *change_buffer* and *change_limit* 89 ⟩ Used in sections 86 and 90.
⟨ Output a control, look ahead in case of line breaks, possibly **goto** *reswitch* 255 ⟩ Used in section 251.
⟨ Output a `\math` operator 254 ⟩ Used in section 251.
⟨ Output a module name 257 ⟩ Used in section 251.
⟨ Output all the module names 300 ⟩ Used in section 282.
⟨ Output all the module numbers on the reference list *cur_xref* 280 ⟩ Used in section 279.
⟨ Output an identifier 253 ⟩ Used in section 251.
⟨ Output index entries for the list at *sort_ptr* 294 ⟩ Used in section 292.
⟨ Output the code for the beginning of a new module 265 ⟩ Used in section 264.
⟨ Output the code for the end of a module 281 ⟩ Used in section 264.
⟨ Output the cross-references at *cur_name* 296 ⟩ Used in section 294.
⟨ Output the digits of a 153 ⟩ Used in section 152(2).
⟨ Output the name at *cur_name* 295 ⟩ Used in section 294.
⟨ Output the text of the module name 258 ⟩ Used in section 257.
⟨ Phase I: Read all the user's text and store the cross references 126 ⟩ Used in section 302.
⟨ Phase II: Read all the text again and translate it to TEX form 262 ⟩ Used in section 304.
⟨ Phase III: Output the cross-reference index 282 ⟩ Used in section 306.
⟨ Print error location based on input buffer 37 ⟩ Used in section 36.
⟨ Print error messages about unused or undefined module names 138 ⟩ Used in section 126.
⟨ Print statistics about memory usage 309 ⟩ Used in section 308*.
⟨ Print the job *history* 310 ⟩ Used in section 308*.

⟨ Print warning message, break the line, **return** 149 ⟩ Used in section 148.
⟨ Procedures in *data_structures* 49, 54, 55 ⟩ Used in section 3*.
⟨ Procedures in *first_phase* 129, 130, 137, 302 ⟩ Used in section 7*.
⟨ Procedures in *input_output* 28, 30, 33, 36 ⟩ Used in section 4*.
⟨ Procedures in *lexical_scanning* 85, 86, 90, 93, 95, 101, 103, 105, 107, 111 ⟩ Used in section 6*.
⟨ Procedures in *output* 141, 143, 148, 152, 155, 157, 160, 163 ⟩ Used in section 8*.
⟨ Procedures in *parsing* 212, 214, 218, 222, 238, 240 ⟩ Used in section 9*.
⟨ Procedures in *searching* 61, 76, 80 ⟩ Used in section 5*.
⟨ Procedures in *second_phase* 245, 248, 250, 251, 268, 279, 304 ⟩ Used in section 10*.
⟨ Procedures in *third_phase* 291, 299, 306 ⟩ Used in section 11*.
⟨ Process a format definition 134 ⟩ Used in section 133.
⟨ Put module name into *mod_text*(1 . . *k*) 118 ⟩ Used in section 116.
⟨ Read from *aweb_file* and maybe turn on *changing* 96 ⟩ Used in section 95.
⟨ Read from *change_file* and maybe turn off *changing* 97 ⟩ Used in section 95.
⟨ Rearrange the list pointed to by *cur_xref* 277 ⟩ Used in section 275.
⟨ Reduce the scraps using the productions until no more rules apply 215 ⟩ Used in section 219*.
⟨ Scan a verbatim or output string 122 ⟩ Used in section 115.
⟨ Scan an ASCII token 124 ⟩ Used in section 115.
⟨ Scan the module name and make *cur_module* point to it 116 ⟩ Used in section 115.
⟨ Scan to the next `@>` 121 ⟩ Used in section 115.
⟨ Set initial values 19, 21, 22, 44, 47, 52, 58, 117, 144, 146, 289 ⟩ Used in section 3*.
⟨ Set variable *c* to the result of comparing the given name to name *p* 78 ⟩ Used in sections 76 and 80.
⟨ Show cross references to this module 275 ⟩ Used in section 264.
⟨ Skip next character, give error if not '`@`' 259 ⟩ Used in section 258.
⟨ Skip over comment lines in the change file; **return** if end of file 87 ⟩ Used in section 86.
⟨ Skip to the next nonblank line; **return** if end of file 88 ⟩ Used in section 86.
⟨ Sort and output the index 292 ⟩ Used in section 282.
⟨ Specification of procedures visible from *data_structures* 48, 53 ⟩ Used in section 3*.
⟨ Specification of procedures visible from *first_phase* 128, 301 ⟩ Used in section 7*.
⟨ Specification of procedures visible from *input_output* 27, 32, 35 ⟩ Used in section 4*.
⟨ Specification of procedures visible from *lexical_scanning* 92, 94, 100, 102, 104, 106, 110 ⟩ Used in section 6*.
⟨ Specification of procedures visible from *output* 140, 142, 147, 151, 154, 156, 159, 162 ⟩ Used in section 8*.
⟨ Specification of procedures visible from *parsing* 217, 221, 237, 239 ⟩ Used in section 9*.
⟨ Specification of procedures visible from *searching* 60, 75, 79 ⟩ Used in section 5*.
⟨ Specification of procedures visible from *second_phase* 244, 249, 278, 303 ⟩ Used in section 10*.
⟨ Specification of procedures visible from *third_phase* 298, 305 ⟩ Used in section 11*.
⟨ Split the list at *sort_ptr* into further lists 293 ⟩ Used in section 292.
⟨ Start a format definition 270 ⟩ Used in section 267.
⟨ Start a macro definition 269 ⟩ Used in section 267.
⟨ Store all the predefined identifiers 69, 70, 71, 72 ⟩ Used in section 308*.
⟨ Store all the reserved words 67, 68 ⟩ Used in section 308*.
⟨ Store cross reference data for the current module 127 ⟩ Used in section 126.
⟨ Store cross references in the ADA part of a module 135 ⟩ Used in section 127.
⟨ Store cross references in the definition part of a module 133 ⟩ Used in section 127.
⟨ Store cross references in the TEX part of a module 132 ⟩ Used in section 127.
⟨ Tell about changed modules 284 ⟩ Used in section 282.
⟨ Translate the ADA part of the current module 272 ⟩ Used in section 264.
⟨ Translate the current module 264 ⟩ Used in section 262.
⟨ Translate the definition part of the current module 267 ⟩ Used in section 264.
⟨ Translate the TEX part of the current module 266 ⟩ Used in section 264.
⟨ Types and objects visible from *data_structures* 14, 15, 16, 17, 18, 20, 26, 29, 31, 34, 38, 41, 42, 43, 45, 46, 50, 51, 56, 57, 59, 66, 73, 74, 82, 84, 99, 109, 125, 131, 136, 139, 150, 167, 168, 171, 172, 243, 247, 263, 271, 276, 283, 285, 287, 288,

290, 307 ⟩ Used in section 3*.

The AWEAVE processor

(based on WEAVE Version 2.8)
[ADA-Version 1.0]

	Section	Page
Introduction	1	14
The character set	16	19
Input and output	23	22
Reporting errors to the user	34	25
Data structures	41	27
Searching for identifiers	59	34
Initializing the table of reserved and predefined words	66	36
Searching for module names	73	45
Lexical scanning	81	48
Inputting the next token	109	58
Phase one processing	125	66
Low-level output routines	139	71
Routines that copy TEX material	156	75
Parsing	166	78
Implementing the productions	171	87
Initializing the scraps	221	102
Output of tokens	242	110
Phase two processing	262	119
Phase three processing	282	126
The main program	301	133
System-dependent changes	311	136
Index	312	137